

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
« _____ » _____ 202 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА МЕТОДИКИ ДЛЯ СУММАРИЗАЦИИ ДИАЛОГОВ НА
ОСНОВЕ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ В УСЛОВИЯХ ДЕФИЦИТА
РАЗМЕЧЕННЫХ ДАННЫХ

по направлению подготовки 09.03.03 Прикладная информатика

Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникационные технологии

Выполнил

студент гр. з5130903/00301

Д.Е. Пенясов

Руководитель,

доцент ВШ ПИ, к.т.н.

К.А. Туральчук

Консультант

по нормоконтролю

В.А. Пархоменко

Санкт-Петербург - 2025

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Пенясову Дмитрию Евгеньевичу гр. з5130903/00301

1. Тема работы: Разработка методики для суммаризации диалогов на основе большой языковой модели в условиях дефицита размеченных данных
2. Срок сдачи студентом законченной работы: 11.01.2025.
3. Исходные данные по работе: энкодер для русского языка от компании VK [3.1], LLM модель Vikhr-Llama [3.3], а также семейства моделей *Mistral*, *Llama*, *OpenAI GPT-3.5*; наборы данных для обучения и оценки моделей диалоговой суммаризации [3.2; 3.4].
 - 3.1. Universal Sentence Encoder for Russian (USER) — это модель преобразователь предложений в эмбеддинги исключительно для русского языка . — URL: <https://huggingface.co/deepvk/USER-bge-m3> (дата обращения: 17.11.2024).
 - 3.2. Русскоязычный набор диалогов и аннотаций. — URL: https://huggingface.co/datasets/bragovo/dsum_ru (дата обращения: 17.11.2024).
 - 3.3. *Nikolich A., Korolev K., Shelmanov A. Vikhr: The Family of Open-Source Instruction-Tuned Large Language Models for Russian // arXiv preprint arXiv:2405.13929. — 2024.*
 - 3.4. SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization. — URL: <https://huggingface.co/datasets/Samsung/samsum> (visited on 17.11.2024).
4. Содержание работы (перечень подлежащих разработке вопросов):

- 4.1. Обзор литературы в области суммаризации текстовых данных.
- 4.2. Анализ современных подходов к диалоговой суммаризации с использованием больших языковых моделей.
- 4.3. Разработка методики автоматической аннотации диалогов с учётом ограниченного объёма размеченных данных.
- 4.4. Создание обучающих наборов данных с использованием предложенного подхода.
- 4.5. Сравнение качества обучающих наборов данных: эталонных и созданных с помощью разрабатываемого инструмента.
- 4.6. Анализ влияния методов выбора демонстрационных примеров на качество результатов.
- 4.7. Сравнение производительности моделей, обученных на эталонных и сгенерированных данных, в задачах диалоговой суммаризации.
5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Графические схемы архитектуры нового подхода и алгоритмов работы.
 - 5.2. Результаты сравнения качества моделей и данных по основным метрикам (ROUGE, BERTScore, GPT-based metrics).
6. Консультанты по работе:
 - 6.1. Ст. преподаватель ВШ ПИ, В.А. Пархоменко (нормоконтроль).
7. Дата выдачи задания: 01.10.2024.

Руководитель ВКР _____ К.А. Туральчук

Задание принял к исполнению 01.10.2024

Студент _____ Д.Е. Пенясов

РЕФЕРАТ

На 104 с., 15 рисунков, 8 таблиц, 2 приложения

КЛЮЧЕВЫЕ СЛОВА: СУММАРИЗАЦИЯ ДИАЛОГОВ, БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ, СИНТЕТИЧЕСКИЕ ДАННЫЕ, ДЕФИЦИТ РАЗМЕЧЕННЫХ ДАННЫХ.

Тема выпускной квалификационной работы: «Разработка методики для суммаризации диалогов на основе большой языковой модели в условиях дефицита размеченных данных».

В данной работе объектом исследования выступает процесс автоматической аннотации диалогов на базе LLM. Предмет исследования — методы контекстного обучения и методы RAG для отбора демонстрационных примеров, повышающих качество суммаризации. Цель — разработка методики и фреймворка, позволяющего автоматически формировать обучающие наборы и аннотации с минимальными затратами на ручную разметку, а также оценка его эффективности относительно альтернативных решений. Методологическая основа работы — применение методов машинного обучения, контекстного обучения и механизмов RAG. В ходе исследования проанализированы возможности совместной интеграции контекстного обучения и RAG, проведён сравнительный анализ стратегий отбора демонстраций и реализованы эксперименты на англоязычных, русскоязычных и медицинских диалогах с использованием метрик ROUGE и BERTScore. Результаты подтвердили существенное улучшение качества резюмирования, особенно в узкоспециализированных сценариях, и показали, что предложенный подход может обходиться без масштабной ручной аннотации при формировании обучающих выборок. Разработанный подход применим для автоматической разметки диалогов в колл-центрах, медицинских и иных сервисах, облегчая аннотацию и повышая точность итоговых текстов. Выводы подтверждают, что использование RAG и контекстного обучения LLM повышает качество суммаризаций и упрощает формирование обучающих наборов. Предложенная методика и фреймворк могут быть внедрены в широкий спектр приложений, требующих точной и экономичной диалоговой суммаризации.

ABSTRACT

104 pages, 15 figures, 8 tables, 2 appendices

KEYWORDS: DIALOGUE SUMMARIZATION, LARGE LANGUAGE MODELS, SYNTHETIC DATA, LIMITED LABELED DATA.

The subject of the graduate qualification work is «Developing a Method for Dialogue Summarization Using a LLM with Limited Labeled Data».

In this paper, the object of the study is the process of automatic annotation of dialogues based on LLM. The subject of the study is contextual learning methods and RAG methods for selecting demonstration examples that improve the quality of summarization. The goal is to develop a methodology and framework that allows for automatic formation of training sets and annotations with minimal costs for manual labeling, as well as to evaluate its effectiveness relative to alternative solutions. The methodological basis of the work is the use of machine learning methods, contextual learning and RAG mechanisms. During the study, the possibilities of joint integration of contextual learning and RAG were analyzed, a comparative analysis of demonstration selection strategies was carried out, and experiments were carried out on English-language, Russian-language and medical dialogues using the ROUGE and BERTScore metrics. The results confirmed a significant improvement in the quality of summarization, especially in highly specialized scenarios, and showed that the proposed approach can do without large-scale manual annotation when forming training samples. The developed approach is applicable for automatic dialog labeling in call centers, medical and other services, facilitating annotation and increasing the accuracy of the resulting texts. The findings confirm that the use of RAG and contextual LLM learning improves the quality of summaries and simplifies the formation of training sets. The proposed methodology and framework can be implemented in a wide range of applications requiring accurate and cost-effective dialog summaries.

СОДЕРЖАНИЕ

Введение	8
Глава 1. Обзор подходов к построению аннотаций с помощью LLM.....	12
1.1. Метод контекстного обучения ICL	12
1.2. Построение аннотаций с помощью LLM-моделей.....	14
1.3. Размеры LLM модели	15
1.4. Сравнительный анализ аннотаций LLM-моделей	16
1.5. Аннотации моделей «трансформер»	17
1.6. Дообучение моделей «трансформер» с помощью аннотаций LLM.....	18
1.7. Эффективность подходов Few-shot и Zero-shot	19
1.8. Важность отбора демонстрационных примеров для подхода Few-shot.	20
1.9. Техника Instruction Tuning для итоговой модели суммаризации	21
1.10. Выводы	22
Глава 2. Проектирование и разработка методики и фреймворка для суммаризации.....	25
2.1. Постановка задачи	25
2.2. Формализация методики	27
2.3. Алгоритм предлагаемого фреймворка	30
2.3.1. Дополнительный сценарий: только неразмеченные данные	35
2.4. Архитектура фреймворка с использованием IDEF0	36
2.5. Потенциальные клиенты и пользователи	40
2.5.1. Компании и разработчики виртуальных ассистентов	40
2.5.2. Колл-центры и службы поддержки клиентов	41
2.5.3. Аналитические платформы.....	42
2.5.4. Научно-исследовательские организации и лаборатории NLP	42
2.5.5. Пути внедрения фреймворка.....	43
2.6. Выводы	47
Глава 3. Практическая реализация.....	47
3.1. Общая структура и архитектура фреймворка.....	48
3.2. Подготовка и предобработка данных	53
3.3. Реализация ключевых модулей фреймворка	57
3.3.1. Генерация эмбеддингов	57
3.3.2. Модуль поиска демонстрационных примеров.....	59
3.3.3. Генерация синтетических аннотаций	62
3.3.4. Модуль обновления базы данных	63

3.3.5. Модуль оценки и контроля качества результатов	66
3.4. Структура и организация кода проекта.....	69
3.5. Выводы	71
Глава 4. Экспериментальный анализ методики.....	72
4.1. Цели и задачи экспериментов	73
4.2. Метрики оценивания разработанной методики.	75
4.3. Эксперимент 1: Оценка влияния стратегии выбора демонстрационных примеров на качество суммаризаций	76
4.3.1. Оценка методики на примере общих диалогов	78
4.3.2. Оценка методики на специализированных диалогах	82
4.3.3. Оптимизированная оценка методики.	90
4.4. Эксперимент 2: Instruction Tuning на эталонных и синтетических данных	94
4.5. Выводы	97
Заключение	99
Список сокращений и условных обозначений	101
Словарь терминов.....	102
Список использованных источников.....	103
Приложение 1. Основные модули фреймворка.....	105
Приложение 2. Содержимое /scripts	145

ВВЕДЕНИЕ

Суммаризация диалогов — одна из ключевых и одновременно сложных задач в области обработки естественного языка (NLP). Она требует от моделей способности улавливать разговорные нюансы, учитывать разнообразие стилей общения и сохранять фактическую точность при формулировке кратких, содержательных резюме. Однако обучение языковых моделей для выполнения этой задачи сталкивается с рядом значительных проблем, включая ограниченность и высокую стоимость создания обучающих наборов данных.

На сегодняшний день большая часть доступных наборов данных для обучения моделей суммаризации ориентирована на задачи обработки новостных статей и документов общего типа (например, CNNDM), тогда как для задач диалоговой суммаризации такие ресурсы встречаются крайне редко. Этот дефицит обучающих наборов данных существенно ограничивает возможности разработки и внедрения алгоритмов, способных эффективно извлекать ключевую информацию из диалогов. Современные большие языковые модели (LLM), такие как GPT-4, демонстрируют впечатляющие результаты в различных задачах NLP, включая суммаризацию текстов, даже без необходимости в использовании специализированных наборов данных для предобучения. Тем не менее, использование такого рода сервисов связано с рядом вызовов и сложностей:

- обработка большого объема данных с использованием крупных моделей, таких как GPT-4, может обходиться дорого. Это создает проблему не только для небольших компаний и исследовательских групп, но и для крупных корпораций;
- закрытый характер подобных моделей предполагает, что данные, отправляемые на серверы сторонних компаний, могут быть использованы для целей, не предусмотренных их создателями. Это повышает риск утечки конфиденциальной информации, включая персональные данные клиентов и корпоративные секреты;
- скорость работы моделей в задачах суммаризации играет критически важную роль. В высоконагруженных приложениях интеграция таких моделей может быть нецелесообразной из-за увеличенного времени отклика;
- зависимость бизнес-процессов от стабильной работы компаний, предоставляющих доступ к закрытым LLM, влечет дополнительные риски,

связанные с возможными сбоями или изменением условий предоставления услуг.

Наибольший интерес к созданию собственных обучающих наборов данных может возникать из необходимости иметь индивидуально настроенную модель для выполнения специфических задач суммаризации. Например, такая модель может быть разработана для выделения узкоспециализированной информации, которую стандартные модели часто упускают. Это может включать, например, определение точек конфликта в диалогах, идентификацию критически важных технических деталей из совещаний или извлечение юридически значимой информации из текстов. Для достижения максимальной эффективности в резюмировании диалогов требуются специализированные обучающие наборы данных. Однако их создание вручную связано с большими временными и финансовыми затратами, а также сложностью в обеспечении качества и полноты данных. Эта проблема становится особенно актуальной для задач, где точность критически важна, а использование стандартных обучающих наборов данных оказывается недостаточным. *Решением этой проблемы мог бы стать подход, позволяющий на основе всего нескольких примеров объяснить LLM все тонкости создаваемого набора данных.* Практической реализации именно такого метода и посвящена текущая работа. Более того, предлагаемый фреймворк для создания обучающих наборов данных может быть адаптирован и применен в других областях машинного обучения, расширяя возможности разработки индивидуализированных решений.

Проблема отсутствия обучающих наборов данных актуальна как для научного сообщества, занимающегося NLP и машинным обучением, так и для различных прикладных сфер, включая разработку голосовых помощников, автоматизацию колл-центров, систем поддержки пользователей и бизнес-аналитику.

Теоретическая значимость этой работы заключается в разработке методов обучения моделей в условиях наличия всего лишь небольшого количества эталонных примеров выполненной задачи. Использование настоящего фреймворка позволяет эффективно адаптировать различные модели к новым задачам, минимизируя необходимость в масштабной ручной разметке данных, что, в свою очередь, способствует созданию более универсальных и адаптивных моделей суммаризации, способных успешно работать в различных контекстах и условиях.

На данный момент существует несколько подходов к повышению качества ответов, генерируемых моделями, особенно в ситуациях, когда объем обучающих данных ограничен. Среди них выделяются методы transfer learning, использование

больших моделей в режимах обучения few-shot и zero-shot, а также подходы, основанные на создании синтетических обучающих данных с помощью самих моделей, например, с использованием LLM для генерации аннотаций. Эти методы могут работать синергично: transfer learning позволяет переносить знания, накопленные в смежных задачах, few-shot и zero-shot режимы дают возможность моделям эффективно адаптироваться к новым задачам с использованием минимального числа примеров, а генерация синтетических аннотаций помогает значительно расширить объем доступных обучающих данных. Такой комбинированный подход способен компенсировать недостаток размеченных обучающих данных и повысить гибкость моделей.

Техники few-shot и zero-shot обучения продемонстрировали значительный потенциал в решении различных задач [9]. Однако их эффективность напрямую зависит от качества предоставленных демонстрационных примеров. Исследования показывают, что тщательно подобранные примеры могут существенно улучшить производительность моделей, повышая как точность, так и релевантность генерируемых ответов.

Перспективным направлением в этой области является подход Retrieval-Augmented Generation (RAG), который объединяет информационный поиск с генерацией ответов на основе найденных данных. Этот метод позволяет моделям извлекать актуальный контекст из внешних источников информации, что обеспечивает адаптивность и актуальность без необходимости полной повторной тренировки. Такой подход особенно полезен в задачах, где требуется оперативная обработка быстро обновляемых данных или работа с ограниченными вычислительными ресурсами.

Тем не менее, перечисленные методы редко объединяются в единую систему, способную эффективно решать задачи диалоговой суммаризации. Их совместное применение остается недостаточно изученным, что ограничивает возможности для дальнейшего совершенствования как диалоговых, так и универсальных моделей суммаризации.

Настоящая работа предлагает новый подход к созданию синтетических наборов данных с использованием метода RAG, где LLM выступают аннотаторами, а информация для обучения ICL формируется из семантически близких примеров решенной задачи. Эти примеры не только повышают качество генерируемых данных, но и объясняют модели тонкости создаваемого набора данных, исключая необходимость разработки дополнительных reward-моделей. Подход позволяет

автоматизировать аннотирование, преодолеть нехватку размеченных данных и обеспечить масштабируемое обучение моделей с минимальными затратами.

Предметом исследования являются методы и подходы, лежащие в основе создания синтетических обучающих наборов данных для тонкой настройки моделей диалоговой суммаризации. В работе рассматривается использование метода RAG, включающего поиск семантически близких к задаче примеров, что позволяет создавать синтетические данные, максимально приближенные к человеческим аннотациям, и учитывать дополнительные неявные инструкции, содержащиеся в контекстных примерах. Итогом данной работы станет разработанный фреймворк, основанный на данных методах, способный эффективно генерировать обучающие данные и адаптировать LLM для задач диалоговой суммаризации на русском языке.

Объектом исследования являются LLM и их применение для создания обучающих данных. Основные исследовательские вопросы данной работы формулируются следующим образом:

- A. Провести обзор современных методов резюмирования диалогов, включая подход LLM-as-referrer и существующие стратегии создания синтетических обучающих наборов данных.
- B. Разработать методику для создания обучающих наборов данных с применением RAG техники few-shot обучения, включающий этапы поиска семантически близких к исходной задаче примеров и их использование в качестве контекста для задачи аннотирования данных.
- C. Сравнить качество данных, созданных с использованием новой методики, с эталонными данными, полученными в результате ручного аннотирования.
- D. Провести экспериментальное обучение небольшой модели диалоговой суммаризации, используя обучающие наборы данных, созданные с помощью нового фреймворка, и оценить производительность этой модели.

ГЛАВА 1. ОБЗОР ПОДХОДОВ К ПОСТРОЕНИЮ АННОТАЦИЙ С ПОМОЩЬЮ LLM

В этом разделе описаны ключевые аспекты анализа предметной области, включая основные принципы и актуальные подходы в области суммаризации диалогов. В конце раздела приведена таблица с выводами, обобщающая основные характеристики методов и тенденции их применения.

1.1. Метод контекстного обучения ICL

ICL — это метод, позволяющий LLM эффективно адаптироваться к новым задачам, используя всего несколько примеров, известных как демонстрации. Эти примеры включают в себя пары ввода и вывода, представленные в формате естественного языка, и служат основой для формирования контекста, в котором модель выполняет поставленную задачу [2; 11]. В отличие от традиционного обучения с учителем, ICL не требует перенастройки весов модели или обновления её параметров. Это позволяет использовать LLM без дополнительного этапа дообучения, существенно снижая необходимость в ресурсоемких вычислениях. Благодаря этому ICL становится особенно привлекательным решением для применения в условиях ограниченных вычислительных ресурсов, сохраняя высокую гибкость и универсальность подхода.

Основная идея ICL заключается в том, что LLM, обученные на больших корпусах данных, способны эффективно решать новые задачи за счёт уже накопленных знаний, без необходимости дополнительного масштабного дообучения [25]. Эта способность основывается на предобучении моделей на разнообразных лингвистических паттернах, которые можно активировать с помощью тщательно подобранных подсказок и примеров. Недавние исследования показали, что при наличии достаточного количества качественных примеров (например, десяти или более) LLM могут демонстрировать производительность, сопоставимую или даже превосходящую традиционно дообученные модели. Это особенно актуально в сложных NLP-задачах, где требуется учитывать множественные намерения, контексты или сложные взаимосвязи в данных [11; 12].

Несмотря на свои преимущества, метод ICL сталкивается с рядом трудностей. Релевантность результатов, получаемых с использованием ICL в LLM, может существенно зависеть от качества и выбора демонстрационных примеров.

Это требует тщательно продуманного выбора и формирования примеров, чтобы обеспечить максимальную эффективность модели [10; 12]. В практических условиях создание или подбор демонстрационных примеров может быть трудоемким и неочевидным процессом, особенно если качество этих примеров играет ключевую роль в достижении желаемой производительности модели. Это ограничение становится ещё более заметным при использовании меньших моделей, таких как LLaMA-1B, которые, даже при наличии демонстраций, часто сталкиваются с трудностями в выполнении сложных инструкций и поддержании глубокого понимания контекста. В таких случаях эффективность метода ICL значительно возрастает при использовании крупных моделей, таких как GPT-3 или LLaMA-65B. Эти модели способны более точно обрабатывать сложные запросы и извлекать пользу из предоставленных демонстрационных примеров и окружающего их контекста, что делает их предпочтительным выбором для задач, требующих высокой точности и минимизации человеческого участия [10].

Одним из инновационных расширений ICL является контекстное обучение с дополнением поиска (retrieval-augmented in-context learning). Этот подход предполагает извлечение релевантных и качественных пар ввода и вывода, соответствующих целевой задаче, и их использование в качестве контекста в запросах. Например, работа [15] демонстрирует применение этого метода для создания качественных ответов, где извлечённые примеры из домена исходной задачи служат для адаптации модели без необходимости её обширного дообучения.

Недавние исследования, такие как ISARA[10] и Codex[12], демонстрируют высокую эффективность ICL в различных условиях, включая как контролируемые, так и неконтролируемые задачи. В этих работах подход ICL использовался для адаптации моделей к целевым задачам без необходимости их дообучения. Это стало возможным благодаря извлечению релевантных примеров из обучающих наборов данных, которые формировали контекст, направляя модель к точным и осмысленным результатам.

Таким образом, контекстное обучение зарекомендовало себя как практичный и эффективный подход к использованию скрытых знаний LLM для решения широкого спектра задач. Метод устраняет необходимость дообучения модели, позволяя быстро разворачивать её в новых задачах с минимальным участием человека, при этом поддерживая высокую производительность и способность адаптироваться к ранее невидимым доменам, обеспечивая релевантные и точные ответы.

1.2. Построение аннотаций с помощью LLM-моделей

Суммаризация с использованием LLM показывает значительные успехи в различных сценариях, особенно в задачах диалоговой суммаризации. Эксперименты продемонстрировали, что LLM способны эффективно резюмировать диалоги, обеспечивая высокую точность и согласованность. В частности, модели LLaMA и Alпаса достигли уровня фактической достоверности, превышающего 90%, согласно результатам автоматической оценки [11].

Одним из ключевых результатов исследований является то, что аннотации, созданные с использованием LLaMA и других LLM, демонстрируют наивысшую релевантность с точки зрения соответствия содержанию исходных данных и точности передаваемой информации. Однако, несмотря на обнадеживающие результаты автоматической оценки, человеческая эвалюация выявляет различия в производительности, указывая на недостатки моделей LLM, которые могут быть незаметны при машинном анализе. Исследования [11] также показывают, что LLM иногда склонны игнорировать числовое содержание, что может приводить к упущению важных данных в процессе суммаризации диалогов. Это подчёркивает необходимость внимательного подхода при работе с числовыми фактами и может требовать дополнительных корректировок для повышения точности моделей.

Интересно, что, хотя уровень фактических ошибок в обобщениях, генерируемых LLM, остаётся низким при работе с популярными наборами данных, такими как CNNDM, эксперты выявляют существенные расхождения в производительности моделей в других доменных областях. Например, в задачах суммаризации текстов из специализированных или узкопрофильных областей наблюдается более высокий уровень несоответствий в аннотациях. Исследования показывают, что в таких доменах более 30% аннотаций могут содержать ошибки, связанные с неверной интерпретацией исходных данных, что значительно превышает уровень ошибок, наблюдаемый при работе с новостными корпусами [8; 22].

Таким образом, несмотря на общие успехи, LLM имеют свои особенности и ограничения, которые необходимо учитывать при их использовании в задачах суммаризации. Тем не менее, по сравнению с существующими альтернативами, аннотации, создаваемые LLM, содержат наименьшее количество фактологических ошибок [16].

1.3. Размеры LLM модели

Размер модели LLM является важным аспектом, влияющим на её производительность при решении задач суммаризации. Предыдущие исследования [11] показывают, что при достижении определённого размера параметров производительность LLM в задаче суммаризации диалогов становится относительно стабильной, и различия между моделями начинают сглаживаться [24]. Это наблюдение позволяет оптимизировать выбор модели, учитывая баланс между производительностью и временем задержки при выводе результатов. В данной работе предпочтение отдается моделям LLM-as-referense с количеством параметров не менее 10 миллиардов, чтобы обеспечить эффективность при сохранении достаточной точности.

Однако не все задачи могут быть решены с использованием небольших моделей. Например, для выполнения сложных задач по самообучению с использованием ICL зачастую требуются модели размером не менее 65 миллиардов параметров [10]. Более мелкие модели, как правило, испытывают трудности с пониманием и выполнением сложных инструкций человека, что ограничивает их применимость в таких сценариях.

В то же время, были продемонстрированы успешные примеры передачи знаний от крупных моделей к более компактным [18]. В частности, эффективно реализована дистилляция знаний от LLaMA-3-70B к небольшому экстрактивному суммаризатору на основе BERT. Это показывает, что даже при использовании меньших моделей можно добиться значительных результатов, используя подходы переноса знаний (transfer learning).

Использование открытых моделей, таких как LLaMA-3-70B, представляет собой привлекательную альтернативу дорогим закрытым решениям, таким как GPT-4 [18]. Эти модели с большим числом параметров обеспечивают высокую производительность и гибкость, что делает их особенно ценными для задач суммаризации. Кроме того, их открытость упрощает доступ к технологиям для создания эталонных обучающих наборов данных, стимулируя исследования и внедрение новых решений в прикладные области.

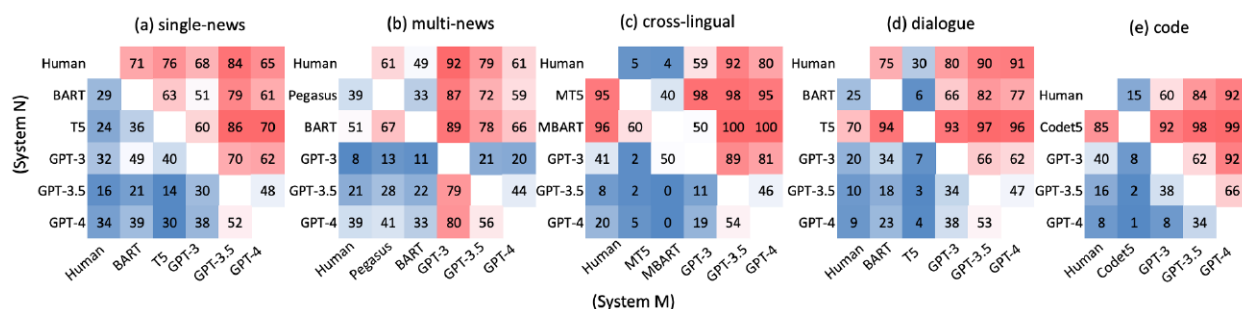


Рис.1.1. Парные показатели выигрыша (%) между различными системами в 5 задачах. Каждая точка данных представляет собой долю случаев, когда система М (горизонтальная ось) предпочтительнее системы N (вертикальная ось) в сравнениях. Красный указывает на показатель выигрыша более 50%, что указывает на предпочтение системы М, тогда как синий указывает на показатель выигрыша менее 50%, что указывает на предпочтение системы N, а более темные цвета указывают на большую разницу в показателях выигрыша между двумя системами [15]

1.4. Сравнительный анализ аннотаций LLM-моделей

Сравнение аннотаций, создаваемых LLM, с аннотациями, написанными людьми, стало одной из ключевых тем в исследованиях обработки естественного языка и машинного обучения [13]. Недавние работы [8; 15; 17] показали, что такие модели, как GPT-3.5 [21], способны создавать обобщения, которые получают более высокие оценки при человеческой оценке рис.1.1 по сравнению с эталонными обобщениями из широко используемых наборов данных, таких как CNN/DailyMail [3]. Эксперименты с использованием GPT-4 и GPTRank подтвердили, что LLM могут превосходить отдельных аннотаторов в производительности, предлагая более релевантные и точные аннотации. Тем не менее, аннотации, созданные методом коллективного голосования экспертов, по-прежнему считаются эталоном качества в задачах суммаризации, демонстрируя преимущество коллективного подхода над индивидуальными усилиями, включая модели LLM.

Количественные и качественные сравнения между обобщениями, созданными LLM, человеческими аннотациями и аннотациями, создаваемыми дообученными моделями, показали, что LLM-обобщения значительно предпочтительнее с точки зрения человеческих оценщиков. Они также демонстрируют более высокий уровень фактической точности [15]. Эти результаты подчёркивают потенциал LLM как инструмента, способного превосходить традиционные методы резюмирования текста.

Однако следует учитывать, что аннотации, созданные людьми, также имеют свои недостатки. В частности, они могут страдать от недостатка речевой плавности, что снижает их воспринимаемость [15]. Кроме того, некоторые эталон-

ные аннотации содержат неполную информацию или включают так называемые "галлюцинации"— случаи, когда в итоговый вариант обобщения добавляются данные, не подтверждённые в исходном тексте. Эти факторы поднимают вопросы о надёжности и точности человеческих аннотаций, особенно в сравнении с выводами, генерируемыми LLM, которые зачастую демонстрируют более высокую согласованность и соответствие исходным данным.

Эксперименты показали, что использование LLM в качестве эталонных моделей для обучения меньших моделей суммаризации может значительно повысить производительность последних [13]. Подход "LLM-as-reference" продемонстрировал заметные улучшения: модели, обученные с его использованием, показали высокие результаты, подтверждённые как автоматической оценкой, так и человеческой экспертизой. Это подчёркивает потенциал LLM в создании аннотаций, которые не только соответствуют, но в некоторых случаях даже превосходят человеческие аналоги.

1.5. Аннотации моделей «трансформер»

Сравнение суммаризации, выполняемой LLM, и моделями, основанными на дообученных трансформерах (Fine-Tuned Transformer, FTSumm), выявляет интересные различия в их производительности и особенностях. Исследования показывают, что аннотации, создаваемые LLM, стабильно превосходят как ручную созданные аннотации, так и те, что были сформированы дообученными трансформерными моделями, практически во всех тестируемых задачах [15]. Однако экспертная оценка выявляет и нюансы: в некоторых аспектах суммаризации дообученные модели демонстрируют более высокую точность или лучше справляются с конкретными особенностями текста, которые LLM могут упустить.

Одним из ключевых отличий между LLM и FTSumm является гибкость в длине генерируемого текста [15]. Дообученные модели, как правило, создают аннотации фиксированной длины, что может ограничивать их способность адекватно представлять содержимое текста. В отличие от них, LLM способны адаптировать длину результата в зависимости от объема и сложности входных данных. Это преимущество становится особенно важным при обработке текстов с несколькими темами. В таких случаях аннотации, создаваемые FTSumm-моделями, часто охватывают лишь часть обсуждаемых вопросов, тогда как LLM более успешно

передают весь спектр затронутых тем, обеспечивая более полное и информативное обобщение

Однако LLM не лишены недостатков. Примерно 38% ошибок в их обобщениях связаны с выводами на основе косвенных доказательств, так называемыми обстоятельственными выводами [16]. Эти ошибки встречаются значительно реже в обобщениях, созданных моделями BART, где они составляют лишь 1% от общего числа ошибок. С другой стороны, LLM демонстрируют заметное преимущество в снижении количества логических ошибок: только 17% всех ошибок относятся к этой категории, тогда как для моделей FTSumm этот показатель превышает 50% [16]. Это подчёркивает способность LLM более эффективно извлекать логически корректные выводы из диалогов, что особенно важно в задачах, где критичность передаваемой информации играет ключевую роль.

Таким образом, LLM демонстрируют высокую гибкость и способность охватывать сложные темы, создавая более полные и информативные обобщения. Однако их склонность к ошибкам, связанным с косвенными выводами, контрастирует с моделями FTSumm, которые, хотя и избегают подобных ошибок, страдают от большей доли логических неточностей и фактических несоответствий.

1.6. Дообучение моделей «трансформер» с помощью аннотаций LLM

Подход к обучению моделей суммаризации на основе трансформеров, использующий LLM в качестве источников аннотаций, демонстрирует значительные преимущества. В этом подходе LLM выполняют две важные функции: они предоставляют эталонные аннотации для обучения моделей и служат инструментом для оценки их точности. Одним из ключевых результатов этого подхода является то, что модели суммаризации, обученные на аннотациях, созданных с помощью LLM-as-reference, стабильно показывают более высокие результаты по сравнению с моделями, использующими традиционные эталонные аннотации, созданные вручную. Это подчёркивает потенциал LLM не только в улучшении качества обучающих данных, но и в повышении точности и эффективности моделей суммаризации [13].

Экспериментальные данные [13] подтверждают, что аннотации, созданные с помощью LLM, являются более предпочтительным источником обучающих данных, чем вручную созданные экспертами аннотации. Это преимущество было продемонстрировано на примере дообученной модели BART, которая в рамках экспериментов смогла превзойти GPT-3.5 при оценке с использованием GPT-4.

Кроме того, экспертная оценка показала, что модель BART, обученная на аннотациях LLM, достигает схожей или лучшей производительности по сравнению с GPT-3.5 в 50% случаев. Для сравнения, версия модели BART, обученная на человеческих аннотациях, демонстрирует такие результаты лишь в 4% случаев. Эти данные подчёркивают потенциал использования LLM-аннотаций для существенного повышения эффективности дообученных моделей.

Основные выводы работы подчёркивают важность использования высококачественных эталонных данных, особенно в процессе дообучения как трансформеров, так и LLM для задач суммаризации. (1) Производительность меньших моделей суммаризации значительно улучшается при использовании более качественных обучающих данных, и (2) модели, обученные на синтетических наборах данных, часто превосходят варианты моделей, обученные на стандартных наборах данных, размеченных вручную.

Несмотря на очевидные преимущества использования синтетических данных для обучения, проблема согласованности между оценками, предоставляемыми LLM, и результатами человеческой оценки остаётся нерешённой и требует дальнейшего изучения. Тем не менее, полученные результаты подчёркивают потенциал LLM в создании высококачественных эталонных наборов данных, которые могут значительно повысить точность, производительность и обобщающую способность моделей суммаризации.

1.7. Эффективность подходов Few-shot и Zero-shot

Один из наиболее интересных аспектов использования LLM заключается в их способности обучаться в zero-shot, one-shot и few-shot режимах. В отличие от традиционного подхода, требующего большого количества примеров для обучения (от сотен до сотен тысяч), техника обучения few-shot использует всего несколько пар "вход — выход" в качестве демонстраций, одновременно формируя запрос для выполнения задачи. В этом подходе модель не изменяет свои веса, а использует предоставленные данные в качестве префикса, который задаёт контекст для выполнения задачи. Это становится возможным благодаря предобучению моделей на обширных и разнообразных корпусах данных, что позволяет им извлекать скрытые закономерности и эффективно адаптироваться к новым задачам на основе предоставленного контекста.

Например, для задачи комментирования фрагментов кода (code summarization) исследование [4] демонстрирует, что при наличии всего 10 примеров модель Codex превосходит все дообученные модели, такие как CodeT5, CodeBERT и другие. Более того, Codex показывает лучшие результаты в решении задач на шести языках программирования, даже если дообученные модели были обучены на нескольких тысячах примеров.

Дополнительное наблюдение в исследовании [4] показывает, что few-shot обучение на данных одного проекта улучшает производительность модели Codex по всем 8 проектам, что подчеркивает значительные преимущества использования подобных подходов. Zero-shot режим также демонстрирует свою эффективность, однако few-shot обучение, при котором предоставляется ограниченное число релевантных примеров, позволяет LLM достигать заметного прироста производительности, эффективно используя возможности предварительно обученных моделей.

1.8. Важность отбора демонстрационных примеров для подхода Few-shot

Отбор демонстрационных примеров для few-shot learning играет ключевую роль в повышении качества выполнения задач крупными языковыми моделями [5; 10; 12]. Стратегии выбора примеров помогают моделям лучше интерпретировать задачу, что приводит к значительному увеличению их производительности. Например, при использовании десяти примеров, отобранных на основе их семантического сходства с целевым кодом, значения оценки BLEU модели Codex увеличиваются на 97% и 131% по двум различным наборам данных по сравнению с результатами при случайном выборе примеров [12].

Исследования также подтверждают, что качество демонстрационных примеров напрямую влияет на эффективность выполнения задач в контексте [12]. Использование семантического и токенового сходства при отборе примеров позволяет моделям значительно превосходить результаты, полученные при случайном подборе данных. Более того, оказалось, что производительность модели в 1-shot режиме, при условии правильного подбора примеров, может превосходить 10-shot режим с случайно отобранными демонстрационными примерами.

Одним из лучших подходов является использование стратегий поиска и отбора высококачественных примеров из смежных доменов для предоставления их в качестве демонстраций. Эти примеры способствуют улучшению понимания

модели и повышают качество генерируемых выходных данных. Результаты исследований подтверждают, что использование семантического отбора примеров в few-shot сценариях позволяет моделям, таким как Codex [12], достигать более высоких показателей по сравнению с передовыми методами, такими как DOME.

Итеративные механизмы, применяемые в ISARA, показывают, что подходы на основе выборки примеров могут быть эффективны даже для моделей с меньшим количеством параметров. Методы, такие как RAG, подчеркивают значимость объединения механизмов информационного поиска с моделями текстовой генерации, особенно в задачах, требующих работы с обширными и сложными знаниями. Совокупный сбор и использование примеров, семантически близких к вводным запросам значительно улучшают качество создаваемых суммаризаций, позволяя моделям более точно отражать ключевые аспекты исходных данных. [10].

1.9. Техника Instruction Tuning для итоговой модели суммаризации

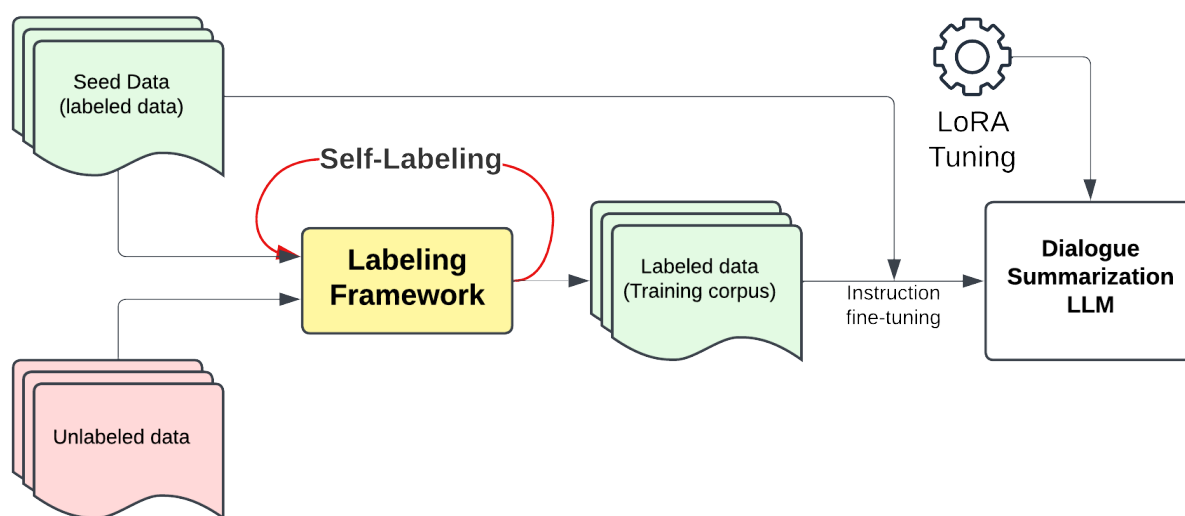


Рис.1.2. Полная методика процесса тонкой настройки диалогового LLM суммаризатора

Заключительным этапом данной работы является процесс тонкой настройки базовой LLM специально для задачи диалоговой суммаризации рис.1.2.

Instruction-tuning был выбран в качестве основного подхода, поскольку он существенно изменяет способность больших языковых моделей следовать инструкциям и адаптироваться к разнообразным задачам, включая суммаризацию диалогов. В отличие от традиционных методов тонкой настройки, которые требуют крупных вручную размеченных наборов данных, данный подход позволяет обучать модель на парах "инструкция — ответ включая данные, созданные с использованием синтетических алгоритмов и основанные на заранее определённых примерах. Хотя этот

подход по-прежнему требует ручную разметку данных для базового обучения, он делает процесс настройки модели более гибким, позволяя использовать синтетически созданные пары "инструкция — ответ" для повышения эффективности при выполнении конкретных задач, включая диалоговую суммаризацию.

Эффективность данного метода была убедительно продемонстрирована в исследовании [6], где использование instruction-tuning позволило достичь значительных улучшений. Например, в работе [20] модель Alpaca-7B, обученная на наборе из 52 тысяч инструкций, продемонстрировала производительность, сопоставимую с OpenAI text-davinci-003, несмотря на меньший объём параметров. Аналогичные результаты были получены для мультимодальной модели LLaVA [23], что подчеркивает универсальность подхода instruction-tuning и его применимость к различным типам задач.

Для оптимизации вычислительных затрат в данной работе дополнительно используется LoRA. Это решение позволило существенно снизить ресурсы, необходимые для обучения, сохранив высокую производительность итоговой модели.

1.10. Выводы

В данной работе были проанализированы основные подходы к суммаризации, где LLM используется в качестве аннотатора для создания размеченных данных. Также рассмотрены современные методы работы с LLM и ключевые технологии, лежащие в основе гипотез и принципов, используемых в исследовании. Для более детального анализа представлена сравнительная таблица табл.1.1, в которой обобщены наиболее значимые и релевантные исследования, упомянутые ранее.

ISARA использует многие из рассмотренных технологий, но обладает рядом ограничений, которые сужают её применимость. Во-первых, метод ориентирован на итеративную настройку относительно небольших моделей (не более 7 миллиардов параметров), что ограничивает его потенциал в задачах, требующих работы с крупными языковыми моделями. Во-вторых, ISARA не предназначена для задач создания обучающих наборов данных, так как её основное применение заключается в адаптации моделей для задач с минимальным количеством доступных примеров, что делает её более подходящей для узких доменов. В отличие от этого, предлагаемый фреймворк фокусируется на генерации высококачественных обучающих наборов данных, которые могут использоваться для тонкой настройки

моделей, включая задачи диалоговой суммаризации. Это различие подчёркивает, что наш подход нацелен на обеспечение универсальности и масштабируемости в широком спектре задач, тогда как ISARA имеет более ограниченную сферу применения.

Codex[12] реализует концепцию, которая в некоторых аспектах схожа с предлагаемым фреймворком, однако его применение ограничено задачей генерации комментариев к коду с использованием few-shot подхода и подхода ICL. Хотя *Codex* демонстрирует высокую производительность в генерации многоцелевых комментариев к коду, его методология сосредоточена на предоставлении ответов в рамках одной итерации, без дальнейшего использования сгенерированных данных для последующих этапов обучения или улучшения модели. В отличие от *Codex*, разрабатываемый фреймворк ориентирован на создание высококачественных обучающих наборов данных, которые могут быть использованы для итеративной доработки моделей, включая задачи диалоговой суммаризации. Это позволяет нашему подходу быть более универсальным и применимым к широкому спектру задач, выходящих за пределы генерации комментариев к коду. Более того, фреймворк поддерживает настройку моделей для работы в многоступенчатом процессе, что повышает его способность к адаптации и улучшению результатов на основе накопленных данных.

Таблица 1.1

Сравнение подходов в работе с LLM

Работа / Метод	Data	Few-Shot prompting	Retrieved examples	Trained model type	Task	No Reward System
ISARA[10]	QA dataset (English)	Yes	Yes	Transformer/ LLM	QA generation	Yes
CODEX[12]	Code snippets	Yes	Yes	LLM	Code– summarization	Yes
MixSumm[18]	Variable (English)	Yes	No	Transformer	Summarization	Yes
Baize[7]	QA dataset (English)	No	No	LLM	Dialogue model	Yes
Salmon[19]	Question– only dataset	Yes	No	LLM	Model tuning	Yes
Разрабатыва- емая модель	Dialogues (Russian)	Yes	Yes	LLM	Summarization	Yes

MixSumm[18] - это метод аугментации данных, направленный на улучшение экстрактивной суммаризации в условиях ограниченных ресурсов. Он использует LLM, такие как LLaMA-3-70b, для генерации документов, объединяющих информацию из нескольких тем, что повышает разнообразие и качество обучающих данных. Однако *MixSumm* требует значительных вычислительных ресурсов для генерации и обработки таких синтетических данных, что может быть затруднительно при ограниченных ресурсах. В отличие от *MixSumm*, разрабатываемый фреймворк фокусируется на создании обучающих наборов данных с минимальными затратами ресурсов, что делает его более применимым в условиях ограниченных вычислительных возможностей.

Baize[7] — это открытая модель чата, разработанная с использованием параметрически эффективной настройки на основе данных, сгенерированных методом самочата. В отличие от нашего фреймворка, ориентированного на создание обучающих наборов данных для задач суммаризации, *Baize* фокусируется на улучшении моделей для многократных диалогов. Кроме того, *Baize* применяет технику Self-Distill with Feedback для повышения производительности, тогда как наш подход направлен на генерацию и использование синтетических данных для обучения моделей суммаризации.

На основе рассмотренных исследований и методов были сформулированы несколько ключевых гипотез, лежащих в основе разрабатываемого фреймворка. Основная идея заключается в том, что комбинированное использование техник ICL, методов отбора few-shot примеров и RAG позволяет не только повысить производительность LLM в задачах диалоговой суммаризации, но и оптимизировать процесс создания инструкций для продвинутых методов суммаризации, используя минимальное количество вручную размеченных данных. Эти гипотезы могут быть сформулированы следующим образом:

- Демонстрационные few-shot примеры, отобранные с учётом их семантической близости к целевому диалогу, помогают модели лучше выделять ключевые элементы разговора, исключая второстепенные детали и повышая точность суммаризации;
- примеры, отражающие особенности стилистики общения (например, формального или неформального тона), позволяют модели подстраиваться под специфические контексты, улучшая восприятие результатов;

- использование эталонных примеров и механизма обратной связи даёт модели возможность корректировать свои ответы, повышая их точность и согласованность.

Эти гипотезы подчёркивают потенциал комбинирования ICL, few-shot подходов и RAG для значительного повышения качества диалоговой суммаризации. Предлагаемый фреймворк минимизирует необходимость в ручной разметке, обеспечивая не только адаптацию модели к специфическим задачам, но и создание высококачественных обучающих наборов данных с минимальным участием человека. Такой подход открывает новые возможности для масштабирования задач суммаризации в различных доменах, от поддержки пользователей до анализа бизнес-данных.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА МЕТОДИКИ И ФРЕЙМВОРКА ДЛЯ СУММАРИЗАЦИИ

Во второй главе данной работы излагаются теоретические основы, необходимые для решения поставленной задачи. Вначале даётся вербальная постановка проблемы, рассматриваются её особенности и основные ограничения, а также формулируются цели и ожидаемые результаты исследования. Далее следует математическая постановка задачи, позволяющая строго определить входные данные, параметры, критерии качества и формальные ограничения. На основе полученной формализации предлагается теоретический фреймворк, интегрирующий методы RAG и ICL для автоматической генерации синтетических обучающих данных. В заключение главы приводятся схемы архитектуры и описываются возможности практического внедрения разработанного подхода.

2.1. Постановка задачи

Цель данной работы — разработать и апробировать итеративный фреймворк, который, опираясь на современные подходы RAG и ICL, позволит автоматизировать процесс генерации синтетических обучающих данных для диалоговой суммаризации. Предполагается, что предложенный подход сократит необходимость в ручной разметке и обеспечит возможность получения качественных суммаризаций в широком круге диалоговых доменов.

Для достижения поставленной цели формулируются следующие задачи:

А. Формализация проблемы.

Разработать математическую постановку, учитывающую особенности диалоговых данных, а также определяющую способы оценки качества сгенерированных суммаризаций.

В. Реализация фреймворка на основе новой методики.

Создать концептуальный подход, интегрирующий поиск релевантных примеров (RAG) и использование их в качестве демонстраций для LLM (ICL), повышая точность и достоверность автоматически сгенерированных суммаризаций.

С. Итеративное обновление обучающей базы.

Организовать процесс поэтапного (итеративного) пополнения обучающей базы синтетическими аннотациями, что позволит постепенно повышать качество суммаризаций на каждом шаге.

Д. Валидация и сравнение.

Оценить предложенный подход, сравнив качество сгенерированных данных с эталонными ручными аннотациями, и продемонстрировать эффективность решения относительно альтернативных стратегий.

В результате создания итеративного фреймворка, сочетающего RAG, ICL и использование LLM в роли «аннотаторов», мы адресуем ключевую проблему нехватки вручную размеченных данных. Предполагается, что подобный комплексный подход не только снизит зависимость от ручной аннотации, но и позволит постепенно повышать качество итоговых суммаризаций за счёт регулярного обновления обучающей базы. Экспериментальная проверка и анализ результатов дадут возможность подтвердить или опровергнуть гипотезу о превосходстве данной методики над существующими альтернативами в области диалоговой суммаризации.

В ходе выполнения данной работы ожидается достижение следующих результатов:

- *Эффективная методология создания обучающих данных:* разработанный фреймворк будет способен автоматически генерировать синтетические обучающие наборы данных, существенно снижая объём ручной разметки.
- *Улучшение качества суммаризаций:* итоговые суммаризации станут сопоставимы по качеству с ручными аннотациями, а в определённых случаях — превосходить их уровень.

- *Сокращение затрат на ручную аннотацию:* благодаря автоматизации ключевых шагов формирования обучающего набора, трудозатраты экспертов и финансовые издержки значительно уменьшатся.
- *Итеративное улучшение:* на каждом новом цикле пополнения базы ожидается улучшение метрик качества по сравнению с базовыми методиками без итеративного компонента.
- *Универсальность подхода:* возможность эффективного применения фреймворка в различных предметных областях, обеспечивая гибкость, адаптивность и масштабируемость решения.
- *Преимущество комплексного подхода над существующими аналогами:* интеграция RAG, ICL и возможностей LLM в единый фреймворк демонстрирует более высокую эффективность по сравнению с менее комплексными или односторонними решениями.

Предполагается, что предложенная методика внесёт значимый вклад в развитие NLP и расширит горизонты применения больших языковых моделей, позволяя им адаптироваться к новым задачам в условиях ограниченности ресурсов.

2.2. Формализация методики

В данном разделе последовательно вводятся используемые обозначения, ставятся задачи оптимизации, формализуются метрики оценки качества сгенерированных суммаризаций, а также рассматриваются дополнительные аспекты, такие как влияние параметров и итеративный характер подхода.

Пусть дан большой набор неразмеченных диалогов:

$$D = \{d_1, d_2, \dots, d_n\},$$

где каждый диалог d_i представлен как последовательность реплик:

$$d_i = \{r_{i1}, r_{i2}, \dots, r_{im}\},$$

здесь r_{ij} — j -ая реплика в диалоге d_i , а m — переменная, указывающая на общее количество реплик в диалоге. Предполагается, что каждый диалог состоит из текстовых реплик без мультимодальной (например, аудио или видео) информации.

Кроме того, имеется небольшой эталонный (ручной) набор:

$$S_{\text{seed}} = \{(d_j^{\text{seed}}, s_j^{\text{seed}}) \mid j = 1, \dots, k\}, \quad k \ll n.$$

Здесь s_j^{seed} — это эталонная суммаризация диалога d_j^{seed} , созданная вручную, а не моделью. Она определяет требуемый формат и уровень качества и служит ориентиром для начальной настройки процесса автоматической генерации.

Необходимо для каждого неразмеченного диалога $d_i \in D$ автоматически получить итоговую суммаризацию s_i таким образом, чтобы пара (d_i, s_i) могла быть добавлена в обучающий набор:

$$S = S_{\text{seed}} \cup \{(d_1, s_1), (d_2, s_2), \dots, (d_n, s_n)\},$$

где s_i — автоматически сгенерированная суммаризация диалога d_i . Итоговый набор S в идеале должен позволять обучать или дообучать модели суммаризации при минимальном участии человека.

При этом формально заданная функция качества (целевая функция) может быть неявной, а оценка генерируемых суммаризаций производится внешними метриками, такими как ROUGE или BERTScore. В данном контексте задача оптимизации сводится к выбору стратегии генерации s_i , при которой указанные метрики достигают наилучших значений или удовлетворяют заданным критериям качества. Более подробное описание используемых метрик и критериев оценки планируется представить в отдельном разделе (см. раздел 4.2), где будут даны их формальные определения и методики расчёта.

Для решения поставленной задачи используются следующие компоненты:

Для выбора релевантных демонстрационных примеров используется функция кодирования Encoder, преобразующая диалоги в векторные представления:

$$e_i = \text{Encoder}(d_i) \in \mathbb{R}^m,$$

где e_i — векторное представление диалога d_i .

Эталонные данные и ранее сгенерированные пары хранятся в базе эмбеддингов:

$$E = \{e_j^{\text{seed}} : j = 1, \dots, k\} \cup \{e_l^{\text{gen}} : l \in \text{индексы сгенерированных данных}\}.$$

Функция поиска релевантных примеров определяется как:

$$D_i = \mathcal{R}(e_i, E, K),$$

где D_i содержит K наиболее близких примеров, найденных по метрике косинусного сходства (или иной выбранной мере). Параметр K регулирует количество демонстрационных примеров, предоставляемых модели.

На основе выбранных примеров D_i и текущего диалога d_i формируется запрос P_i :

$$P_i = \text{Format}(D_i, d_i),$$

где Format — это функция, объединяющая демонстрационные примеры и текущий диалог в структуру (промпт), подходящую для работы модели $\mathcal{M}_{LLM}$. Конкретный способ форматирования (например, последовательность пар «Диалог $\rightarrow$ Суммаризация», за которым следует новый диалог) выбирается в соответствии со стандартами ICL.

Пусть $\mathcal{M}_{LLM}$ обозначает языковую модель, способную по входному запросу P_i генерировать текст:

$$s_i = \mathcal{M}_{LLM}(P_i),$$

где P_i включает демонстрационные примеры из D_i и сам диалог d_i . Итогом является сгенерированная суммаризация s_i .

Решение задачи предусматривает итеративную схему. Для каждого диалога $d_i \in D$ формируется пара (d_i, s_i) , где s_i — полученная на основе шага 3 суммаризация. Эта пара добавляется в обучающий набор:

$$S_{t+1} = S_t \cup \{(d_i, s_i)\}.$$

После этого база эмбедингов обновляется:

$$E_{t+1} = E_t \cup \{e_i^{\text{gen}}\}, \quad e_i^{\text{gen}} = \text{Encoder}(d_i).$$

Новый набор S_{t+1} используется для выбора демонстрационных примеров на следующем шаге, а обновлённая база эмбедингов повышает охват потенциально релевантных данных.

С точки зрения качества итоговых суммаризаций, ожидается, что на ранних этапах роль $\{(d_j^{\text{seed}}, s_j^{\text{seed}})\}$ будет доминировать. По мере расширения синтетического набора доля автоматически созданных (и уже проверенных по внешним метрикам) примеров будет возрастать, что при правильном выборе параметров и тщательном контроле качества позволит модели постепенно улучшать свои результаты. Критерием остановки итераций может служить достижение желаемых значений метрик суммаризации или исчерпание неразмеченных диалогов.

Эффективность и качество предложенного метода зависят от нескольких ключевых параметров, влияющих на процесс генерации синтетических данных и итоговые результаты суммаризации:

- *Чувствительность к количеству демонстраций K .* Недостаточно большое K может приводить к недостаточному контексту для модели, а слишком большое K — к размыванию важности наиболее релевантных примеров. Оптимальное значение K зависит от сложности задачи и характеристик данных.
- *Чувствительность к размеру эталонного набора S_{seed} .* Небольшой объём эталонных данных может ухудшить качество на ранних итерациях, в то время как увеличение $|S_{seed}|$ улучшает начальные результаты, задавая более высокую планку качества.
- *Выбор модели и параметров энкодера.* Различные модели-энкодеры создают разные векторные представления, что влияет на точность выбора ближайших демонстрационных примеров. Оптимальный выбор энкодера повышает эффективность всего процесса суммаризации.

Оптимизация указанных параметров повышает адаптивность фреймворка и минимизирует риски деградации качества итоговых суммаризаций. В данной работе подбор значений K , размера S_{seed} и конфигурации энкодера осуществлялся преимущественно эмпирическим путём. Однако возможно также использовать более формальные или автоматизированные стратегии подбора: например, метод перебора (grid search), эвристические алгоритмы, Bayesian Optimization или другие подходы, основанные на обратной связи от метрик качества.

Таким образом, предложенный подход обеспечивает основу для итеративной генерации и дообучения на синтетических суммаризациях, позволяя по мере роста набора автоматически аннотированных диалогов постепенно улучшать итоговое качество модели, измеряемое по внешним метрикам (ROUGE, BERTScore и др.). Все основные аспекты (от неявной целевой функции и влияния параметров до итеративного характера метода) учитываются в представленной математической постановке задачи.

2.3. Алгоритм предлагаемого фреймворка

Предлагаемый алгоритм предназначен для автоматической генерации новых пар «диалог — итоговая суммаризация», используя небольшой изначальный набор эталонных данных (seed), а также возможности LLM. Основная идея состоит в том, чтобы на каждом шаге (итерации):

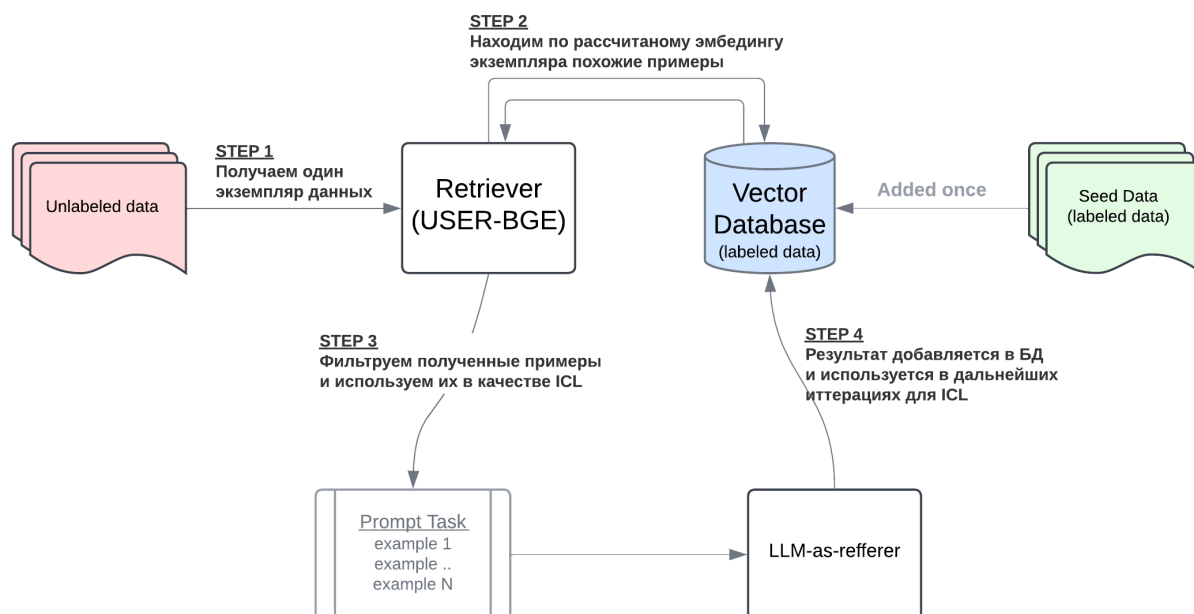


Рис.2.1. Общая логика работы алгоритма

Предлагаемый алгоритм направлен на автоматическую генерацию синтетических обучающих наборов данных для задачи диалоговой суммаризации. Алгоритм использует подходы RAG и ICL, а также LLM для итеративного создания новых пар «диалог — итоговая суммаризация». Основной процесс состоит из следующих этапов:

А. Извлечение диалога:

- Как показано на рисунке 2.1, один экземпляр (*dialogue*) выбирается из пула *Unlabeled data*. Этот процесс представляет собой последовательную выборку. Если все диалоги исчерпаны, итерации завершаются.

В. Поиск релевантных демонстрационных примеров:

- Функция $\text{Encoder}(d_i)$ формирует вектор e_i .
- Выполняется обращение к базе E (*Vector Database*), содержащей эмбединги seed-диалогов или ранее обработанных данных.
- Из базы извлекаются K ближайших векторов, соответствующих K диалогам, которые затем используются в качестве демонстраций (*ICL*).

С. Формирование промпта:

- На основе найденных примеров собираются K пар «диалог, суммаризация» путём сопоставления эмбедингов с соответствующими текстами.

- В запрос добавляется текущий диалог d_i , но пока без итоговой суммаризации.
- Итоговый текстовый запрос (*prompt*) передаётся в LLM.

D. Добавление результата:

- Модель LLM (в блок-схеме обозначена как *LLM-as-referrer*) генерирует итоговую суммаризацию s_i .
- Полученная пара (d_i, s_i) сохраняется в общий набор S .
- В базу E записывается только эмбединг e_i , что позволяет использовать данный диалог при последующих итерациях как потенциальный демонстрационный пример.

На этапе инициализации выполняются следующие шаги:

- *Формирование эталонного набора S_{seed}* : Небольшой вручную размеченный набор данных, состоящий из пар (d_j^{seed}, s_j^{seed}) , формируется для задания стандарта качества итоговых суммаризаций. Этот набор обеспечивает начальные ориентиры для алгоритма и служит основой для дальнейших итераций.
- *Формирование векторной базы данных E* : Для каждого диалога d_j^{seed} из эталонного набора вычисляются эмбединги $e_j^{seed} = \text{Encoder}(d_j^{seed})$, которые затем добавляются в векторную базу данных E . Векторная база используется для поиска релевантных демонстрационных примеров, необходимых на последующих этапах алгоритма.
- *Хранение исходных текстов*: Векторная база E содержит только эмбединги исходных диалогов, а тексты самих диалогов и соответствующих суммаризаций s_j^{seed} хранятся в отдельной структуре данных. Это позволяет эффективно комбинировать поиск по эмбедингам с доступом к текстовым данным.

Ниже приведен псевдокод 2.2, демонстрирующий логику алгоритма. Его можно сопоставлять с блок-схемой на рисунке 2.1, где шаги нумеруются (*Step 1–4*). Предполагается, что на этапе инициализации у нас уже есть некоторое (пусть и небольшое) количество вручную размеченных данных S_{seed} .

Данный раздел описывает дополнительные аспекты и рекомендации для эффективной реализации предложенного фреймворка, включая ключевые детали алгоритма, гибкость его параметров и возможные оптимизации.

На этапе инициализации мы не создаём векторные представления для всех диалогов (включая неразмеченные), а только для исходных размеченных данных

Input:

- $\mathcal{M}_{\text{LLM}}$: Предобученная крупная языковая модель
- S_{seed} : Эталонный набор пар $(d_j^{\text{seed}}, s_j^{\text{seed}})$
- D : Пул неразмеченных диалогов $\{d_i\}$
- Encoder: Функция эмбединга $e = \text{Encoder}(d)$
- E : Векторная база эмбедингов, инициализированная эталонными данными
- K : Число демонстрационных примеров для ICL
- *StopCondition*: Критерий завершения (например, исчерпание данных или достижение заданной метрики)

Output: S : Итоговый набор пар (d_i, s_i)

1.Инициализация:

2. $S \leftarrow S_{\text{seed}}$; // Инициализируем обучающий набор эталонными данными
3. $E \leftarrow \emptyset$; // Пустая база эмбедингов
4. **foreach** $(d_j^{\text{seed}}, s_j^{\text{seed}}) \in S_{\text{seed}}$ **do**
5. $e_j^{\text{seed}} \leftarrow \text{Encoder}(d_j^{\text{seed}})$; // Кодлируем эталонные диалоги
6. $E \leftarrow E \cup \{e_j^{\text{seed}}\}$; // Добавляем эмбединги в базу

7.Основной цикл:**8.repeat**

- // Шаг 1: Извлечение диалога
9. $d_i \leftarrow \text{GetNextUnlabeled}(D)$; // Берём очередной диалог из пула
10. **if** $d_i = \emptyset$ **then**
11. **break** ; // Если диалоги закончились, выходим
- // Шаг 2: Поиск релевантных примеров
12. $e_i \leftarrow \text{Encoder}(d_i)$; // Кодлируем текущий диалог
13. $D_i \leftarrow \mathcal{R}(e_i, E, K)$; // Находим K ближайших примеров
- // Шаг 3: Формирование промпта
14. $P_i \leftarrow \text{Format}(D_i, d_i)$; // Создаём текстовый запрос для LLM
- // Шаг 4: Генерация и обновление
15. $s_i \leftarrow \mathcal{M}_{\text{LLM}}(P_i)$; // Генерируем суммаризацию
16. $S \leftarrow S \cup \{(d_i, s_i)\}$; // Добавляем пару в обучающий набор
17. $E \leftarrow E \cup \{\text{Encoder}(d_i)\}$; // Обновляем базу эмбедингов

18.until *StopCondition*(S);**19.return** S

Рис.2.2. Псевдокод алгоритма фреймворка

(S_{seed}). Однако как только берется новый неразмеченный диалог d_i , мы незамедлительно кодируем его. Это может быть как единоразовый процесс (для каждого диалога по мере поступления), так и заранее проведённый, если хочется уменьшить задержку во время итераций.

Количество демонстрационных примеров K , используемых в ICL, может изменяться на каждой итерации. Например, значение K может увеличиваться по мере накопления синтетически созданных данных, чтобы учесть растущую сложность задачи. В начале, с ограниченным объёмом эталонных данных, K может быть небольшим, а затем постепенно расти, обеспечивая более широкий контекст для модели.

Функция $\mathcal{R}(e_i, E, K)$ отвечает за выбор K ближайших эмбеддингов из базы E . Каждому эмбеддингу в E сопоставлены исходный диалог и его итоговая суммаризация, которые хранятся во внешней структуре. Метрика сходства (например, косинусная близость) используется для определения релевантности. Для повышения качества демонстраций можно включить этап фильтрации, исключая демонстрационные варианты с недостаточной релевантностью, нежелательными характеристиками или потенциальными «галлюцинациями».

Формирование промпта для LLM выполняется функцией $\text{Format}(D_i, d_i)$, которая преобразует выбранные демонстрации и текущий диалог d_i в удобный для модели формат. Обычно это последовательность пар вида (Диалог, Суммаризация), за которыми следует текущий диалог d_i без аннотации.

Сгенерированные суммаризации s_i не переводятся в эмбеддинги и не добавляются в базу эмбеддингов E , поскольку задача поиска сосредоточена на исходных диалогах. Итоговые пары (d_i, s_i) сохраняются в обучающем наборе S и могут быть повторного использованы в дальнейших итерациях.

В базовом варианте предполагается, что крупная языковая модель M_{LLM} остаётся неизменной на всех итерациях алгоритма. Однако, если возникает необходимость повышения качества итоговых ответов, можно использовать текущий обучающий набор S_k для instruction-tuning модели на k -й итерации.

В этом случае параметры модели M_{LLM} обновляются на основе синтетически сгенерированных данных и эталонного набора, что позволяет улучшать её производительность на последующих шагах. Однако данный подход требует дополнительных вычислительных ресурсов и времени, что делает его необязательным, но совместимым с описанным фреймворком.

2.3.1. Дополнительный сценарий: только неразмеченные данные

В ряде случаев может возникнуть ситуация, когда изначально отсутствует размеченный набор данных (*SeedData*), который используется для инициализации процесса генерации синтетических обучающих данных. Тем не менее, предложенный фреймворк может быть адаптирован для работы исключительно с неразмеченными диалогами. В таких условиях возможно использование следующего алгоритма для формирования минимального начального набора размеченных данных:

- A. *Начальный набор данных.* Изначально имеется только неразмеченный корпус диалогов $\{d_i\}$, который служит входными данными.
- B. *Вычисление эмбедингов.* Для каждого диалога d_i из корпуса вычисляются векторные представления с использованием функции эмбединга $e_i = \text{Encoder}(d_i)$. Эти векторы позволяют перейти от текстового представления к семантическому пространству.
- C. *Кластеризация данных.* На основе вычисленных эмбедингов проводится кластеризация неразмеченных диалогов с использованием алгоритмов, таких как k-means, HDBSCAN или иерархические методы. Это позволяет объединить диалоги в несколько групп, каждая из которых характеризуется схожей семантической структурой.
- D. *Выбор репрезентативных диалогов.* Для каждой сформированной группы (кластера) автоматически выбираются 1–2 наиболее репрезентативных диалога, которые могут быть интерпретированы как «типичные» или центральные элементы кластера. Эти диалоги предлагаются экспертам для ручной аннотации.
- E. *Создание минимального набора данных.* На основе вручную размеченных репрезентативных диалогов формируется небольшой, но семантически разнообразный начальный набор данных (*Seed*), который покрывает основные зоны распределения исходного корпуса.
- F. *Инициализация основного процесса.* После создания минимального набора данных запускается описанный ранее итеративный процесс генерации синтетических данных. Новый *Seed* используется в качестве отправной точки для обучения модели.

Такой подход позволяет эффективно компенсировать отсутствие заранее размеченных данных. Формируемый минимальный набор данных обеспечивает

необходимое начальное качество и разнообразие, что критически важно для корректной работы фреймворка. Преимущество данной методики заключается в том, что экспертам предлагается аннотировать наиболее характерные диалоги из каждого кластера, что минимизирует трудозатраты, одновременно повышая качество и обоснованность обучающих данных.

2.4. Архитектура фреймворка с использованием IDEF0

Для визуализации архитектуры фреймворка предложено использовать методологию IDEF0, которая позволяет наглядно представить функциональную структуру системы, её основные компоненты, взаимодействие между ними, а также входы, выходы, механизмы и управляющие воздействия. На основе математической постановки задачи была разработана серия диаграмм IDEF0, описывающих работу фреймворка.

В данном разделе представлена верхнеуровневая диаграмма (A0), отображающая основные элементы и взаимодействия фреймворка для создания обучающих наборов данных. Диаграмма описывает фреймворк как «чёрный ящик», включая его входы, выходы, механизмы и управляющие факторы.

Основное назначение фреймворка направлено на автоматическую генерацию синтетических обучающих наборов данных, используя неразмеченные диалоги и небольшой набор эталонных данных (*seed data*). Его основная цель — создать качественный обучающий набор пар «диалог — суммаризация» для обучения моделей суммаризации. На диаграмме A0 (см. Рисунок 2.3) представлены ключевые взаимодействия:

Входные данные:

- *Неразмеченные данные*. Основной входной поток, представляющий собой сырые диалоги, требующие обработки.
- *Эталонные данные (seed)*. Небольшой вручную размеченный набор данных, служащий для инициализации генерации.

Управляющие факторы:

- *Гиперпараметры*. Конфигурационные параметры фреймворка, такие как K (размер выборки), используемый энкодер и настройки LLM.
- *Политика фильтрации*. Набор правил для отбора данных, включая критерии релевантности и качество итоговых суммаризаций.

Механизмы:

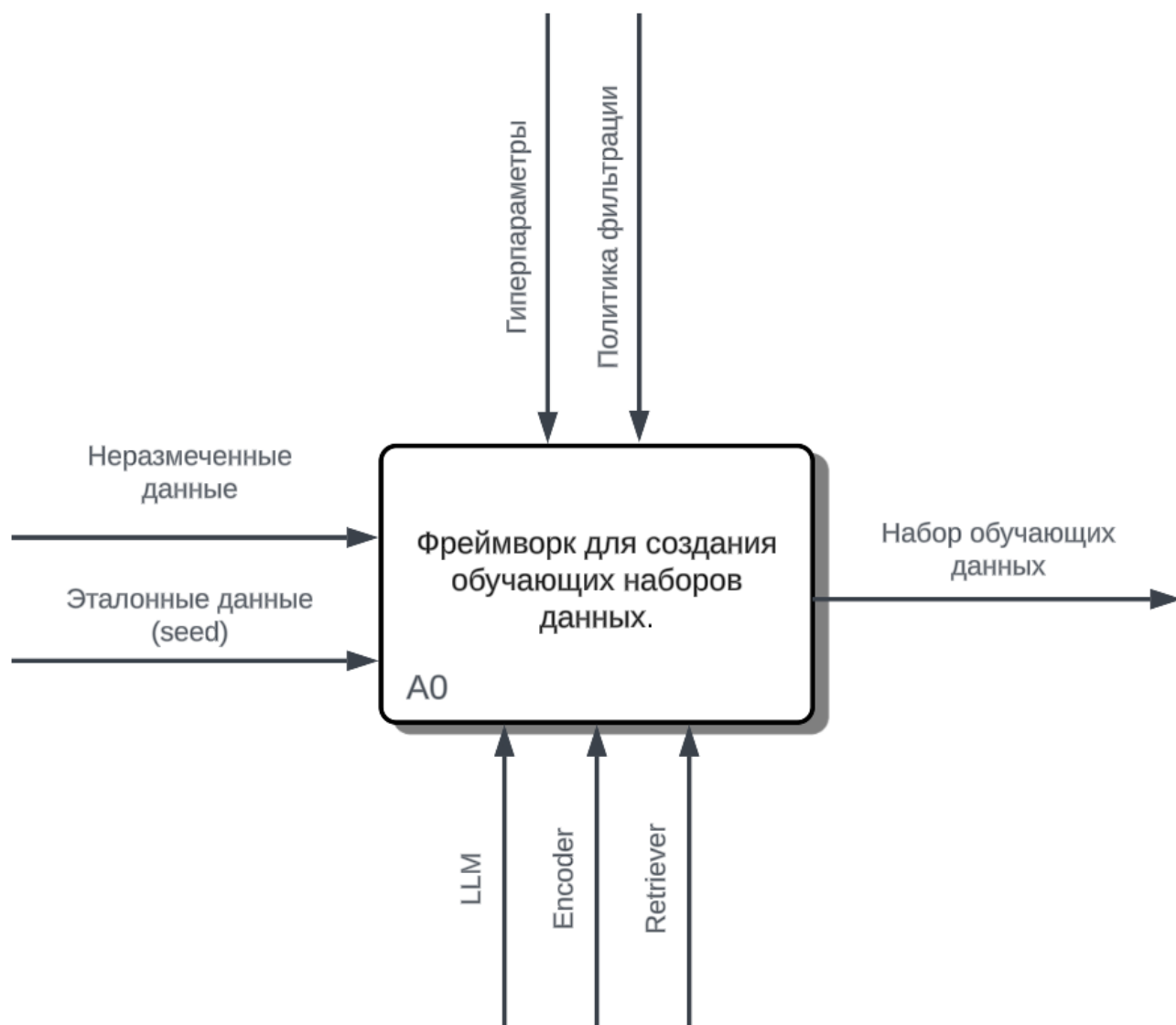


Рис.2.3. A0 диаграмма алгоритма в нотации IDEF0

- *LLM*. Крупная языковая модель, генерирующая суммаризации на основе сформированных промптов.
- *Encoder*. Модуль для преобразования диалогов в векторные представления.
- *Retriever*. Механизм поиска релевантных примеров в векторной базе данных.

Выходные данные:

- *Набор обучающих данных*. Итоговый результат работы фреймворка, содержащий как эталонные, так и автоматически сгенерированные пары «диалог — суммаризация».

Для детального представления процессов, происходящих внутри предложенного фреймворка, в нотации IDEF0 представлена декомпозиция верхнеуровневого блока A0 на отдельные этапы. Диаграмма A1 описывает внутренние шаги выпол-

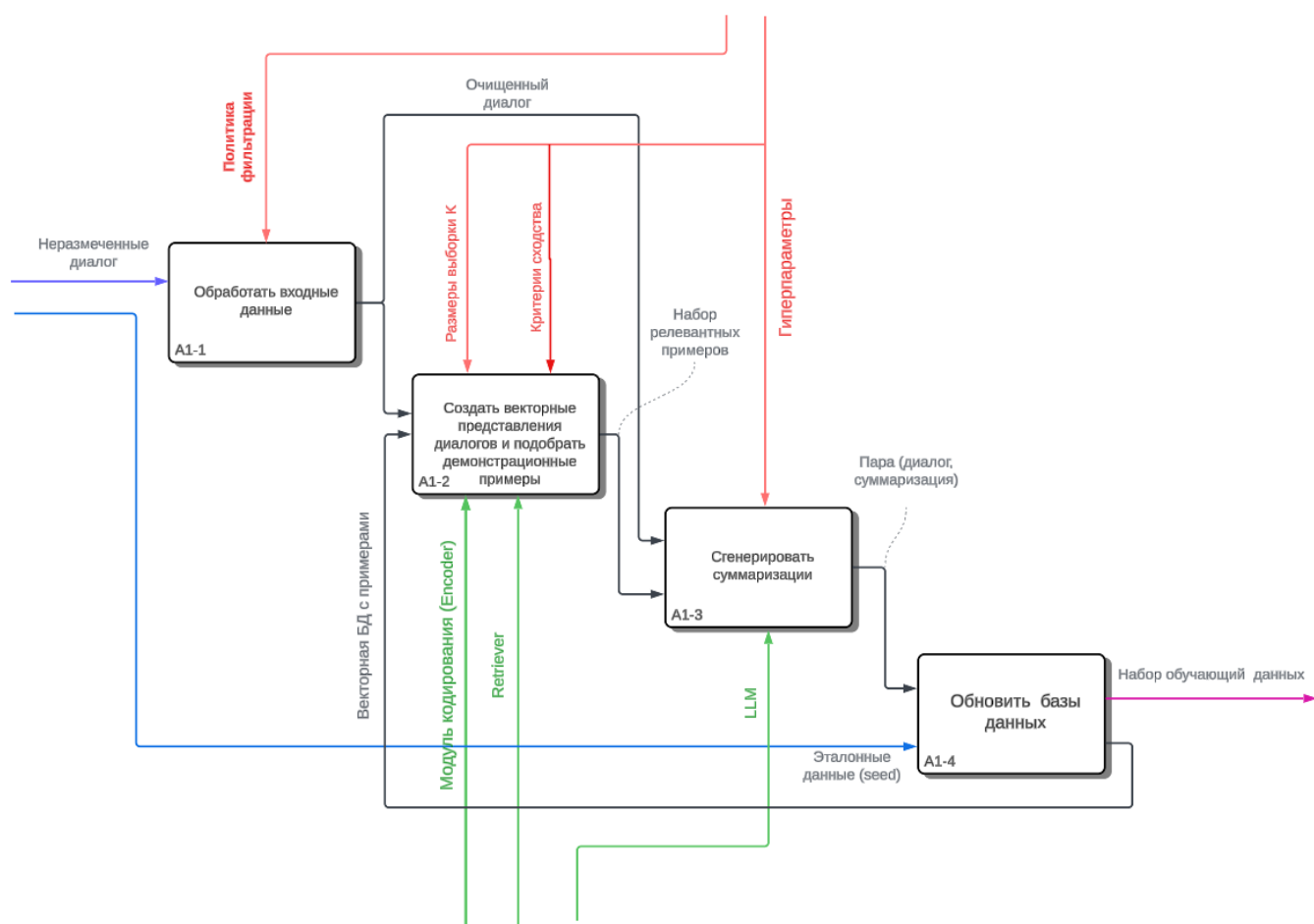


Рис.2.4. Детализированная IDEF0 диаграмма уровня A0

нения алгоритма, начиная с обработки входных данных и заканчивая обновлением базы синтетически созданных обучающих данных. Каждый этап сопровождается входными и выходными данными, управляющими факторами и вспомогательными механизмами.

- (1) A1-1. Предобработка данных. Этот этап отвечает за нормализацию и подготовку входных диалогов. Сырые диалоги проходят процессы очистки, устранения дублирующихся или нерелевантных данных, токенизации, а также приведения к унифицированному формату.

Входные данные:

- Неразмеченные диалог (d_i).

Выходные данные:

- Очищенный диалог (d_i).

Механизмы:

- Инструменты предпроцессинга текста.

Управляющие факторы:

- Политика предобработки данных.

(2) A1-2. Создание векторных представлений и поиск демонстрационных примеров. На данном этапе происходит вычисление эмбедингов для очищенных диалогов и выбор релевантных примеров из базы данных E .
Входные данные:

- Очищенные диалог (d_i).

Выходные данные:

- Набор релевантных примеров для каждого диалога (d_i).

Механизмы:

- Модуль кодирования (например, USER или другой подход для генерации эмбедингов).
- Ретривер ($\mathcal{R}(e_i, E, K)$) для поиска ближайших соседей на основе метрики сходства (косинусная близость или евклидово расстояние).
- Векторная база данных (E), хранящая эмбединги эталонных и ранее обработанных данных.

Управляющие факторы:

- Размер выборки K для ICL.
- Критерии сходства релевантных примеров (пороговые значение для метрики сходства).

(3) A1-3. Генерация суммаризаций с использованием LLM. Основной этап, в котором происходит формирование текстового промпта и использование LLM для создания итоговых суммаризаций.

Входные данные:

- Очищенные диалог (d_i).
- Релевантные примеры для (d_i).

Выходные данные:

- Пара «диалог — суммаризация» ((d_i, s_i)).

Механизмы:

- Предобученная крупная языковая модель (M_{LLM}).

Управляющие факторы:

- Конфигурация LLM.

(4) A1-4. Обновление базы данных. Заключительный этап итерации, на котором происходит обновление базы эмбедингов E и обучающего набора S .

Входные данные:

- Пара «диалог — суммаризация» $((d_i, s_i))$.
- Эталонные данные (единовременный процесс добавления) (S_{seed}) .

Выходные данные:

- Обновлённая база данных (E_{new}) .
- Итоговый обучающий набор (S_{new}) .

На основе описанных шагов формируется замкнутый итеративный процесс, который постепенно расширяет и улучшает обучающий набор данных, минимизируя зависимость от ручной разметки. Декомпозиция верхнеуровневой диаграммы А0 на этапы А1 демонстрирует внутреннюю логику работы фреймворка и взаимосвязь его компонентов.

2.5. Потенциальные клиенты и пользователи

Современные решения в области обработки естественного языка (NLP) постоянно сталкиваются с потребностью в актуальных обучающих данных, особенно когда меняются темы, домены и сценарии общения. Предлагаемый фреймворк, ориентированный на автоматическую генерацию синтетических пар «(диалог, итоговая суммаризация)» на основе небольшого seed-набора, призван решать проблему нехватки таких данных. Ниже рассмотрены основные категории клиентов и пользователей, для которых этот подход может оказаться наиболее полезным.

2.5.1. Компании и разработчики виртуальных ассистентов

Проблематика. Компании, разрабатывающие чат-ботов, регулярно сталкиваются с необходимостью расширения их функционала: добавления новых тем, продуктов и сервисов, отражающих изменяющиеся запросы пользователей. Традиционная ручная разметка данных требует значительных ресурсов, увеличивает стоимость и замедляет внедрение обновлений.

Как помогает фреймворк:

- *Ускорение внедрения:* Фреймворк генерирует обучающие наборы данных «(диалог, итоговая суммаризация)» на основе ограниченного числа seed-примеров, автоматически адаптируясь к новым темам.

- *Снижение издержек:* Даже небольшой seed-набор, размеченный неспециалистами, позволяет модели через ICL распространить логику аннотации на весь набор данных.
- *Гибкость обновлений:* Добавление новых сценариев и тем требует лишь минимального количества дополнительных примеров, что устраняет необходимость сложных скриптов или reward-моделей.

2.5.2. Колл-центры и службы поддержки клиентов

Колл-центры ежедневно обрабатывают значительный объём звонков и сообщений от клиентов, что порождает несколько ключевых трудностей:

- *Сложности в аналитике и обучении:* Ручная разметка для анализа обращений (выделение ключевых тем, составление полезных ответов) замедляет внедрение автоматизированных решений и увеличивает нагрузку на сотрудников.
- *Частая адаптация сценариев:* Новые продукты, маркетинговые кампании и изменения условий обслуживания требуют регулярного обновления данных для корректной работы автоматизированных систем.

Как помогает фреймворк:

- *Автоматизация формирования данных:* На основе небольшого seed-набора фреймворк генерирует обучающие пары «(диалог, итоговая выжимка)», которые могут быть использованы для: - обучения систем NLU, анализирующих запросы клиентов; - составления аналитических обзоров бесед для оценки качества обслуживания; - подготовки обучающих материалов (например, FAQ) для новых сотрудников.
- *Гибкая адаптация:* При появлении новых продуктов или сценариев достаточно разметить несколько эталонных примеров, после чего фреймворк автоматически обновит набор данных, минимизируя время и затраты.

Преимущества:

- *Снижение затрат:* Уменьшается потребность в найме дополнительных аннотаторов, так как система автоматизирует процесс разметки данных, используя небольшой seed-набор.
- *Быстрое внедрение изменений:* Новые сценарии обработки обращений могут быть адаптированы за счёт разметки нескольких эталонных примеров, что ускоряет обновление моделей.

- *Улучшение качества:* Более точные обучающие наборы позволяют повысить производительность автоматических систем и улучшить качество взаимодействия с клиентами.

2.5.3. Аналитические платформы

Проблематика. Аналитические платформы, работающие с диалогами, часто сталкиваются с уникальной терминологией и процессами, требующими специализированного подхода. Например, медицинские чаты обрабатывают данные о симптомах и диагнозах, юридические платформы анализируют нормативные акты, а в бизнес-аналитике ключевую роль играют совещания и отчёты. Классические общие датасеты здесь малоэффективны, а создание новых требует значительных затрат на привлечение профильных экспертов.

Как помогает фреймворк:

- *Автоматизация аннотации:* модель создаёт пары «(диалог, итоговая суммаризация)» даже для специализированных областей, используя ограниченный seed-набор с тщательно размеченными примерами.
- *Сокращение участия экспертов:* вместо масштабной аннотации профильные специалисты верифицируют небольшое количество seed-примеров, после чего модель распространяет логику на весь корпус.
- *Гибкость в адаптации:* модель учитывает доменные особенности и новый контекст, извлекая структуру резюме из предоставленных примеров.

Преимущества:

- *Обновляемость:* появление новых терминов, продуктов или регуляций легко интегрируется в модель путём добавления нескольких новых примеров в seed-набор.
- *Скорость внедрения:* система оперативно адаптируется к изменениям домена, снижая время на ручную аннотацию.
- *Экономия ресурсов:* специалисты тратят время только на ключевые кейсы, остальное создаётся автоматически, что значительно снижает затраты.

2.5.4. Научно-исследовательские организации и лаборатории NLP

Проблематика. Научным коллективам и лабораториям NLP часто требуется формировать собственные датасеты для изучения новых аспектов диалоговой суммаризации. Публичные наборы данных зачастую не соответствуют требованиям

по тематике или формату, а ручная разметка специфических диалогов требует значительных временных и ресурсных затрат.

Как помогает фреймворк:

- *Быстрое создание новых корпусов:* достаточно выбрать небольшое ядро (seed) и аннотировать его аспирантам или сотрудникам, после чего фреймворк автоматически синтезирует остальную часть датасета.
- *Упрощение экспериментов:* LLM адаптируется к предоставленным примерам, автоматически перенимая стилистику и формат аннотации. Это устраняет необходимость в ручном написании политик суммаризации и сложных reward-моделей.

Преимущества:

- *Ускоренный цикл исследований:* любой тип диалогов (например, научные интервью или конференционные обсуждения) может быть оперативно превращён в набор «(диалог, итоговая суммаризация)».
- *Экономия человеко-часов:* эксперты и исследователи сосредотачиваются на анализе данных и проверке гипотез, а не на рутинной аннотации.

Во всех вышеперечисленных сценариях фреймворк снимает барьер, связанный с острой нехваткой вручную размеченных диалогов. Он даёт возможность даже неквалифицированным аннотаторам сформировать небольшой seed - набор, который становится для крупной языковой модели «неявным руководством» к тому, как обрабатывать диалоги. Это упрощает и ускоряет обучение и дообучение моделей суммаризации в постоянно меняющихся условиях, позволяя избегать сложных reward-моделей и долгой ручной разметки всего массива данных.

2.5.5. Пути внедрения фреймворка

Разные компании и исследовательские коллективы могут использовать предложенный фреймворк в зависимости от их существующей инфраструктуры и технологических потребностей. Ниже представлены основные стратегии внедрения:

- Фреймворк подключается как модуль, который регулярно обрабатывает неразмеченные диалоги и возвращает пары «(диалог, суммаризация)». Автоматическое пополнение обучающего корпуса позволяет увеличить покрытие сценариев и снизить нагрузку на ручную аннотацию, повышая эффективность работы NLP-систем.

- Сгенерированные данные могут быть напрямую использованы для обучения компактных seq2seq-моделей, предназначенных для «онлайн»-применения (например, на серверах компании). Такие модели становятся узконаправленными и адаптированными к конкретной предметной области, например, медицинским консультациям или юридическим вопросам.
- Для быстрого перехода к новому домену (например, сложные переговоры, HR-интервью, узкие технические консультации) можно сформировать небольшой начальный набор демонстрационных данных. После этого фреймворк итеративно создаст объёмный синтетический корпус, отражающий специфику данного домена. Высокая гибкость и экономия ресурсов. Достаточно предоставить несколько эталонных примеров, а фреймворк самостоятельно сформирует масштабный набор данных, адаптированных под новые требования.

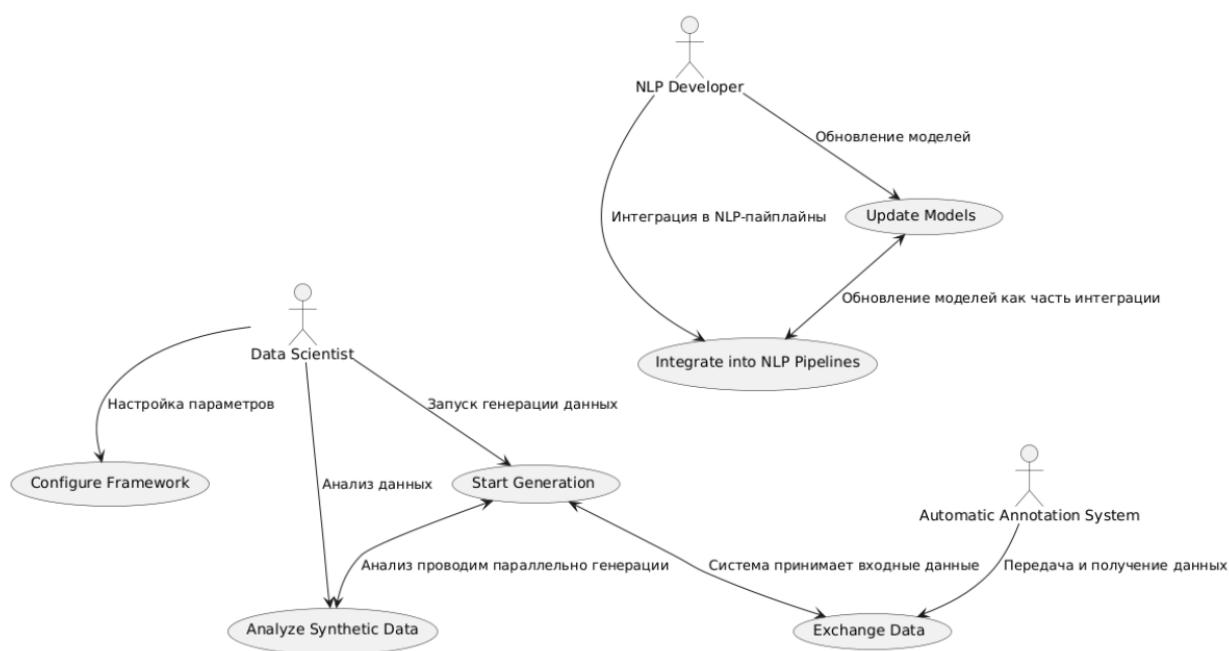


Рис.2.5. UML-диаграмма вариантов использования

Данный раздел описывает взаимодействие пользователей и систем с фреймворком в формате UML-диаграммы вариантов использования (Use Case). Основными участниками являются следующие роли:

А. Акторы:

- *Data Scientist*: Настраивает глобальные параметры (число итераций, критерий качества, K и другие), запускает процесс гене-

рации, анализирует результаты и при необходимости добавляет новые seed-примеры.

- *NLP Developer*: Интегрирует фреймворк в существующие пайплайны, обновляет внутренние компоненты (например, заменяет LLM или энкодер) для повышения точности и производительности, обеспечивает техническую поддержку и развёртывание.
- *Automatic Annotation System*: Передаёт фреймворку потоки неразмеченных диалогов, получает пары «(диалог, суммаризация)», использует их для обучения других моделей, а также может предоставлять обратную связь для улучшения фреймворка.

B. Use Cases:

- *Configure Framework*:
 - *Actor*: Data Scientist.
 - *Описание*: Настройка ключевых параметров запуска, таких как K (количество демонстраций), метрики качества, длина суммаризаций, а также выбор моделей (LLM, энкодера и других).
 - *Цель*: Адаптировать фреймворк под конкретные требования (быстрая генерация, высокое качество и т.д.).
 - *Результат*: Фреймворк сконфигурирован и готов к запуску генерации.
- *Start Generation*:
 - *Actor*: Data Scientist.
 - *Описание*: Запуск итеративного процесса генерации синтетических данных на основе настроенных параметров. Фреймворк получает неразмеченные диалоги и формирует пары «(диалог, суммаризация)».
 - *Цель*: Создать или дополнить обучающий набор автоматически, с минимальным ручным вмешательством.
 - *Результат*: Автоматически созданный обучающий набор данных.
- *Analyze Synthetic Data*:
 - *Actor*: Data Scientist.

- *Описание:* Проверка сгенерированных данных на корректность, отсутствие ошибок, анализ метрик (например, ROUGE, BLEU). Возможность добавления новых seed-примеров для улучшения качества.
- *Цель:* Убедиться в соответствии итоговых данных требованиям задачи.
- *Результат:* Улучшенные настройки фреймворка или добавление новых seed-примеров.
- *Integrate into NLP Pipelines:*
 - *Actor:* NLP Developer.
 - *Описание:* Интеграция фреймворка в существующие NLP-пайплайны для автоматической разметки данных. Результаты могут быть использованы для обучения, валидации или других задач.
 - *Цель:* Автоматизация процесса разметки в существующей экосистеме.
 - *Результат:* Интеграция фреймворка в существующий рабочий процесс.
- *Update Models:*
 - *Actor:* NLP Developer.
 - *Описание:* Замена LLM или энкодера при обнаружении более продвинутых моделей для повышения точности и производительности.
 - *Цель:* Поддерживать высокое качество работы фреймворка.
 - *Результат:* Обновлённые внутренние компоненты фреймворка.
- *Exchange Data:*
 - *Actor:* Automatic Annotation System.
 - *Описание:* Передача неразмеченных диалогов во фреймворк и получение сгенерированных данных для обучения, валидации или других задач.
 - *Цель:* Непрерывное пополнение обучающего корпуса.
 - *Результат:* Постоянное обновление датасетов с минимальным ручным вмешательством.

С. Взаимосвязи:

- *Analyze Synthetic Data <-> Start Generation*: Анализ может проводиться параллельно с генерацией или после каждой итерации.
- *Update Models <-> Integrate into NLP Pipelines*: Обновление моделей может быть частью расширения процесса интеграции.
- *Exchange Data <-> Start Generation*: Передача данных может быть встроена в процесс генерации или выполняться независимо.

2.6. Выводы

Во второй главе была проведена разработка методики и фреймворка для суммаризации диалогов, основанного на интеграции современных методов обработки естественного языка. Рассмотрены теоретические основы задачи, включая её формальную постановку, определение входных данных, критериев качества и ограничений, что позволило сформулировать основные требования к разрабатываемому решению.

В рамках главы предложен итеративный фреймворк, сочетающий механизмы поиска релевантных примеров и использования их в качестве демонстрационных данных для модели. Это позволяет значительно повысить точность и согласованность сгенерированных суммаризаций. Методика направлена на минимизацию потребности в ручной разметке за счёт автоматического формирования синтетических обучающих данных, что делает её пригодной для масштабирования в различных предметных областях. В следующих главах будет рассмотрена практическая реализация методики и проведена её экспериментальная проверка.

ГЛАВА 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

Данная глава посвящена описанию структуры и архитектуры фреймворка, разработанного для автоматической генерации обучающих данных в задачах диалоговой суммаризации. Она охватывает ключевые аспекты реализации, включая модульный подход, взаимодействие компонентов, выбор технологий и алгоритмов.

В первой части главы представлена общая структура фреймворка, включая его функциональные модули и способы их взаимодействия. Далее обсуждается подготовка данных, включающая выбор корпуса, предобработку и возможное

использование кластеризации для формирования начального набора данных (seed-набора).

Заключительные разделы главы посвящены архитектурным решениям, организации кода и возможностям расширения фреймворка для интеграции новых технологий или применения в различных доменах.

3.1. Общая структура и архитектура фреймворка

Фреймворк предназначен для автоматической генерации обучающих данных, необходимых для диалоговой суммаризации. Его основная задача — упростить процесс расширения LLM, исключая необходимость ручного создания моделей вознаграждения и написания сложных правил. Фреймворк автоматизирует ключевые этапы обработки данных: от предобработки до формирования итоговых аннотаций.

Архитектура фреймворка строится на модульном подходе, где каждый компонент отвечает за строго определённую функцию. Такой подход обеспечивает гибкость системы, позволяя заменять или обновлять отдельные модули без значительных изменений в общей структуре.

В рамках работы фреймворка данные проходят через несколько этапов обработки. На вход подаются неразмеченные диалоги, которые затем обрабатываются следующим образом:

1. На первом этапе данные проходят предобработку, включающую очистку текста, токенизацию и приведение к стандартному формату. Это позволяет подготовить сырой корпус для дальнейшей работы.

2. Затем обработанные данные передаются в модуль генерации эмбедингов. Здесь с использованием выбранного энкодера (например, Sentence-BERT или USER-BGE) создаются векторные представления текстов. Эти эмбединги сохраняются в базе данных для дальнейшего поиска.

3. После этого модуль поиска релевантных демонстраций использует сохранённые эмбединги для поиска похожих диалогов. Это позволяет подобрать примеры, которые могут быть использованы при генерации аннотаций.

4. Модуль взаимодействия с языковой моделью формирует запросы (промты) на основе найденных демонстраций. Промты передаются в LLM (например, GPT-3.5 или Mistral-7b), которая генерирует итоговую суммаризацию диалогов.

5. На последнем этапе база данных обновляется новыми парами «(диалог, суммаризация)», которые могут быть использованы для обучения или тестирования моделей.

Во фреймворке заложено несколько ключевых подсистем, каждая из которых решает собственную задачу, но совместно они формируют замкнутый цикл обработки диалогов: от поступления неразмеченных данных до получения итоговых суммаризаций и оценки их качества. Сначала сырые тексты проходят предобработку в модуле DataCleaner, где удаляются HTML-теги, лишние пробелы, а при необходимости выполняются токенизация и лемматизация. После этого полученные упорядоченные диалоги направляются в модуль генерации эмбедингов EmbeddingGenerator, который с помощью выбранного энкодера (например, Sentence-BERT или USER-BGE) формирует векторные представления. Эти вектора сохраняются в векторной базе (Milvus, Faiss и др.) для быстрого поиска.

Следующий важный шаг — поиск релевантных демонстраций посредством модуля RAGRetriever, который выбирает диалоги, наиболее похожие на текущий входной диалог. Такие демонстрации затем подготавливаются для ICL, передаваясь в модуль взаимодействия с крупной языковой моделью LLMInterface. Здесь, на основе найденных примеров и исходного текста, формируется промпт (с указанием желаемого формата ответа или структуры выжимки), а сама LLM (GPT-3.5, Mistral-7b, Llama2 и т. д.) генерирует итоговую суммаризацию.

По завершении генерации вступает в силу модуль QualityAssessment. Он анализирует полученные результаты: вычисляет выбранные метрики (ROUGE, BERTScore, возможно GPT-based оценку) и фиксирует значения в базе данных или журнале событий. В зависимости от обнаруженных проблем (например, слишком низкие оценки на отдельных диалогах) система может настроить стратегию отбора демонстраций или параметры промпта при последующих итерациях. Заключительная фаза — обновление центрального хранилища новыми парами «Диалог → Суммаризация», чтобы они могли использоваться в будущем как дополнительные примеры для модели. Таким образом, все компоненты (DataCleaner, EmbeddingGenerator, RAGRetriever, LLMInterface, QualityAssessment и модуль обновления базы) объединены в единый конвейер, поддерживающий как итеративное наращивание обучающего пула, так и постоянный мониторинг качества генерируемых суммаризаций. Данная архитектура придаёт фреймворку гибкость и позволяет эффективно работать как с небольшими наборами данных, так и с крупномасштабными корпоративными диалогами в производственных условиях.

Фреймворк обеспечивает плавное взаимодействие между модулями. Данные передаются в стандартизированном формате, таком как JSON, а для хранения и извлечения эмбедингов используются специализированные инструменты, такие как Milvus. Такой подход снижает зависимость от конкретных технологий и упрощает интеграцию новых моделей и методов.

Модульность позволяет системе быть гибкой и масштабируемой. Например, можно легко заменить энкодер (на GPT-4 или другой метод) или интегрировать новую векторную базу данных (например, Faiss вместо Milvus). Поддержка стандартизированного интерфейса между компонентами позволяет минимизировать затраты на адаптацию при изменении архитектуры.

На диаграмме представлены ключевые компоненты системы, их роли и взаимосвязи:

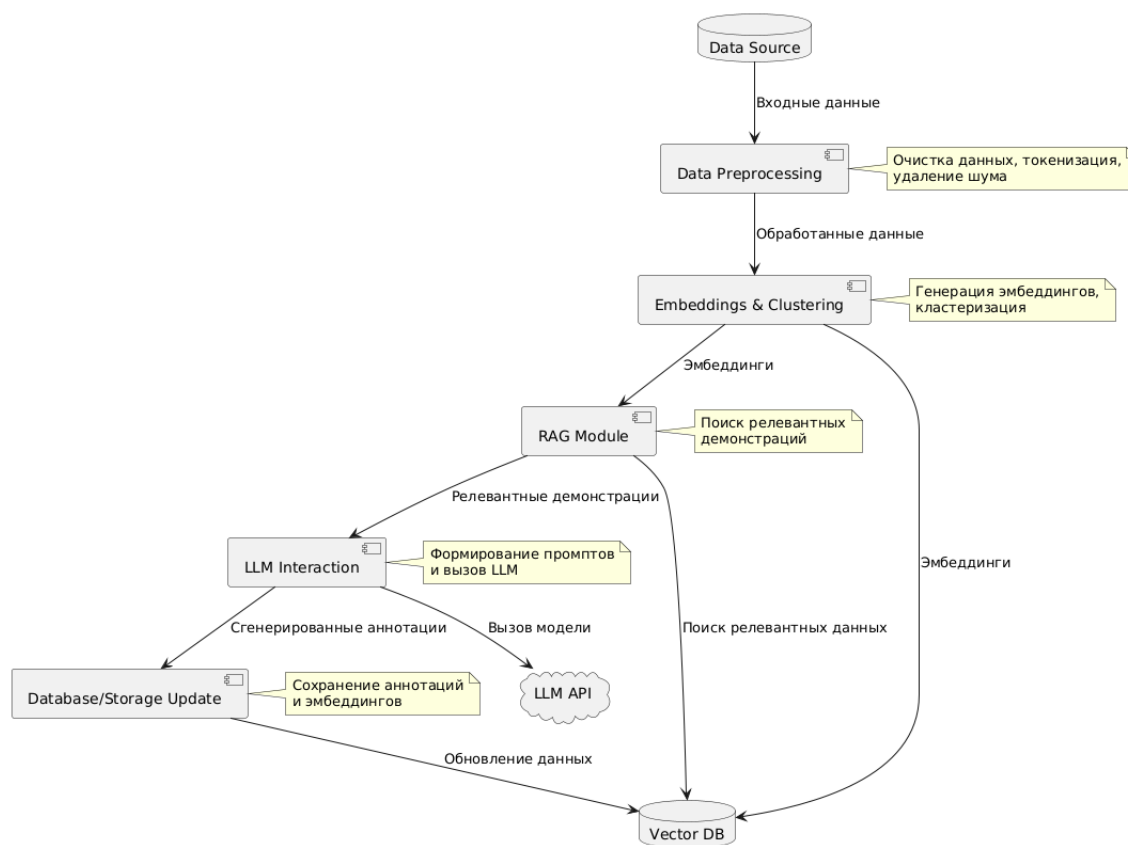


Рис.3.1. UML-диаграмма компонентов

A. Data Preprocessing (предобработка данных):

- *Описание:* Этот компонент отвечает за начальную обработку и подготовку входных данных перед генерацией эмбедингов. Включает в себя очистку данных от шума, токенизацию, удаление

специфичных символов и приведение текстов к стандартному формату.

- *Взаимодействие*: Принимает входные данные из источника данных (например, файлы диалогов), обрабатывает и передает обработанные данные в компонент *Embeddings & Clustering* для генерации эмбеддингов.

B. *Embeddings & Clustering* (генерация векторных представлений, при необходимости кластеризация):

- *Описание*: Этот компонент отвечает за создание векторных представлений диалогов и суммаризаций с использованием выбранного энкодера (например, USER-BGE или Sentence-BERT). После генерации эмбеддингов может проводиться кластеризация для группировки похожих диалогов.
- *Взаимодействие*: Принимает обработанные данные от компонента *Data Preprocessing*, генерирует эмбеддинги с помощью выбранного энкодера и сохраняет их в базу данных векторных представлений.

C. *RAG Module* (поиск релевантных демонстраций):

- *Описание*: Модуль *RAG* отвечает за поиск релевантных демонстраций на основе созданных эмбеддингов. Использует векторную базу данных (например, Milvus) для эффективного поиска и извлечения похожих примеров, которые могут быть использованы для дальнейшей обработки.
- *Взаимодействие*: Получает эмбеддинги от *Embeddings & Clustering*, проводит поиск похожих демонстраций в базе данных Milvus и передает результаты в *LLM Interaction* для генерации аннотаций.

D. *LLM Interaction* (формирование промптов, вызов LLM):

- *Описание*: Этот компонент отвечает за взаимодействие с *Language Model (LLM)*, таким как Mistral-7b, для генерации синтетических аннотаций. Формирует промпты на основе найденных релевантных демонстраций и текущего диалога, передавая их в LLM для получения суммаризации.

- *Взаимодействие*: Получает результаты поиска демонстраций от *RAG Module*, формирует промпты для LLM, вызывает LLM для генерации суммаризации и возвращает результаты.

Е. Database/Storage Update (обновление базы эмбеддингов и сгенерированных аннотаций):

- *Описание*: Этот компонент отвечает за обновление базы данных, где хранятся эмбеддинги диалогов и сгенерированные аннотации. Включает в себя добавление новых пар (диалог, суммаризация) в базу данных и обновление эмбеддингов при необходимости.
- *Взаимодействие*: Получает сгенерированные аннотации от *LLM Interaction*, сохраняет их в базу данных вместе с соответствующими эмбеддингами и обеспечивает актуализацию данных.

Система разработана с учётом возможности замены и расширения компонентов для обеспечения гибкости и масштабируемости:

- *Энкодеры*: Возможность интеграции различных энкодеров (например, Sentence-BERT или USER-BGE) с минимальными изменениями в основном коде благодаря модульной структуре и абстракции доступа к данным.
- *Векторные базы данных*: Лёгкость перехода от одной векторной базы данных (например, Faiss) к другой (например, Milvus) благодаря унифицированному интерфейсу и структурированному обмену данными между компонентами.
- *LLM*: Возможность подключения различных *Language Models (LLM)* (например, GPT-3.5, LLAMA, Alпаса) с настройкой API-ключей и параметров вызова для выбора модели в зависимости от требований задачи.

Каждый компонент системы имеет чётко определённую роль и взаимодействует с другими компонентами через структурированные форматы данных и API:

- *Data Preprocessing*: Подготовленные данные передаются в блок *Embeddings & Clustering* для генерации эмбеддингов.
- *Embeddings & Clustering*: Сгенерированные эмбеддинги используются модулем *RAG* для поиска и извлечения релевантных демонстраций.
- *RAG Module*: Возвращает найденные демонстрации в *LLM Interaction* для генерации итоговых суммаризаций.

- *LLM Interaction*: Формирует промпты для LLM на основе найденных демонстраций, вызывает LLM для генерации суммаризации и возвращает результаты.
- *Database/Storage Update*: Отвечает за хранение и обновление данных в базе, обеспечивает актуальность эмбедингов и сгенерированных аннотаций.

Эта архитектура обеспечивает понятность взаимодействия между компонентами, поддерживаемость системы и лёгкость внесения изменений или добавления новых функциональных возможностей.

3.2. Подготовка и предобработка данных

Перед проведением кластеризации диалогов необходимо выполнить их предварительную обработку. Этот процесс играет ключевую роль в обеспечении качества итоговых кластеров и суммаризаций, так как исходные данные могут содержать шум, неформатированные конструкции и неравномерные структуры. Данный раздел описывает последовательность этапов обработки данных, начиная с очистки текстов, создания эмбедингов, масштабирования, применения различных алгоритмов кластеризации, оценки их качества и визуализации результатов.

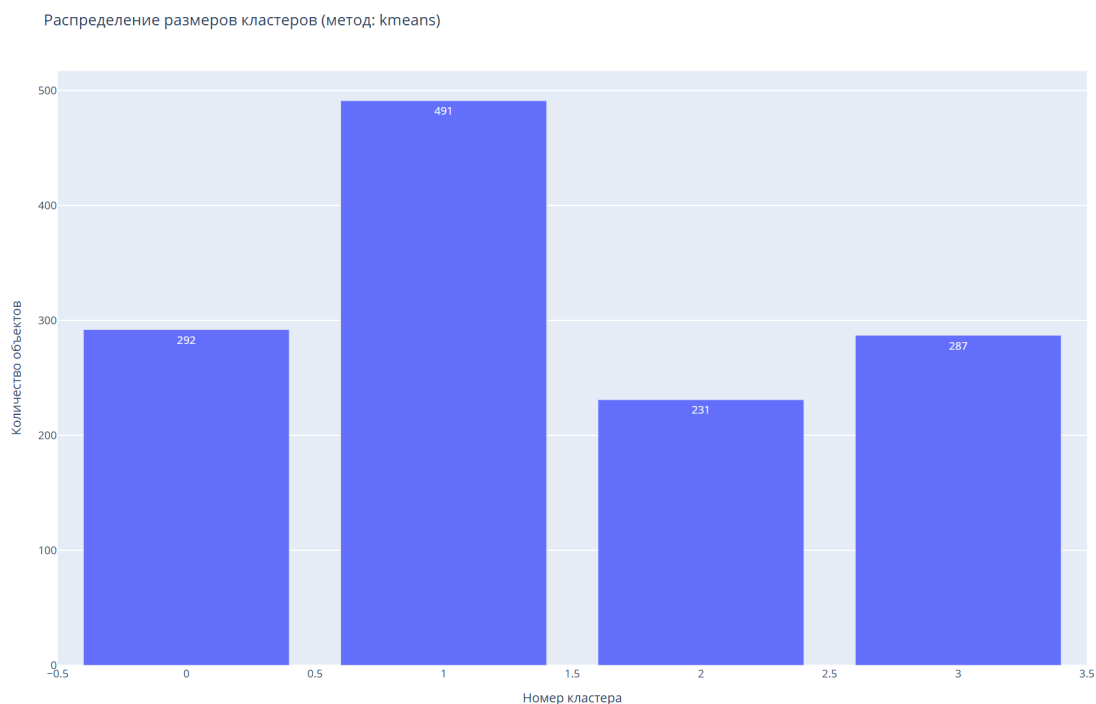
Исходные диалоги, как правило, содержат различные артефакты, такие как HTML-теги, специальные символы, дополнительные пробелы и нестандартные обозначения имен. Для приведения текстов к единому формату используется класс `DialoguePreprocessor`, реализованный в файле `data_preprocessing.py`.

Процесс очистки включает в себя несколько ключевых шагов:

- Удаление специальных конструкций, таких как `#Имя#:`, которые заменяются на стандартные обозначения говорящего, например, `Имя:`.
- Устранение лишних пробелов и символов форматирования.
- Удаление пустых строк и лишних переносов.

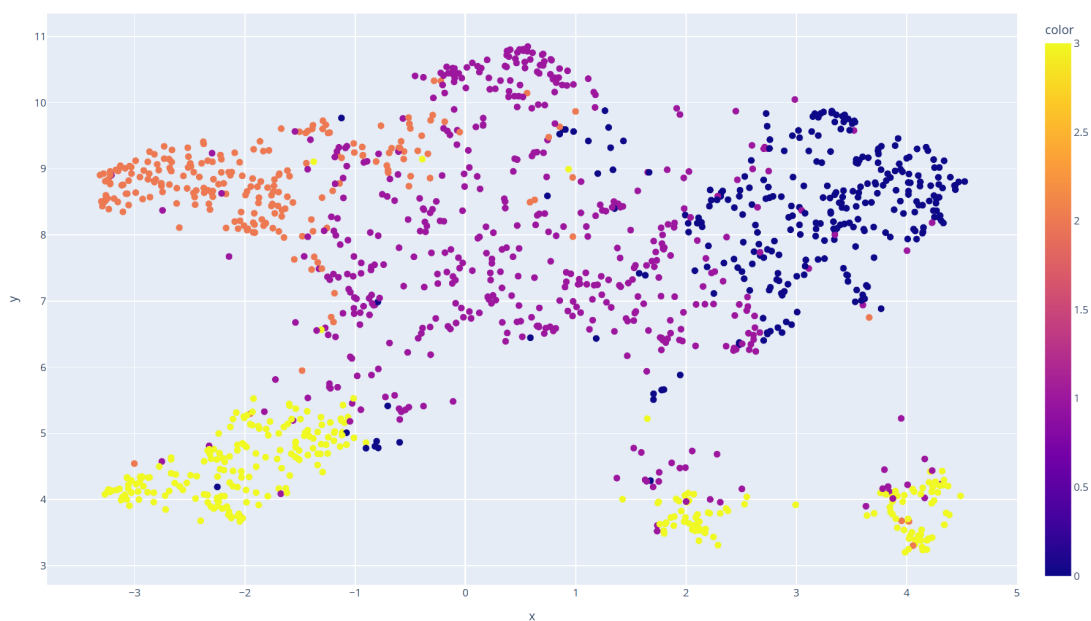
После очистки проводится токенизация с использованием модели `transformers`, позволяющей разбивать текст на токены и приводить их к заданной максимальной длине. В данном случае используется параметр `max_length=512`, что гарантирует, что каждый диалог не превысит установленного лимита. Тексты, превышающие данный порог, обрезаются, а короткие заполняются дополнительными токенами.

После очистки текстов следующим шагом является их преобразование в числовые представления (эмбединги). Данный процесс выполняется с исполь-



а) Распределение размеров кластеров

Визуализация кластеров. Метод: kmeans



б) Двумерная визуализация кластеров

Рис.3.2. Результат работы модуля интерактивной визуализации кластеров данных

зованием предобученных трансформерных моделей, таких как Sentence-BERT или USER-BGE, которые позволяют кодировать диалоги в многомерное векторное пространство.

Эмбединги представляют собой числовые массивы, в которых отражены смысловые особенности диалогов. Это необходимо для дальнейшего анализа,

так как векторное представление позволяет сравнивать семантическую близость различных текстов, находить схожие структуры и формировать кластеры.

В коде данный процесс реализован в файле `run_clustering.py`, где загруженные эмбединги передаются в класс `DialogClustering` для последующего анализа.

Перед выполнением кластеризации векторы эмбедингов проходят дополнительную обработку с целью приведения их к единому масштабу. Это особенно важно, если используются алгоритмы, чувствительные к масштабу входных данных, например, `KMeans`.

Для нормализации данных в коде используется `StandardScaler`, который стандартизирует эмбединги, приводя их к нулевому среднему и единичной дисперсии. Данный процесс описан в методе `_preprocess_embeddings` класса `DialogClustering`, реализованном в файле `clustering.py`.

После подготовки эмбедингов выполняется процесс кластеризации, цель которого — объединить схожие диалоги в группы. В данном проекте используется несколько различных методов кластеризации:

- *KMeans* — классический метод, разделяющий данные на фиксированное число кластеров. Оптимальное количество кластеров определяется с использованием метода “локтя” и силуэтного анализа.
- *Agglomerative Clustering* — иерархический метод, объединяющий схожие объекты в группы. Подходит для выделения вложенных структур.
- *Gaussian Mixture Model (GMM)* — вероятностный метод, моделирующий данные как смесь нескольких гауссовских распределений.
- *DBSCAN* и *HDBSCAN* — алгоритмы, ориентированные на плотностное разделение данных. Позволяют выявлять сложные структуры и игнорировать выбросы.

Оптимальный метод выбирается автоматически на основе метрик качества кластеризации.

Для объективного сравнения алгоритмов и выбора наилучшего метода кластеризации используются следующие метрики:

- *Silhouette Score* — оценивает, насколько точки внутри одного кластера схожи друг с другом.
- *Davies-Bouldin Score* — отражает разделимость кластеров, чем ниже значение, тем лучше.
- *Calinski-Harabasz Score* — определяет плотность кластеров.

– *Stability* — измеряет устойчивость кластеризации при разбиении выборки.

Метрики рассчитываются в методе `_evaluate_clustering` класса `DialogClustering`. После автоматической встроенной в класс оценки всех методов выбирается алгоритм, обеспечивающий наилучшее разделение данных.

Для наглядного представления результатов кластеризации выполняется визуализация распределения данных.

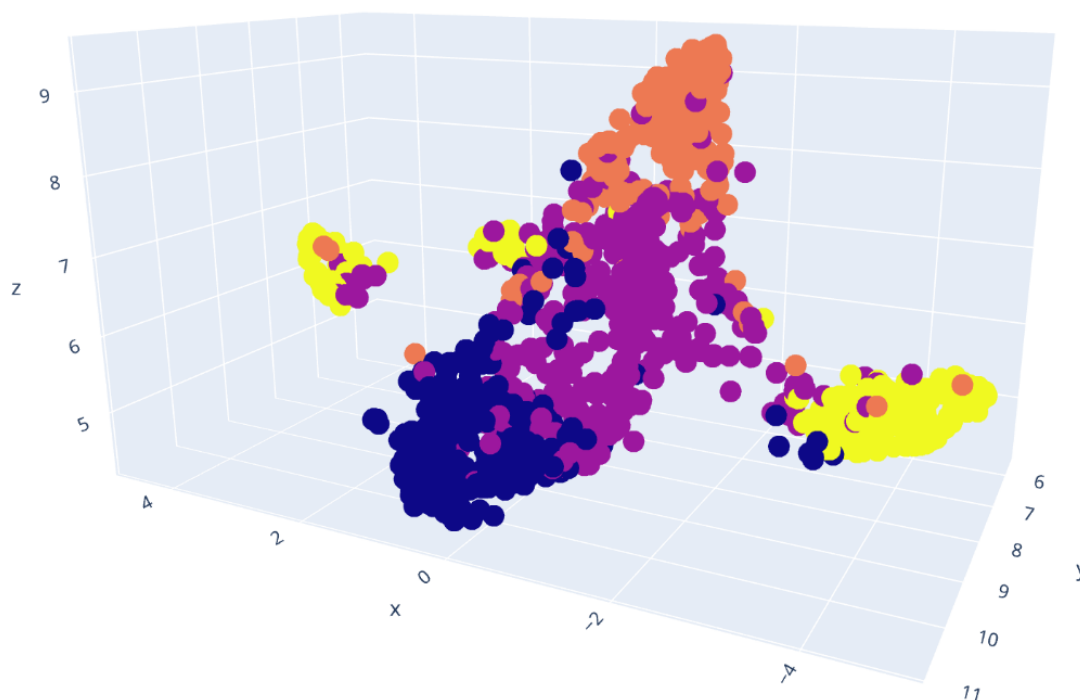


Рис.3.3. Трёхмерная интерактивная визуализация кластеров данных, полученная с использованием метода `visualize_clusters()`

На рисунке 3.3 представлена трёхмерная проекция кластеров, полученных в результате обработки данных. Используется метод понижения размерности UMAP, позволяющий уменьшить число измерений до 3D, сохраняя основные структуры в данных. Цвет точек соответствует различным кластерам.

На рисунке 3.2a представлена гистограмма, отражающая распределение размеров кластеров. Она показывает количество элементов в каждой группе, позволяя понять, какие кластеры являются основными, а какие могут представлять редкие случаи или выбросы.

На рисунке 3.2b представлена двумерная проекция кластеров, также полученная с использованием UMAP. Она позволяет выявить плотность расположения данных и возможные пересечения между кластерами.

Таким образом, этап предобработки данных включает очистку текстов, создание эмбеддингов, их нормализацию, кластеризацию, оценку качества и визуализацию. Использование нескольких методов кластеризации позволяет выбрать наиболее подходящий алгоритм, а визуализация даёт возможность анализировать структуру данных и принимать решения о дальнейшей обработке.

Этот метод полезен для предварительной фильтрации данных и позволяет быстро создать seed-набор, используя репрезентативные диалоги из различных кластеров. Это значительно упрощает задачу аннотации данных и улучшает общее качество обучения моделей.

3.3. Реализация ключевых модулей фреймворка

3.3.1. Генерация эмбеддингов

Эмбеддинги являются центральной частью фреймворка, так как они представляют семантическое содержание диалогов и обеспечивают необходимый базис для всех последующих операций, включая кластеризацию, отбор репрезентативных примеров и дальнейший анализ. Данный модуль выполняет преобразование текстов диалогов в их компактные векторные представления.

Фреймворк использует энкодеры для преобразования текстовых данных в векторные представления, обеспечивая высокую степень семантического соответствия между схожими текстами. В качестве основного энкодера выбран *USER-BGE* (*User Behavior Generated Embeddings*), который был эмпирически определён как наиболее эффективная модель для русского языка. Это обусловлено следующими характеристиками:

- *Адаптация под русский язык:* *USER-BGE* отличается превосходной производительностью при работе с русскоязычными текстами. Эта модель демонстрирует высокое качество генерации эмбеддингов для задач, связанных с русским языком, включая обработку контекстных диалогов.
- *Учёт пользовательского поведения:* Модель построена с учётом особенностей пользовательских взаимодействий, что делает её особенно эффективной в задачах, связанных с диалоговой суммаризацией и семантическим анализом.
- *Производительность на практике:* В экспериментальных условиях и реальных кейсах, в том числе при обучении моделей на корпусах русскоязыч-

ных данных, *USER-BGE* показал более высокую точность и адаптивность по сравнению с альтернативами, такими как *Sentence-BERT*.

USER-BGE был выбран благодаря его способности эффективно улавливать смысловые взаимосвязи в русскоязычных текстах и гарантировать качественное представление данных для последующих задач, таких как кластеризация и обучение моделей.

Процесс построен на трёх ключевых этапах:

- A. *Токенизация и предобработка.* Исходные тексты диалогов проходят этапы очистки и токенизации с использованием предобученных токенайзеров из библиотеки *Hugging Face Transformers*. Ограничение по длине текста (например, 512 токенов) гарантирует, что входные данные будут соответствовать архитектуре модели. Учитываются особенности диалогов, такие как разделение на реплики и удаление служебной информации.
- B. *Интеграция с моделью.* Выбранный энкодер (*USER-BGE*) вызывается через Python API библиотеки *Transformers*. Такой подход предоставляет гибкость в настройке модели и позволяет напрямую использовать методы генерации эмбеддингов масштабируемым способом, включая GPU-ускорение для крупных корпусов.
- C. *Хранение результатов.* Сгенерированные эмбеддинги диалогов сохраняются в виде массивов NumPy. Этот формат хранения оптимален для последующей обработки, таких как кластеризация, анализ схожести данных и вычислений.

Для ускорения обработки больших объёмов данных и повышения вычислительной эффективности реализован ряд оптимизаций:

- *Параллельная обработка данных:* Кодирование диалогов выполняется батчами, что минимизирует затраты памяти и позволяет использовать ресурсы GPU в полной мере.
- *Механизм "сегментной обработки":* Большие диалоги разбиваются на логические сегменты (реплики), обеспечивая более точное представление для длинных взаимодействий пользователей без потери контекстной информации.
- *Сжатие результатов:* Для хранения и передачи эмбеддингов используются дополнительные методы сжатия, такие как квантование векторов или преобразование в формат float16.

3.3.2. Модуль поиска демонстрационных примеров

В рамках задачи диалоговой суммаризации, модуль *RAG* обеспечивает поиск наиболее релевантных диалогов для **ICL** и формирование демонстраций, способствующих повышению точности итоговой выжимки. Ниже описана структура классов, алгоритм поиска и некоторые особенности реализации, ориентированные на гибкость и высокую производительность системы.

- *Семантическая релевантность*. Модуль *RAG* подбирает диалоги, близкие к текущему по смыслу, чтобы модель могла использовать подходящие примеры при генерации суммаризации.
- *Работа с низкоресурсными данными*. За счёт быстрого поиска релевантных демонстраций мы компенсируем малый объём размеченных диалогов.
- *Лёгкая интеграция с ICL*. Модуль возвращает топ-*k* диалогов, на базе которых формируется промпт «*Dialog* → *Summary*» для **ICL**.

Для удобства разработки и будущего масштабирования выделен специальный класс, условно назовём его «**RAGRetriever**», который инкапсулирует следующие аспекты:

- *Подключение к базе* (`connect_db()`): создание соединения с *Milvus* и необходимых коллекций;
- *Поиск похожих диалогов* (`retrieve_demonstrations()`): метод, принимающий на вход векторное представление диалога и возвращающий список релевантных примеров;
- *Сортировка и фильтрация* (`compute_similarity()`, `post_filter()`): вспомогательные методы, позволяющие реализовать дополнительную логику — например, сортировку по убыванию релевантности, отсеивание слишком длинных диалогов и т. д.;
- *Закрытие соединения* (`close_db()`): метод для корректного завершения работы с *Milvus*.

Данный подход (отдельный класс «**RAGRetriever**») облегчает замену *Milvus* на иные векторные движки (*Faiss*, *Weaviate*, *Pinecone*), сохраняя общий интерфейс и логику запроса.

Алгоритм поиска и некоторые особенности реализации, ориентированные на гибкость и высокую производительность системы.

- A. *Генерация эмбединга диалога*. С помощью выделенного `EncoderModel` (например, `Sentence-BERT` или `USER-BGE`) получаем вектор:

```
dialogue_embedding = encoder_model.encode(new_dialogue_text)
```

- В. *Запрос к базе Milvus*. Метод `retrieve_demonstrations(dialogue_embedding, k)` формирует запрос вида:

```
results = self.milvus_client.search(
    collection_name="dialogues_collection",
    data=[dialogue_embedding],
    param={ "metric_type": "IP", "params": {"nprobe": 10} },
    limit=k
)
```

Здесь "IP" (Inner Product) обычно соответствует косинусному сходству, если векторы нормированы.

- С. *Сортировка и отбор топ-k*. Полученный список `results` сортируется по полю `score`, и выбираются записи с наибольшим значением:

```
sorted_results = sorted(
    results,
    key=lambda r: r.score,
    reverse=True
)[:k]
```

- Д. *Дополнительная фильтрация (опционально)*. На этом шаге можно исключать:

- диалоги, выходящие за рамки нужного домена (например, только техническая поддержка);
- слишком короткие/длинные диалоги;
- дублирующиеся записи.

- Е. *Возврат набора демонстраций*. Отсортированные и при необходимости отфильтрованные результаты возвращаются, чтобы стать частью промпта.

Выбор косинусного сходства (*Inner Product* при нормированных векторах) оптимален для семантических эмбеддингов, однако при создании `RAGRetriever` можно подменить:

- *Метрики* — "L2", "HAMMING", "Jaccard" (при нужном типе вектора);
- *Векторное хранилище* — *Faiss*, *Weaviate* или *Pinecone*, меняя лишь логику подключения.

Таким образом, код остаётся гибким: если в будущем улучшится модель энкодера или изменится задача (тональный анализ, рекомендационные системы), достаточно скорректировать класс **RAGRetriever**.

- *Пул соединений с Milvus*: создан для снижения сетевых задержек при множестве параллельных запросов.
- *Асинхронность* (опционально): при большом объёме новых диалогов возможно использовать *asyncio* или *multiprocessing* для одновременной обработки нескольких векторных запросов.
- *Переиндексация* (re-indexing): если появляется большое количество новых размеченных диалогов или улучшается модель энкодера, можно запустить регулярную переиндексацию, чтобы база оставалась актуальной.
- *Post-Filtering* (на стороне Python): например, дополнительно сравнивать векторы методом `compute_similarity()` и удалять потенциальные конфликты или нерелевантные ответы (не только по домену, но и по длине, временным меткам и т. д.).

Благодаря модулю **RAG**, система диалоговой суммаризации способна автоматически подбирать **семантически близкие** диалоги и использовать их как демонстрации в ICL. Это особенно ценно при ограниченном количестве размеченных примеров и при необходимости быстро адаптировать модель (без переобучения) к новым темам или доменам. Гибкая смена метрик, возможность дополнительной фильтрации и удобный класс **RAGRetriever** делают подход масштабируемым и удобным в промышленной эксплуатации.

Данная реализация обладает следующими ключевыми преимуществами:

- *Поддержка нескольких метрик*: Можно гибко менять метрику сходства в зависимости от задач.
- *Масштабируемость*: Использование *Milvus* позволяет работать с миллионами векторных представлений без значительных потерь производительности.
- *Простая адаптация*: Код легко модифицируется для работы с альтернативными базами данных, такими как *Faiss* или *Pinecone*.
- *Гибкость K-значений*: Возможность задания точного числа релевантных соседей (*K*) повышает универсальность подхода.

3.3.3. Генерация синтетических аннотаций

Для реализации модуля генерации синтетических аннотаций была выбрана модель **Mistral-7b**. *Mistral-7b* представляет собой современную языковую модель, известную своей способностью к высококачественной генерации текстовых данных, что делает её идеальной для задачи синтетической аннотации диалогов. Основными преимуществами модели являются:

- Обучение на объёмах данных большого масштаба, что позволяет модели улавливать сложные зависимости и контекст, характерные для естественного языка.
- Возможность генерировать грамматически правильные и содержательно значимые текстовые данные, что критически важно для создания релевантных аннотаций.
- Стабильная производительность при обработке сложных промптов и интеграции задач аннотирования, что обеспечивает высокую точность и воспроизводимость результатов.

Для взаимодействия с моделью *Mistral-7b* используется универсальный API, который обеспечивает гибкость и простоту интеграции. В коде настройки API (`settings.py`) задаются ключевые параметры для работы с моделью, включая API-ключ и настройки промптов:

- `OPENROUTER_API_KEY`: Уникальный ключ для доступа к API, обеспечивающий авторизацию и управление запросами.
- `DEFAULT_MODEL`: Указание на модель `mistralai/mistral-7b-instruct` как основную для выполнения задачи аннотирования.
- `TASK_DESCRIPTION` и `EMPTY_TASK_DESCRIPTION`: Промпты, которые используются для описания задачи аннотирования. Первый промпт включает примеры диалогов и их аннотаций, что позволяет модели лучше понять формат и требования задачи. Второй промпт используется для случаев, когда примеры отсутствуют, и является общим описанием задачи.
- `INPUT_PREFIX` и `OUTPUT_PREFIX`: Префиксы, которые добавляются к входным данным и результатам аннотаций соответственно. Эти префиксы структурируют формат запроса и ответа, что облегчает обработку данных.

Для работы с данными в различных форматах (например, `.pkl`, `.csv`, `.parquet`) используется функция `load_file`, которая автоматически определяет формат файла и загружает его в виде таблицы данных. Эта функция обеспечивает

гибкость при работе с различными источниками данных и упрощает их подготовку для дальнейшей обработки. Основные особенности функции:

- Поддержка нескольких форматов файлов, включая бинарные (.pkl), текстовые (.csv) и колоночные (.parquet).
- Автоматическое определение формата на основе расширения файла, что устраняет необходимость вручную указывать тип данных.
- Генерация исключений в случае использования неподдерживаемого формата, что обеспечивает корректную обработку ошибок.

Входные данные для генерации аннотаций представляют собой диалоги, которые загружаются в виде таблицы с двумя основными колонками:

- `embedding`: Векторное представление диалога, используемое для кластеризации и анализа.
- `dialogue`: Оригинальный текст диалога, который передаётся модели для генерации аннотации.

Таким образом, процесс генерации синтетических аннотаций включает загрузку данных, подготовку промптов, взаимодействие с моделью через API и обработку результатов. Это обеспечивает создание качественных аннотаций, которые могут быть использованы для дальнейшего анализа и обучения моделей.

3.3.4. Модуль обновления базы данных

Модуль обновления базы данных является ключевым компонентом системы, обеспечивающим взаимодействие с хранилищем векторных представлений, диалогов и сгенерированных аннотаций. Его основная задача заключается в управлении данными в базе *Milvus*, включая их загрузку, обновление, удаление и поиск. Этот модуль играет важную роль в поддержании консистентности данных, их актуальности и доступности для последующей обработки и анализа. Функционал модуля ориентирован на работу с векторными представлениями, что требует тщательной обработки данных и их адаптации для хранения в специализированных коллекциях *Milvus*.

А. Инициализация базы данных:

- Создание и настройка коллекций в *Milvus* для хранения векторных представлений, диалогов и аннотаций. Каждая коллекция создается с учетом специфики данных, включая структуру полей и типы данных.

- Проверка существующих коллекций на соответствие требуемой структуре. В случае обнаружения несоответствий модуль автоматически корректирует структуру коллекций для обеспечения совместимости с данными.

В. Загрузка данных:

- Импорт данных из внешних источников, таких как файлы формата `.pkl`, `.json` или `.parquet`. Модуль поддерживает автоматическое преобразование данных в структурированный формат, пригодный для хранения в базе данных.
- Преобразование текстовых данных в векторные представления (эмбединги) с использованием моделей обработки естественного языка (NLP). В случае, если данные уже содержат эмбединги, модуль проверяет их корректность перед сохранением.

С. Добавление новых данных:

- Вставка новых записей, включающих диалоги, их векторные представления и сгенерированные аннотации. Модуль автоматически проверяет уникальность данных, чтобы избежать дублирования.
- Генерация векторных представлений для новых диалогов с использованием моделей NLP, если они отсутствуют в исходных данных. Это обеспечивает готовность данных для последующей обработки.

Д. Обновление существующих данных:

- Замена устаревших данных, например, при обновлении векторных представлений в случае изменения модели NLP. Модуль автоматически идентифицирует записи, требующие обновления, и выполняет их замену.
- Корректировка метаданных, таких как время создания записи, источник данных или статус аннотации. Это позволяет поддерживать актуальность информации в базе данных.

Е. Удаление данных:

- Удаление избыточных или устаревших записей, таких как дублирующиеся диалоги или данные, не соответствующие текущим требованиям. Модуль обеспечивает эффективное освобождение ресурсов базы данных.

- Очистка коллекций по запросу пользователя, что позволяет удалить все записи и подготовить базу данных для нового набора данных.

Г. Итеративный процесс обновления:

- Пошаговое добавление новых данных в базу данных, что позволяет интегрировать информацию по мере её поступления. Каждая итерация включает проверку семантической уникальности нового диалога относительно существующих записей.
- Использование моделей NLP для автоматической генерации аннотаций для новых диалогов. Это обеспечивает полноту данных и их готовность для последующего анализа.

Г. Функции поиска:

- Быстрый поиск по векторным представлениям для выявления семантически близких записей. Это позволяет находить диалоги, схожие по смыслу с заданным запросом.
- Поддержка сложных запросов для анализа качества данных, таких как поиск нерелевантных или дублирующихся записей. Это помогает поддерживать высокое качество данных в базе.

Н. Мониторинг и логирование:

- Ведение подробных логов всех операций, выполняемых модулем, включая добавление, обновление и удаление данных. Это обеспечивает прозрачность работы системы и позволяет отслеживать изменения.
- Мониторинг производительности операций, таких как время выполнения запросов и скорость обработки данных. Это позволяет выявлять узкие места и оптимизировать работу модуля.

Модуль обновления базы данных отвечает за актуализацию и управление данными, которые используются в процессе кластеризации. В контексте данного проекта, обновление базы данных связано с загрузкой предварительно сгенерированных эмбедингов и текстовых диалогов, их преобразованием в удобный для анализа формат и обеспечением целостности и актуальности данных. Основная задача модуля – обеспечить корректное функционирование кластеризации, предоставляя ей актуальные и подготовленные данные.

Процесс начинается с загрузки данных из файла формата Parquet, который содержит эмбединги и текстовые диалоги. Для этого используется функция

`load_file`, которая принимает путь к файлу и возвращает данные в виде `DataFrame`. Это позволяет эффективно работать с большими объемами данных, сохраняя их структуру и целостность. Пример кода:

```
file_name = 'har1_mts_dialogue-clinical.parquet'
file_path = embeddings_dir / file_name
data_df = load_file(str(file_path))
```

После загрузки данных выполняется преобразование эмбедингов из списка в массив `NumPy`, что обеспечивает совместимость с функциями кластеризации:

```
embeddings = np.array(data_df['embedding'].tolist())
original_data = data_df['dialogue'].tolist()
```

Этот шаг упрощает дальнейшую обработку данных и делает их пригодными для анализа.

В приведённом коде модуль используется для загрузки данных и преобразования их в формат, пригодный для кластеризации. Векторные представления (эмбединги) извлекаются из данных и передаются в модуль кластеризации для дальнейшего анализа. Этапы инициализации, загрузки и обработки данных демонстрируют ключевые функции модуля обновления базы данных, обеспечивая подготовку данных для последующих этапов работы системы.

Таким образом, модуль обновления базы данных является неотъемлемой частью системы, обеспечивающей эффективное управление данными и их подготовку для анализа. Его функционал ориентирован на работу с большими объемами данных и их адаптацию для использования в задачах машинного обучения и обработки естественного языка.

Модуль обновления базы данных является неотъемлемой частью всей системы фреймворка, так как он позволяет обеспечивать доступность, актуальность и консистентность данных. С помощью возможностей *Milvus*, таких как управление коллекциями и поддержка семантического поиска, достигается высокая производительность модуля и удобство его интеграции в общий процесс работы системы.

3.3.5. Модуль оценки и контроля качества результатов

Модуль оценки и контроля качества является одним из центральных компонентов фреймворка, отвечающим за автоматизированный анализ полученных суммаризаций диалогов. Он не только осуществляет базовую проверку с помощью стандартных метрик (ROUGE, BERTScore), но и предоставляет ряд дополнитель-

ных возможностей: асинхронную обработку больших массивов, гибкую настройку вычислений, логирование и визуализацию результатов. Ниже описана детальная структура классов, программные методы и особенности реализации модуля.

- *Мониторинг качества на каждом цикле*: после генерации суммаризаций модуль анализирует их метрики, выявляет проблемные кейсы и передаёт информацию в другие компоненты.
- *Гибкая архитектура*: поддерживает разные типы метрик (ROUGE, BERTScore, GPT-based), различные режимы вычисления (синхронный/асинхронный), а также интеграцию кастомных алгоритмов.

В общем рабочем потоке фреймворка модуль оценивания включён в несколько этапов:

- A. *Генерация суммаризаций* — после выполнения ICL-механизма получаем кандидаты «Диалог → Итоговая выжимка».
- B. *Оценка результатов* — модуль асинхронно рассчитывает метрики и определяет качество полученных текстов.
- C. *Запись в базу/логирование* — результаты сохраняются в БД и при необходимости визуализируются.
- D. *Анализ ошибок* — систематически выявляются диалоги с низкими баллами, которые могут стать «сигналом» для изменения стратегии отбора демонстраций.

Для реализации данного модуля выделено несколько классов и методов:

- *QualityAssessment*: основной класс, содержащий логику вычисления метрик (ROUGE, BERTScore и др.).
 - `evaluate_batch(references, hypotheses)` — метод, принимающий батч эталонных текстов (*references*) и сгенерированных (*hypotheses*) и возвращающий структуру с рассчитанными метриками.
 - `process_batches(references, hypotheses, batch_size)` — метод, формирующий задачи асинхронной оценки большими порциями (batch-wise) и обрабатывающий их параллельно.
 - `log_results(results)` — вспомогательная функция для записи итогов (метрики, тексты) в базу или в лог-файл.
- *ErrorAnalysis*: вспомогательный класс (или модуль), занимающийся выявлением «проблемных» случаев:

- `filter_low_scores(results, threshold)` — метод, возвращающий список диалогов с оценками ниже заданного порога.
- `generate_error_report(problematic_dialogs)` — строит таблицу или JSON-отчёт для детального ручного анализа.

Наличие отдельных классов **QualityAssessment** и **ErrorAnalysis** повышает модульность кода: вычисление метрик и анализ ошибок логически разведены.

Модуль реализует поддержку `asyncio` в Python, что позволяет обрабатывать сотни (и тысячи) диалогов параллельно, не блокируя основной поток выполнения. При необходимости можно задействовать `multiprocessing` для распределения нагрузки на несколько ядер процессора.

```
import asyncio
from rouge_score import rouge_scorer
from bert_score import score as bert_score

class QualityAssessment:
    def __init__(self):
        self.rouge_scorer = rouge_scorer.RougeScorer(
            ['rouge1', 'rouge2', 'rougeL'], use_stemmer=True
        )

    async def evaluate_batch(self, batch_refs, batch_hyps):
        results = []
        for ref, hyp in zip(batch_refs, batch_hyps):
            rouge_scores = self.rouge_scorer.score(ref, hyp)
            P, R, F1 = bert_score([hyp], [ref], lang='en')
            results.append({
                "ref": ref,
                "hyp": hyp,
                "rouge1": rouge_scores['rouge1'].fmeasure,
                "rouge2": rouge_scores['rouge2'].fmeasure,
                "rougeL": rouge_scores['rougeL'].fmeasure,
                "bert_f1": float(F1.mean())
            })
        return results
```

- *ROUGE* (*rouge1*, *rouge2*, *rougeL*) — оценивает степень перекрытия n -грамм между итоговым текстом и эталоном.
- *BERTScore* — вычисляется на уровне эмбеддингов (нормализованных), отражая семантическую близость.
- *GPT-based Evaluation* (по желанию) — в модуле предусмотрен хук для внешнего сервиса: модель GPT может «комментировать» согласованность и плавность текста (качественная оценка).

Модуль может подключаться к реляционной (PostgreSQL) или NoSQL (MongoDB) базе, сохраняя там поля:

- `dialog_id`, `hyp_text`, `ref_text`;
- `rouge1`, `rouge2`, `rougeL`, `bert_f1`;
- `timestamp`, `run_config` (для воспроизводимости).

Таким образом, модуль оценки и контроля качества служит важным связующим звеном между процессом генерации суммаризаций и их практическим использованием. Его асинхронный характер, поддержка разнообразных метрик и интеграция с базой данных позволяют не только следить за качеством итогового текста, но и настраивать цепочку ICL для достижения лучших результатов.

3.4. Структура и организация кода проекта

Для поддержания удобства разработки, тестирования, расширяемости и совместной работы над проектом была сформирована многослойная структура репозитория. Она обеспечивает чёткое разделение логики по модулям, а также даёт возможность легко находить нужные участки кода при добавлении новых функций или отладке.

- `LICENSE`: определяет условия распространения/использования кода.
- `README.md`: содержит описание функционала, схему установки, минимальные примеры использования, а также ключевую информацию по запуску.
- `setup.py`: скрипт, позволяющий установить фреймворк как Python-пакет (через `pip install .`).
- `requirements.txt`: фиксирует все необходимые зависимости (версии библиотек), упрощая развёртывание окружения.

Внутри корневой директории выделена папка `my_framework/`, которая рассматривается как самостоятельный Python-пакет. Каждый файл отвечает за свой отдельный аспект работы фреймворка:

- `data_preprocessing.py`: классы и методы (`DataCleaner` и т.д.) для очистки диалогов, лемматизации, нормализации и разбиения по репликам.
- `embeddings.py`: код (`EmbeddingGenerator`) для батчевой генерации векторных представлений диалогов, интеграция с `Sentence-BERT`, `USER-BGE` и т.д.
- `rag.py`: реализация модуля `RAGRetriever`, осуществляющего поиск релевантных диалогов по векторной базе, применяя косинусное сходство или другие метрики.
- `llminteraction.py`: функции для формирования промптов и взаимодействия с крупными языковыми моделями (`GPT-3.5`, `Mistral-7b` и пр.).
- `database.py`: управление соединением с `SQL/NoSQL`-базами, запись эмбеддингов, сохранение аннотаций (*Диалог* → *Суммаризация*) и вспомогательные методы.
- `clustering.py`: опциональные средства для группировки схожих диалогов (например, при выборе `seed`-набора) или анализа структуры корпуса.
- `evaluation.py`: модуль оценки качества (`QualityAssessment`, `ErrorAnalysis` и др.). Содержит логику вычисления метрик (`ROUGE`, `BERTScore`), а также вспомогательные функции для автоматического поиска «проблемных» диалогов.
- `config.py`: хранит глобальные настройки и параметры (пути к данным, ключи `API`, настройки подключения к БД).

Для удобства запуска ключевых стадий работы фреймворка используются отдельные CLI-скрипты в директории `scripts/`:

- `run_preprocessing.py`: вызывает методы из `data_preprocessing.py`, приводя сырые диалоги к единому формату.
- `run_embedding.py`: выполняет батчевую генерацию эмбеддингов, сохраняя их в базу (например, `Milvus` или `Faiss`).
- `run_llminteraction.py`: формирует промпты, используя `rag.py` и `llm_interaction.py`, и генерирует итоговые выжимки.
- `run_evaluation.py`: с помощью `evaluation.py` подсчитывает метрики (`ROUGE`, `BERTScore`) и выделяет проблемные случаи. Может выводить отчёты и логи.

В случае сложного пайплайна создаётся единый скрипт `run_pipeline.py`, последовательно запускающий все этапы.

Директория `examples/` хранит примеры и демо-файлы, полезные для быстрого старта:

- `demo_notebook.ipynb`: Jupyter-ноутбук с пошаговой демонстрацией предобработки, генерации эмбедингов, использования RAG, вызова LLM и расчёта метрик оценки.
- `sample_seed_data.json`: небольшой тестовый набор диалогов, помогающий отладить фреймворк «из коробки».

Данная директория упрощает знакомство с проектом и является «визитной карточкой» фреймворка.

Директория `tests/` содержит набор `unittest` или `pytest`-скриптов, проверяющих корректность основных модулей. Каждому компоненту (`data_preprocessing`, `embeddings`, `evaluation`) обычно соответствует свой файл (например, `test_evaluation.py` для проверки модулей из `evaluation.py`). Это позволяет быстро локализовать регрессии и поддерживать высокий уровень надёжности кода.

Подобная структура поддерживает требуемый баланс между функциональной чёткостью и гибкостью расширения проекта. Модули **оценки** и **генерации** выделены в самостоятельные файлы, что облегчает их тестирование и развитие. Разделение на `scripts`, `examples` и `tests` даёт наглядность: любой новый пользователь быстро найдёт, где запускать пайплайн, где хранить примеры и как проверяются отдельные компоненты. Такая организация позволяет подключить интеграционное и юнит-тестирование в CI/CD-системах, упрощает масштабирование фреймворка и способствует рациональному распределению обязанностей среди разработчиков и исследователей.

3.5. Выводы

В третьей главе представлена практическая реализация фреймворка, предназначенного для автоматической генерации обучающих данных в задачах диалоговой суммаризации. Архитектура фреймворка основана на модульном подходе, что обеспечивает его гибкость, адаптивность и возможность замены отдельных компонентов без необходимости значительных изменений в коде. Это позволя-

ет легко интегрировать новые технологии и адаптировать систему к различным предметным областям.

Ключевым элементом является автоматизация обработки данных на всех этапах — от предобработки текстов до формирования итоговых аннотаций. За счёт использования эмбедингов и механизма поиска релевантных примеров достигается высокая точность суммаризаций. Модуль RAGRetriever позволяет подбирать демонстрационные примеры на основе векторных представлений диалогов, что существенно повышает качество итоговых текстов.

Фреймворк поддерживает взаимодействие с различными языковыми моделями, включая GPT-3.5, Mistral-7b и Llama2, а также интеграцию с векторными базами данных Milvus и Faiss. Это делает систему гибкой и пригодной для работы с разными типами данных. Встроенный модуль контроля качества обеспечивает автоматическую оценку сгенерированных суммаризаций, используя метрики, такие как ROUGE и BERTScore. Это позволяет анализировать точность модели и при необходимости корректировать стратегию генерации.

ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНЫЙ АНАЛИЗ МЕТОДИКИ

Четвёртая глава посвящена экспериментальной оценке разработанного фреймворка, предназначенного прежде всего для задачи автоматической аннотации диалогов или других типов текстовых данных. Основная цель экспериментов — продемонстрировать практическую значимость и универсальность предложенного подхода, а также показать, как использование фреймворка влияет на качество итоговых результатов. Особое внимание уделяется тому, как использование семантически релевантных демонстрационных примеров в few-shot обучении влияет на итоговый результат. Исследуется, в какой степени применение таких примеров может улучшить способность модели к генерации информативных и точных суммаризаций диалогов.

Фреймворк, хотя и был разработан для задач диалоговой суммаризации, имеет более широкую область применения. Его модульная архитектура позволяет использовать его в смежных задачах NLP, таких как автоматическое создание обучающих наборов для чат-ботов, генерация аннотаций для анализа тональности или обучение систем вопрос-ответ. Это делает данный инструмент универсальным решением для многих задач, требующих качественной генерации данных.

В данной главе представлены два независимых эксперимента. Первый эксперимент посвящён сравнению качества суммаризаций, создаваемых языковыми моделями при использовании различных стратегий выбора демонстрационных примеров в few-shot обучении. Рассматриваются два варианта: SEMANTIC - метод, где демонстрационные примеры отбираются с помощью основного метода фреймворка — поиска семантически релевантных примеров ; Baseline-метод, в котором демонстрационные примеры выбираются случайным образом. Основная цель эксперимента — оценить, насколько метод выбора семантически близких примеров позволяет улучшить качество суммаризаций по сравнению с случайным отбором демонстраций и насколько полученные результаты могут конкурировать с эталонными данными.

Второй эксперимент посвящён оценке влияния синтетических данных на обучение LLM (на примере Vikhr-Llama-3.2-1B-Instruct). Один экземпляр модели обучался на эталонных данных, аннотированных вручную, а другой — на синтетических данных, полученных с использованием разработанного фреймворка. После обучения модели сравниваются по ключевым метрикам суммаризации на валидационных наборах данных.

Результаты экспериментов позволят выявить сильные и слабые стороны предложенного подхода, оценить его влияние на качество современных языковых моделей и продемонстрировать потенциал использования фреймворка за пределами задачи диалоговой суммаризации. В рамках этого исследования анализируется, насколько данные, полученные без участия человека, сопоставимы с эталонными в задачах instruction-tuning и могут ли они служить полноценной заменой или дополнением традиционной разметки.

Основные вопросы, рассматриваемые в данной главе, связаны с определением того, насколько метод SEMANTIC SELECTION превосходит BASELINE метод в контексте few-shot обучения, а также с анализом качества автоматически сгенерированных данных при использовании их в обучении языковых моделей. Полученные результаты помогут определить сильные и слабые стороны предложенного подхода и выявить перспективные направления его применения.

4.1. Цели и задачи экспериментов

В данной главе описываются цели и задачи, которые ставились перед серией экспериментов, направленных на оценку эффективности методики. Эксперименты

проводились в контексте задачи автоматической аннотации и суммаризации диалогов, однако подходы, применяемые фреймворком, имеют потенциал для более широкого использования, что делает исследования в данной области актуальными.

Основной целью экспериментов является проверка гипотезы о том, что применение методики текущей работы для автоматической аннотации диалогов позволяет:

- Оценить значимость выбора демонстрационных примеров для задачи суммаризации, анализируя различные стратегии их отбора, включая случайные (*Baseline*) и семантически релевантные демонстрационные примеры (*Semantic Selection*). Оценка проводится на основе стандартных метрик качества суммаризации (ROUGE, BERTScore и других), а также с привлечением методов сравнительного анализа.
- Оценить влияние автоматически аннотированных данных на качество обучаемых моделей по сравнению с эталонными наборами, размеченными вручную. Для этого проводится сравнение моделей, обученных на двух типах данных: (1) эталонных, вручную аннотированных примерах и (2) автоматически сгенерированных данных с использованием предложенной методики. Второй эксперимент направлен на проверку, могут ли синтетически созданные данные служить альтернативой ручной разметке при обучении языковой модели и как они влияют на качество суммаризаций в рамках задачи *instruction tuning*.

В первом эксперименте ожидается выявление различий в качестве суммаризаций между стратегиями *Random demonstrations* и *Relevant demonstrations*. Предполагается, что использование релевантных демонстрационных примеров позволит улучшить качество итоговых суммаризаций за счёт учёта семантической близости демонстрационных примеров.

Во втором эксперименте ожидается, что модель, обученная на данных, сгенерированных фреймворком, продемонстрирует качество, сопоставимое с моделью, обученной на эталонных данных, вручную аннотируемых. Это подтвердит применимость фреймворка для создания синтетических данных, пригодных для использования в задачах обучения языковых моделей.

Проведение данных экспериментов позволит не только проверить применимость разработанного фреймворка, но и внести вклад в развитие методов автоматической аннотации данных, что особенно актуально для задач, требующих обработки больших объёмов информации. Результаты экспериментов также

помогут определить оптимальные стратегии использования демонстрационных примеров в контексте суммаризации диалогов и других задач обработки естественного языка.

4.2. Метрики оценивания разработанной методики.

Для объективной оценки качества результатов, полученных в ходе экспериментов, используются три группы метрик: лексические, семантические и основанные на генеративных моделях. Такой подход позволяет учесть как точное совпадение текстов, так и их смысловую близость, что особенно важно при оценке суммаризаций.

Лексические метрики включают в себя ROUGE (Recall-Oriented Understudy for Gisting Evaluation), который является стандартом для оценки качества суммаризаций. В рамках экспериментов применяются ROUGE-1, ROUGE-2 и ROUGE-L. ROUGE-1 измеряет точность совпадения отдельных униграмм между сгенерированным текстом и эталонным, ROUGE-2 оценивает биграммы, а ROUGE-L использует наибольшую общую подпоследовательность слов, что позволяет учитывать структуру предложения.

Семантическая оценка проводится с использованием BERTScore, который основывается на трансформерных эмбедингах слов. В отличие от ROUGE, данный метод не требует точного совпадения слов, а использует косинусное расстояние между эмбедингами предложений. Это делает его особенно полезным при оценке суммаризаций, где важнее передача смысла, чем буквальное соответствие.

Дополнительно для оценки качества сгенерированных текстов применяется GPT-based эвалюация с использованием модели GPT-3.5 Turbo. Данная методика включает три ключевых компонента: Coherence (согласованность текста), Relevance (релевантность суммаризации к исходному диалогу) и Fluency (гладкость и естественность текста). Оценка проводится автоматически путем использования GPT-3.5 Turbo в качестве «судьи», который анализирует выходные тексты и присваивает им соответствующие оценки.

Применение всех трех групп метрик позволяет всесторонне оценить качество суммаризаций, учитывая не только точность совпадения слов, но и смысловую релевантность, а также читабельность итогового текста. Такой подход особенно важен в задачах автоматической аннотации, где высокую ценность имеют семантически корректные и естественные тексты.

4.3. Эксперимент 1: Оценка влияния стратегии выбора демонстрационных примеров на качество суммаризаций

Данный раздел посвящён анализу первого эксперимента, в котором исследуется влияние стратегии отбора демонстрационных примеров в парадигме *In-Context Learning* (ICL) на качество суммаризации диалогов. Рассматриваются два различных подхода:

- *Baseline ICL* – случайный отбор демонстрационных примеров (*Random Selection*), без учёта их релевантности к входному диалогу.
- *Semantic ICL* – стратегия, основанная на выборе семантически близких примеров (*Semantic Selection*), являющаяся основным методом предлагаемого фреймворка.

Помимо сравнения указанных методик, исследуется, даёт ли сам факт использования *In-Context Learning* (независимо от метода отбора примеров) прирост качества суммаризаций по сравнению со сценарием, где модель не получает никаких демонстраций (*Zero-Shot*).

Для оценки эффективности предложенного метода используются три различных корпуса диалогов, которые различаются по языку, тематике и сложности обработки. Это позволяет исследовать, насколько универсален предложенный подход и как он ведёт себя в условиях различных типов текстовых данных.

Англоязычный корпус диалогов, разработанный для задачи автоматической суммаризации **SamSum** [26]. Данный датасет содержит ~16k кратких повседневных диалогов, аннотированных вручную профессиональными редакторами, что делает его одним из наиболее популярных стандартных наборов данных для задач суммаризации диалогов. Диалоги представляют собой короткие переписки в мессенджерах, что соответствует реальным пользовательским сценариям. Аннотации в датасете формируют короткие, обобщённые резюме, передающие основное содержание беседы.

Русскоязычный корпус диалогов и их суммаризаций **Bragovo/dsum_ru**. [1]. Данный датасет представляет собой адаптированную версию SamSum для русского языка и включает в себя диалоги различной структуры. Он позволяет исследовать применимость предложенной методики в русскоязычных текстах, где формулировки могут быть менее структурированными, а синтаксическая сложность – выше. Включение данного набора данных позволяет оценить эффективность модели на языках с разной морфологией и синтаксисом.

Специализированный корпус диалогов из соревнования **MEDIQA-Chat 2023** [14]. В отличие от предыдущих наборов данных, этот корпус содержит сложные медицинские диалоги между врачами и пациентами, а также автоматические суммаризации в виде клинических записей. Основная сложность работы с данным набором заключается в наличии специализированной терминологии, требующей от модели глубокой интерпретации и способности к обработке сложных доменных знаний. Исследование работы модели на таком корпусе позволяет оценить применимость предложенной методики в специализированных профессиональных областях, где точность суммаризаций критически важна.

Для всех экспериментов размер *seed*-набора был установлен на уровне 10% от N диалогов, использованных в каждом конкретном эксперименте. Это значение подобрано эмпирически, исходя из предположения, что небольшое количество демонстрационных примеров должно обеспечивать достаточный уровень обобщения для модели.

Результаты данного эксперимента критически важны, поскольку они позволяют сделать вывод о необходимости использования предлагаемого фреймворка. В случае отсутствия прироста качества по метрикам *ROUGE*, *BERTScore* и *GPT-based оценок* (Coherence, Relevance, Fluency) можно будет заключить, что методика отбора демонстрационных примеров не вносит значимого вклада в улучшение суммаризаций, а значит, её практическая применимость будет под вопросом.

Стоит также описать базовый подход, используемый в качестве отправной точки для сравнения с основной методикой отбора демонстрационных примеров, предложенной в данной работе. В качестве baseline-метода применяется случайный отбор демонстрационных примеров для ICL, который широко используется в литературе как простая и интуитивная стратегия выбора примеров. Данный метод не учитывает семантическую близость между входным диалогом и демонстрациями, что делает его удобным, но в то же время потенциально менее эффективным в сложных задачах суммаризации.

Случайный отбор демонстраций предполагает выбор k примеров без учёта их релевантности к входному диалогу. Этот процесс включает несколько ключевых этапов. На первом шаге формируется пул демонстрационных примеров, который составляет 10% от общего числа диалогов участвующих в эксперименте, используемых в эксперименте. Затем для каждого нового входного диалога случайным образом выбираются k демонстраций из данного пула. Эти примеры передаются модели в prompt вместе с входным диалогом, после чего модель генерирует итого-

вую суммаризацию. Полученные суммаризации оцениваются с использованием метрик качества: ROUGE, BERTScore, GPT-based evaluation.

Несмотря на простоту, данный метод имеет несколько значительных ограничений. Во-первых, он не учитывает смысловую структуру входного диалога, что может привести к выбору тематически нерелевантных демонстраций. Это особенно критично для специализированных областей, например медицины, где точность в передаче информации играет ключевую роль. Во-вторых, качество итоговых суммаризаций может варьироваться в зависимости от случайно выбранных примеров, что снижает стабильность результатов. В-третьих, в условиях задач с высокой семантической сложностью случайный отбор может привести к некорректным обобщениям и снижению точности.

Случайный отбор демонстрационных примеров используется в данном исследовании как сравнительная точка отсчёта. Он позволит оценить, насколько предложенный метод семантического выбора примеров (*Semantic Selection*) превосходит случайный отбор по ключевым метрикам качества. Ожидается, что использование случайных примеров приведёт к ухудшению качества суммаризаций по сравнению с подходом, учитывающим семантическую близость примеров. Если предложенная методика отбора демонстраций даст статистически значимые улучшения, это подтвердит целесообразность использования разработанного фреймворка для автоматической аннотации диалогов.

4.3.1. Оценка методики на примере общих диалогов

Для оценки влияния стратегии выбора демонстрационных примеров были проведены эксперименты на двух корпусах диалогов: англоязычном **SamSum** и русскоязычном **Bragovo/dsum_ru**. Оба датасета содержат диалоги с профессионально аннотированными краткими выжимками и широко используются в задачах диалоговой суммаризации.

SamSum представляет собой англоязычный корпус, содержащий неформальные диалоги, аннотированные профессиональными редакторами. Важной особенностью SamSum является высокая вариативность структуры диалогов, что делает его хорошим тестовым полигоном для оценки гибкости модели. На основе анализа данных из SamSum были построены распределения длины диалогов и их суммаризаций, которые показаны на рисунке 4.1. Видно, что длины диалогов

значительно превышают длины их суммаризаций, что подчеркивает сложность задачи суммаризации.

В качестве языковой модели (LLM) для генерации суммаризаций была выбрана **mistralai/mistral-7b-instruct**. Эта модель, оптимизированная для инструкционного формата запросов, обеспечивает высокое качество текстов при сравнительно небольшом числе параметров. Модель также демонстрирует хорошую производительность на русскоязычных данных, что позволило исключить необходимость использования переводчиков в пайплайне эксперимента.

Для обеспечения единых условий экспериментов в обоих подходах (Baseline и Semantic Selection) использовался одинаковый гиперпараметр $k = 2$, то есть в prompt включались ровно два демонстрационных примера. Температура модели была зафиксирована на уровне $t = 1.0$, что позволило сбалансировать детерминированность и вариативность выходных текстов.

Эксперимент на общих диалогах позволяет оценить, насколько стратегия выбора демонстрационных примеров влияет на итоговое качество суммаризаций, а также подтвердить целесообразность использования предложенной методики фреймворка в реальных задачах. Полученные данные лягут в основу сравнительного анализа результатов, проведенного в следующих разделах.

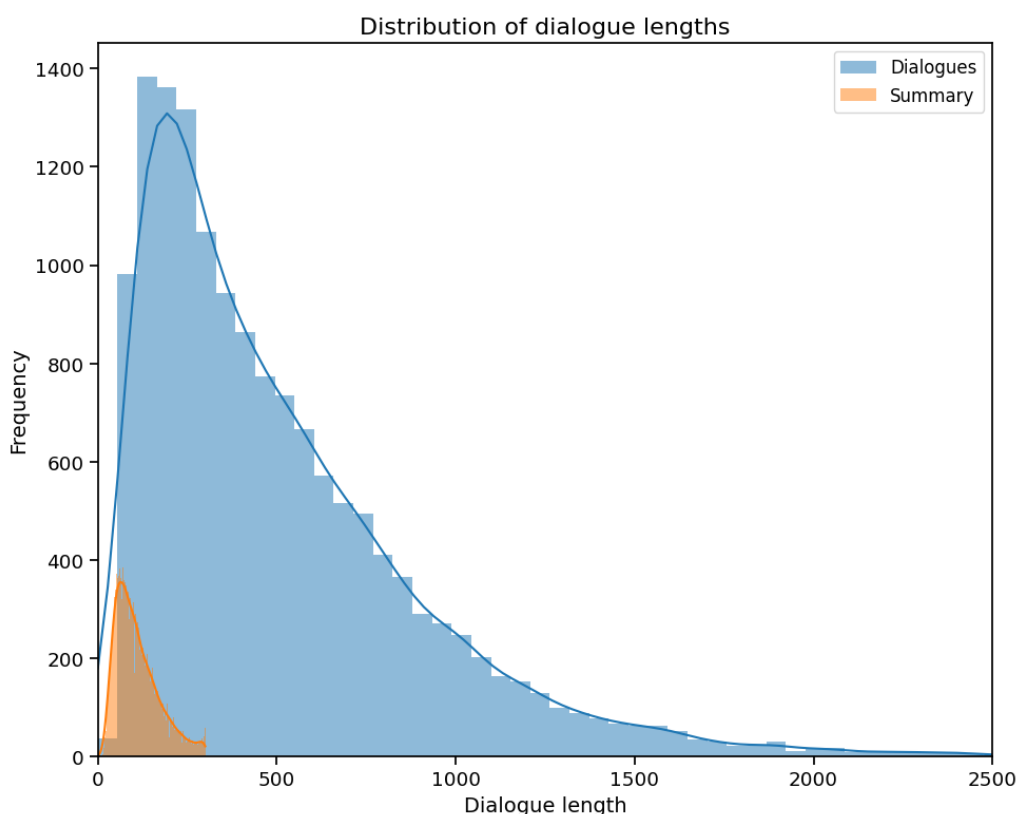


Рис.4.1. Распределение длин диалогов и их суммаризаций в датасете SamSum

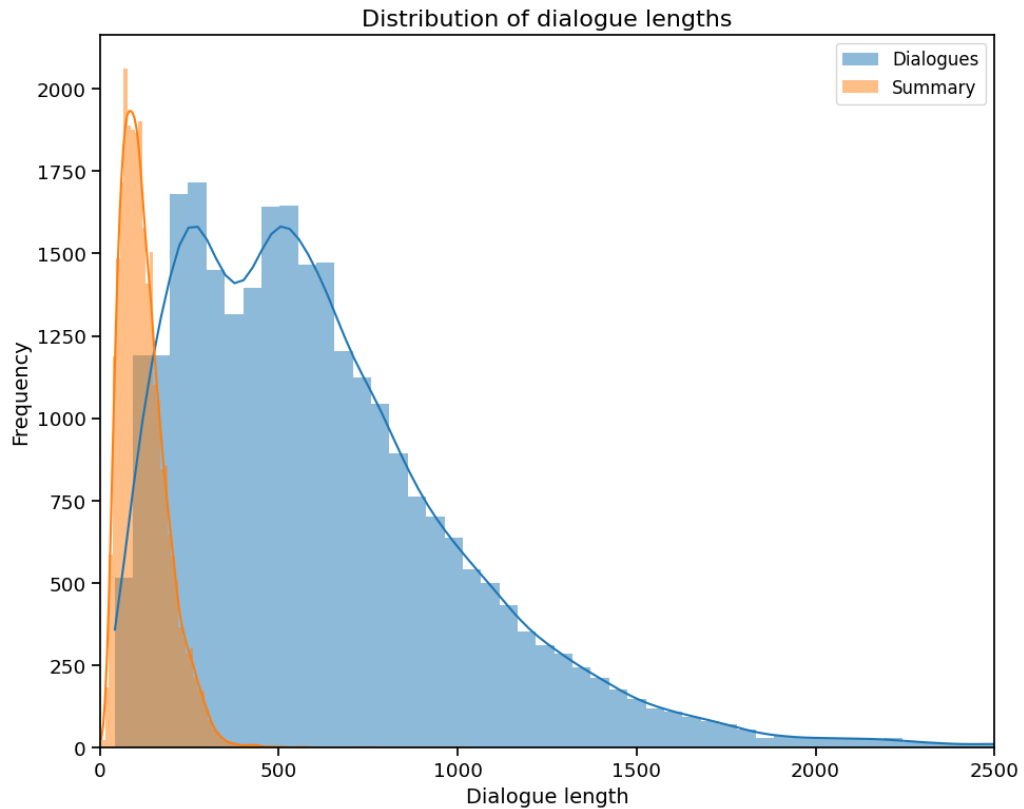


Рис.4.2. Распределение длин диалогов и их суммаризаций в датасете Bragovo/dsum_ru

Таблица 4.1

Сравнение метрик ROUGE и BERTScore на двух датасетах (SamSum и DSumRU) при трёх подходах к выбору демонстраций. Метрики: R1 – ROUGE-1, R2 – ROUGE-2, RL – ROUGE-L, PR – Precision, RC – Recall, F1 – F1 (N = 1000)

Method	R-1	R-2	R-L	PR	RC	F1
No ICL _{SamSum}	0.30	0.10	0.23	0.85	0.91	0.88
Baseline ICL _{SamSum}	0.39	0.14	0.29	0.88	0.91	0.89
Semantic ICL _{SamSum}	0.41	0.15	0.31	0.88	0.92	0.90
No ICL _{DSumRU}	0.25	0.09	0.20	0.80	0.81	0.86
Baseline ICL _{DSumRU}	0.33	0.13	0.25	0.81	0.83	0.84
Semantic ICL _{DSumRU}	0.33	0.14	0.25	0.83	0.83	0.83

Результаты, представленные в Таблице 4.1, показывают, что использование демонстрационных примеров в подходе In-Context Learning (ICL) действительно улучшает качество суммаризаций. Тем не менее, прирост, достигнутый с помощью основной методики фреймворка (Semantic ICL), на этих типах данных оказался сравнительно небольшим. Например, разница между Baseline ICL и Semantic ICL по ключевым метрикам ROUGE и BERTScore составляет в среднем не более 1–5%. Это свидетельствует о том, что в случае с относительно простыми и короткими

Таблица 4.2

Сравнение дополнительных качественных показателей (Coherence, Relevance, Fluency) на двух датасетах (SamSum и DSumRU) при трёх подходах к выбору демонстраций (N = 300)

Method	Coherence	Relevance	Fluency
No ICL _{SamSum}	4.27	4.43	4.53
Baseline ICL _{SamSum}	4.72	4.88	4.84
Semantic ICL _{SamSum}	4.75	4.86	4.86
No ICL _{DSumRU}	4.21	4.43	4.51
Baseline ICL _{DSumRU}	4.52	4.68	4.82
Semantic ICL _{DSumRU}	4.53	4.66	4.82

диалогами, такими как SamSum и Bragovo/dsumru, преимущества семантического отбора демонстрационных примеров становятся менее выраженными.

Таблица 4.2, содержащая оценки качественных метрик (Coherence, Relevance, Fluency), подтверждает эту тенденцию. Хотя Semantic ICL демонстрирует лучшие результаты по сравнению с Baseline ICL, различия минимальны, особенно на русскоязычном датасете Bragovo/dsumru. Разница между этими методами по показателю Coherence составляет всего 0.01 балла, что не является статистически значимым улучшением. На англоязычном датасете SamSum разница чуть более заметна, но также остаётся в пределах 1–2%.

Сравнение с методом без использования демонстрационных примеров (No ICL) показывает более значительные улучшения. Например, переход от No ICL к Baseline ICL приводит к приросту метрик ROUGE-1 и ROUGE-L на 20–25%, что подтверждает обоснованность применения few-shot демонстраций. Однако это улучшение наблюдается независимо от стратегии выбора демонстраций, что указывает на способность модели обобщать даже при случайном отборе примеров.

Эти результаты подчёркивают ограниченность влияния семантического отбора в задачах с короткими и структурированными диалогами. В таких случаях модель, подобная mistralai/mistral-7b-instruct, способна эффективно справляться с задачей суммаризации, даже если демонстрации выбираются случайно. Вероятно, это связано с низкой сложностью данных и коротким контекстом, которые уменьшают роль качественного подбора демонстрационных примеров.

Справедливо отметить, что прирост от использования Semantic ICL мог бы быть более значительным в задачах с большим контекстом или в доменах, требующих глубокой интерпретации текста. Например, для специализированных текстов, таких как медицинские диалоги или технические документы, семантический отбор

может оказаться более важным. Эти аспекты будут рассмотрены в следующем разделе, где анализируется влияние методики на более сложных корпусах.

4.3.2. Оценка методики на специализированных диалогах

В этом разделе представлен анализ влияния выбора демонстрационных примеров на задачу медицинской суммаризации, отличающуюся от общих диалогов значительно большей сложностью и высокой зависимостью от контекста. Для эксперимента был использован специализированный корпус медицинских диалогов, содержащий структурированные данные с множеством деталей и терминологий, которые необходимо корректно интерпретировать для получения качественной суммаризации.

В данном разделе представлен пример диалога из специализированного корпуса медицинских данных, который был использован в экспериментах. Данный корпус содержит высокоструктурированные медицинские диалоги, что делает его сложным для обработки задачами суммаризации. Пример иллюстрирует взаимодействие врача и пациента, а также содержит итоговую суммаризацию, включающую ключевые аспекты диалога.

Таблица 4.3

Пример медицинского диалога и его структурная суммаризация

Dialogue	Doctor: Hi, what brings you here today?
	Patient: I feel this lump on the upper side of my breast.
	Doctor: Both sides?
	Patient: No, only on the left side.
	Doctor: Do you feel it deep inside or on the outer side?
	Patient: Only on the outer side.
Summary	Symptoms: Lump on the upper outer quadrant of the left breast.
	Diagnosis: Suspicious calcifications upper outer quadrant, left breast.
	History of Patient: N/A.
	Plan of Action: N/A.

Как видно из приведённого примера, суммаризация структурирована в формате, подходящем для медицинских задач: секции включают симптомы, историю пациента, диагноз и план действий. Этот формат позволяет легко анализировать ключевую информацию, что особенно важно для медицинских приложений.

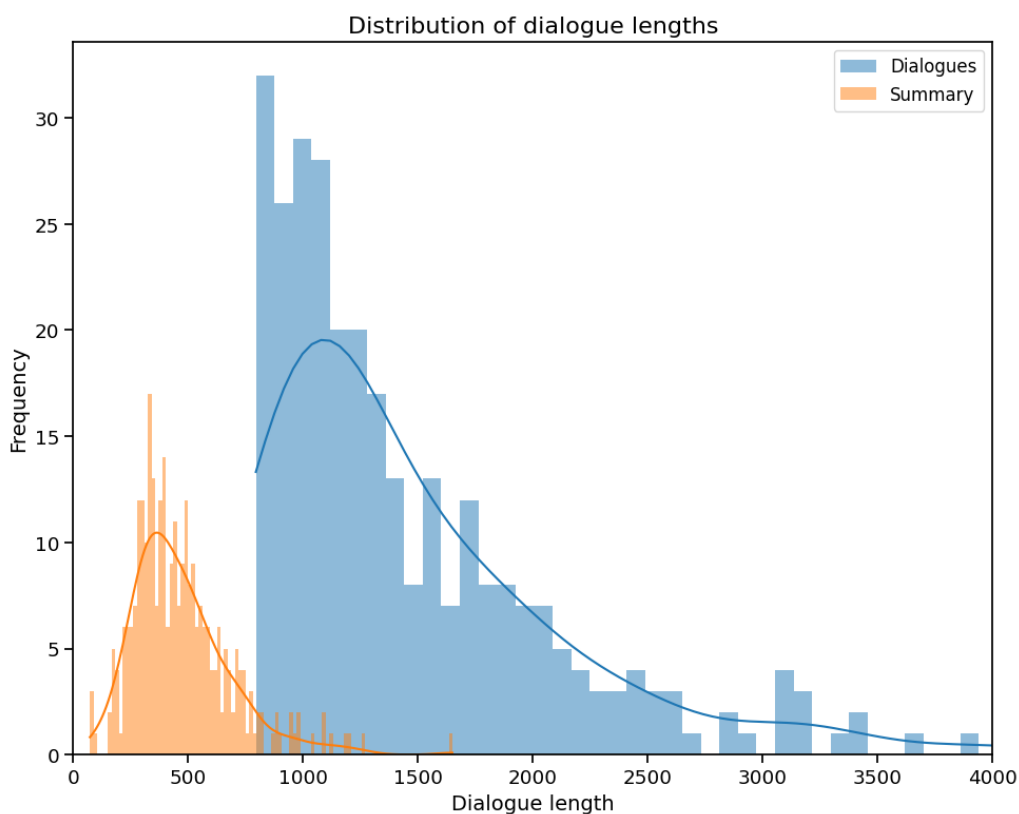


Рис.4.3. Распределение длин диалогов и их суммаризаций в датасете MTS Dialogue-Clinical Note

Методика эксперимента основывалась на подходах, описанных в предыдущих разделах. Для проведения анализа были выбраны две языковые модели (LLM): небольшая **Liquid 3B** и крупная **GPT-3.5 Turbo**. Это позволило оценить, как масштаб модели влияет на способность извлекать полезную информацию из демонстрационных примеров, а также на прирост метрик качества.

Для обеих моделей эксперименты проводились при различных комбинациях двух ключевых гиперпараметров:

- Температура (t): варьировалась между 0.3 (более детерминированное поведение) и 1.0 (более разнообразные генерации).
- Количество демонстрационных примеров (k): тестировались значения $k = 2$ и $k = 4$ для оценки влияния увеличения размера контекста.

Особенностью данного эксперимента стало тестирование на двух различных выборках:

- *Случайная выборка из 300 диалогов* (помечена в таблице как $t = 0.3, k = 2^*$): предназначена для оценки общей стабильности модели на разнообразных данных.

- *300 самых длинных диалогов*: выборка, включающая более сложные примеры, чтобы проверить способность моделей обрабатывать длинные контексты и выдавать более качественные суммаризации.

Результаты экспериментов приведены в Таблице 4.4. Значения показывают прирост метрик ROUGE и BERTScore (в процентах) относительно базового подхода без использования демонстрационных примеров.

Таблица 4.4

Прирост метрик ROUGE и BERTScore для различных гиперпараметров и моделей на медицинских диалогах MTS Dialogue-Clinical Note

LLM	Parameters	R-1	R-2	R-L	PR	RC	F1
LFM 3B	$t = 0.3, k = 2^*$	2%	4%	3%	0%	0%	0%
	$t = 0.3, k = 2$	5%	10%	7%	1%	1%	1%
	$t = 1.0, k = 2$	7%	16%	12%	1%	1%	1%
	$t = 1.0, k = 4$	6%	13%	10%	1%	1%	1%
GPT-3.5 Turbo	$t = 0.3, k = 2^*$	3%	5%	4%	0%	1%	0%
	$t = 0.3, k = 2$	3%	5%	4%	1%	1%	0%
	$t = 1.0, k = 2$	3%	5%	4%	0%	1%	0%
	$t = 1.0, k = 4$	1%	1%	0%	0%	0%	0%

Общий тренд, наблюдаемый в результатах:

- *Прирост метрик более выражен на сложных диалогах*. Например, при переходе от случайной выборки к длинным диалогам увеличение значений наблюдается для всех метрик, особенно для ROUGE-2 и ROUGE-L.
- *Модель GPT-3.5 Turbo демонстрирует более стабильные результаты*. Для неё прирост метрик остаётся небольшим, но стабильным, независимо от длины диалогов. В частности, максимальный прирост по ROUGE-2 составил 6% при $t = 1.0, k = 2$.
- *Liquid 3B показывает более чувствительные изменения*. В условиях сложных диалогов и увеличенного контекста ($k = 4$) прирост по ROUGE-2 достигает 16%, а по ROUGE-L — 12%, что указывает на способность модели лучше извлекать информацию из демонстрационных примеров.
- *Температура влияет на качество*. При $t = 1.0$ прирост метрик в среднем выше, чем при $t = 0.3$. Это может быть связано с тем, что более разнооб-

разные генерации помогают модели находить нестандартные решения в сложных диалогах.

Анализ показал, что использование релевантных демонстрационных примеров даёт ощутимый прирост качества, особенно на длинных и сложных медицинских диалогах. Это подтверждает, что для доменов с высокой сложностью контекста метод семантического отбора (Semantic ICL) является предпочтительным.

Вместе с тем, стоит отметить, что даже в таких условиях результаты крупной модели GPT-3.5 Turbo демонстрируют ограниченный прирост по сравнению с Liquid 3B. Это может быть связано с тем, что более крупные модели уже имеют мощные встроенные представления, которые позволяют им справляться с задачей без значительного увеличения качества от добавления демонстраций.

Таким образом, результаты эксперимента подчёркивают важность выбора гиперпараметров и характеристик данных. В задачах медицинской суммаризации, где длина и сложность контекста являются ключевыми факторами, использование Semantic ICL может существенно улучшить качество итоговых суммаризаций. Однако для задач с меньшими требованиями к контексту такой подход может оказаться избыточным.

Несмотря на положительные результаты, представленные в Таблице 4.4, при более детальном анализе оказалось, что базовый эксперимент нельзя считать полностью репрезентативным. Основная проблема заключается в том, что модели генерируют так называемые «сырые» ответы, которые не всегда соответствуют принятому в эталоне формату из четырёх полей — **Symptoms, Diagnosis, History of Patient, Plan of Action**. К примеру, обнаружены следующие типичные несоответствия:

- *Пропуски важных полей.* Модель может вообще не упомянуть один из разделов (скажем, “*Diagnosis*”), в то время как в эталоне чётко прописано “*Diagnosis: N/A*” или есть конкретные диагнозы. При прямом сравнении текста метрики ROUGE и BERTScore зачастую несправедливо искажают оценку за весь вывод из-за отсутствия ключевых слов в нужном месте, не отражая реального качества по другим полям.
- *Лишние заголовки и расслоение информации.* Вместо канонических заголовков (“*Symptoms*”, “*Plan of Action*” и т. д.) модель порой вставляет заголовки, например, “*Family Observations*”, “*Diagnostic Findings*”, или даже переносит часть данных в поля, которых нет в эталоне. Например, симптомы оказываются под “*Management*”, а диагноз фигурирует в разделе

“Discussion with Patient”. Это приводит к тому, что метрики практически не находят совпадений с эталонными блоками, и итоговые результаты выглядят искусственно заниженными.

В результате, при прямом сравнении “сырого” вывода модели и эталонного текста значительные расхождения могут возникать по формальным признакам, а не по сути передаваемой информации.

Чтобы исключить эти искажения и более объективно оценить качество суммаризаций, был проведён **дополнительный эксперимент с постобработкой**. Такой подход позволил **исправить** большинство несоответствий формата и убрать ложно завышенные или заниженные результаты. Как видно из приведённых примеров, в ряде случаев относительные изменения в ROUGE и BERTScore достаточно существенны. В следующих разделах будут показаны метрики, рассчитанные именно *после* постобработки.

Несмотря на положительные результаты, представленные в Таблице 4.4, важно отметить, что данные эксперименты проводились на «сыром» тексте, сгенерированном моделью. В ряде случаев наблюдались расхождения формата между эталонными сводками и ответами моделей. Например, одна из проблем заключается в том, что если модель (LLM) порождает ответ с новыми или неподходящими заголовками (например, «Medical Events», «Discussion with Family»), а эталон использует другие или меньше заголовков (скажем, только «Symptoms», «Diagnosis», «History of Patient», «Plan of Action»), то:

- На уровне слов (для ROUGE) новые названия заголовков и связанные формулировки могут «раздувать» гипотезу, добавляя лексику, которой нет в эталоне. Это уменьшает пересечение n-грамм и снижает итоговый показатель.
- На уровне семантики (для BERTScore) части гипотезы, не имеющие аналогов в эталоне, «останутся без пары». Это автоматически влечёт снижение метрики, поскольку доля «несопоставленных» токенов возрастает.

Таким образом, при глобальном сравнении текстов мы наблюдаем более низкие метрики из-за различий в структуре и появления «лишних» фрагментов, даже если модель формально содержит нужные факты. Чтобы избежать подобных искажений и более детально оценить содержимое каждого заголовка, мы используем **постобработку**, при которой остаются только те разделы, которые ровно соответствуют исходным заголовкам в задаче.

В эталонной разметке строго присутствуют четыре секции: «Symptoms», «Diagnosis», «History of Patient» и «Plan of Action». При этом каждая из них всегда указана, даже если фактического содержимого нет. Так, при отсутствии симптомов в поле «Symptoms» ставится «N/A», при отсутствии плана — «N/A», и так далее. Это подчёркивает логику: «перечислить все основные аспекты; если данных нет, явно прописать, что они отсутствуют».

Функция `parse_text_to_json()` «заставляет» и модель, и эталон следовать этой структуре:

- Ищутся только четыре заголовка: «Symptoms», «Diagnosis», «History of Patient», «Plan of Action».
- Всё, что следует за «чужими» заголовками (например, «Medical Events») — игнорируется и не учитывается при расчёте метрик.
- Если в гипотезе отсутствует «Symptoms:», поле «Symptoms» остаётся пустым.
- Если в эталоне «Diagnosis: N/A», а гипотеза вообще не вывела «Diagnosis», то мы имеем сравнение «N/A» с «'»». Хотя это даёт почти нулевой результат для ROUGE/BERTScore, такой расклад корректно отражает расхождение: модель не упомянула диагноза вообще, тогда как эталон явно написал, что никаких данных нет.

После этого у нас появляются «сложенные» тексты (или отдельные сегменты) ровно по четырём полям, которые можно сопоставить покомпонентно. Таким образом мы проверяем не только способность модели качественно суммировать диалог, но и её умение классифицировать информацию по нужным разделам. Любые «шумные» и несопоставимые заголовки («Medical Events», «Discussion with Family», и пр.) больше не влияют на метрики, избегая искусственного искажения результатов.

Когда модель вообще не выводит раздел (например, вместо «Symptoms: . . . » переходит сразу к «History of Patient»), нельзя считать это «N/A» по умолчанию. Основные причины следующие:

1. Неизвестно, упустила ли модель этот раздел по случайности, не «дочитала» диалог или потеряла связь с инструкцией.
2. Возможно, модель хотела указать симптомы, но по ошибке перенесла их в другое поле или забыла написать заголовок «Symptoms».
3. Неверно приписывать ей осознанную позицию «Symptoms: N/A», так как тогда мы лишь искусственно замаскировали тот факт, что раздел был пропущен.

Таблица 4.5

Пример медицинского диалога и его структурной суммаризации, сравнение между эталоном, Baseline ICL и Semantic ICL

Dialogue	Doctor: Welcome to the office, Miss A. I am Doctor Luna. Patient: Thank you. It is nice to meet you. Doctor: What is your family medical history? Patient: My mother died about three or maybe it is four years ago? Doctor: I am so sorry to hear that. How did she pass? Patient: She had lung cancer. My mom didn't talk much about her own health problems. Doctor: What about your father? Does he have any health conditions? Patient: My dad has gout. I think he also had blood clots. That is all that I can remember my dad talking about. Doctor: Do you have siblings? Patient: Yes. My brother has asthma and uses an inhaler. My sister had some sort of tumors removed from her stomach or something. Oh, I think I remember my mom talking about some of her family having issues with high blood pressure, high cholesterol and someone had a stroke. I am not sure though. Doctor: Okay.
Gold Summary	Symptoms: N/A Diagnosis: N/A History of Complaint: Difficulty providing familial medical history, mother deceased from lung cancer, father has gout and blood clots, siblings treated for asthma and GI tumors, uncertain about other familial medical conditions Plan of Action: N/A
Baseline ICL	1. Patient has a family history of cancer, gout, asthma, and stroke. 2. Mother died of lung cancer. 3. Father has gout and possibly a history of blood clots. 4. Brother has asthma. 5. Sister had tumors removed from her stomach. 6. Family history of high blood pressure, high cholesterol, and stroke.
Semantic ICL	Symptoms: N/A Diagnosis: N/A History of Patient: Mother died of lung cancer, father has gout and possibly blood clots, brother has asthma, sister had tumors removed from stomach, family history includes high blood pressure, high cholesterol, and stroke Plan of Action: N/A

В формальном смысле модель не соблюла требование «Сформировать четыре раздела», значит, она не выполнила часть задачи. Это рассматривается как ошибка: даже если в реальном диалоге симптомов нет, модель должна была явно указать «Symptoms: N/A», а не оставить поле вовсе без упоминания. Поэтому автозамена «если поле не упомянуто, значит N/A» неверна, ведь мы скрываем реальную ошибку (пропуск требуемого раздела) и тем самым даём модели *необоснованно высокую* оценку при сравнении с эталоном.

В качестве иллюстрации можно взять один из диалогов и три варианта аннотации (включая эталон) см. в Таблице 4.5. Если бы в процессе постобработки мы дописали к условному *Baseline ICL (Symptoms)* строку «N/A» там, где модель не вывела заголовки, мы бы искусственно создали видимость, будто модель *сообщила* об отсутствии информации. Такой подход автоматически повышает значения ROUGE/BERTScore при сопоставлении с эталоном (где тоже может стоять «N/A» или присутствовать «History of Patient» с похожим содержанием). По сути же LLM «random» не соблюла формат и не вывела нужную структуру.

При корректной постобработке разделы «Symptoms», «Diagnosis», «History of Patient», «Plan of Action» заполняются пустыми строками (""), если модель их явно не сформировала. Тогда при сравнении с эталоном, в котором, например, поле «History of Patient» не пустое, метрики показывают низкие совпадения, указывая, что модель проигнорировала часть задания.

ROUGE сопоставляет n-граммы. Если случайным образом поместить чей-то текст под «History of Patient» и одновременно выставить «Symptoms: N/A», «Diagnosis: N/A», «Plan of Action: N/A», можно почти дословно скопировать структуру эталона, существенно повышая число совпадающих n-грамм (особенно для ROUGE-1 и ROUGE-2).

BERTScore ищет семантические соответствия. Когда мы проставляем «N/A» в пропущенные поля, а в эталоне те же поля содержат «N/A», появляется определённое семантическое сходство совпадающее по факту с максимальным значением этой метрики. Более того, если мы «на лету» перенесём перечисления (1,2,3,...) из вывода модели в раздел «History of Patient», BERTScore может «распознать» его близким к эталонному «History. . .», хотя исходно модель очевидно не только не соблюдала требуемую структуру, но и по факту не справилась с задачей.

В итоге модель, которая фактически *не* придерживалась заданных четырёх разделов, может выглядеть (по метрикам) почти так же хорошо, как модель, корректно

заполнившая все поля. С другой стороны, «жёсткая» оценка, при которой пропуски явно снижают итоговые баллы, корректно отражает расхождение: эталонная запись указывает «нет диагноза», а модель этот раздел совсем не упомянула.

4.3.3. Оптимизированная оценка методики.

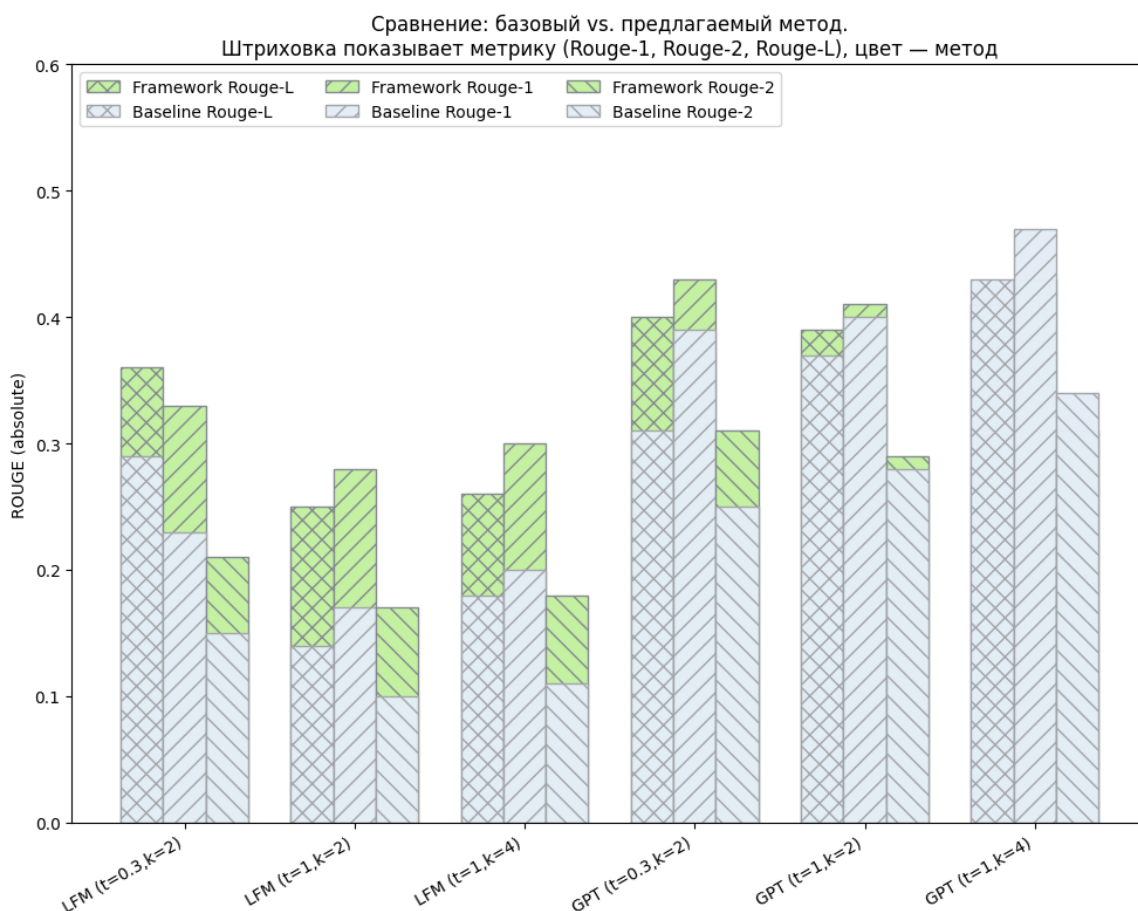


Рис.4.4. Сравнение базового подхода и предложенного метода по метрикам ROUGE. Штриховка обозначает различные метрики (ROUGE-1, ROUGE-2, ROUGE-L), цвет указывает метод

В этом разделе рассматриваются результаты эксперимента, направленного на анализ влияния предложенного фреймворка на улучшение качества суммаризаций диалогов. Эксперимент был проведён с использованием двух языковых моделей: компактной **Liquid 3B** и более крупной **GPT-3.5 Turbo**. Основной задачей эксперимента было сравнить прирост метрик ROUGE и BERTScore при различных значениях гиперпараметров температуры (t) и количества демонстрационных примеров (k).

Все итоговые суммаризации проходили через процедуру постобработки, описанную в предыдущих разделах (см. раздел о необходимости унификации «Symptoms, Diagnosis, History of Patient, Plan of Action»). Также обращалось

внимание на то, что если модель пропустила поле, оно считалось пустым, а не «N/A».

Таблица 4.6 демонстрирует прирост метрик в процентах, а графики на Рисунках 4.4 и 4.5 иллюстрируют абсолютные значения метрик для базового метода и предложенного фреймворка.

Таблица 4.6

Прирост метрик ROUGE и BERTScore для различных гиперпараметров и моделей на медицинских диалогах (MTS Dialogue-Clinical Note)

LLM	Parameters	Sub-dataset	R-1 (growth)	R-2 (growth)	R-L (growth)	PR (growth)	RC (growth)	F1 (growth)
LFM 3B	$t = 0.3, k = 2^*$	Symptoms	4%	4%	4%	2%	2%	2%
		Diagnosis	14%	13%	14%	8%	7%	6%
		HoP	3%	4%	4%	-5%	-5%	-5%
		Plan of Action	5%	4%	5%	6%	5%	6%
		Overall	6%	6%	6%	3%	2%	3%
LFM 3B	$t = 0.3, k = 2$	Symptoms	35%	37%	37%	35%	32%	34%
		Diagnosis	124%	122%	122%	156%	154%	155%
		HoP	37%	37%	34%	27%	26%	27%
		Plan of Action	-6%	-17%	-9%	16%	15%	16%
		Overall	48%	45%	46%	58%	57%	58%
LFM 3B	$t = 1.0, k = 2$	Symptoms	64%	73%	69%	53%	52%	52%
		Diagnosis	112%	112%	112%	120%	119%	120%
		HoP	55%	60%	63%	45%	46%	45%
		Plan of Action	39%	26%	36%	39%	39%	39%
		Overall	68%	68%	70%	77%	77%	77%
LFM 3B	$t = 1.0, k = 4$	Symptoms	42%	51%	43%	26%	24%	25%
		Diagnosis	167%	158%	156%	162%	160%	161%
		HoP	57%	54%	57%	35%	35%	35%
		Plan of Action	-9%	-12%	-9%	18%	18%	18%
		Overall	63%	59%	61%	60%	59%	60%
GPT-3.5	$t = 0.3, k = 2^*$	Symptoms	7%	8%	7%	2%	2%	2%
		Diagnosis	8%	7%	8%	3%	3%	4%
		HoP	-5%	-6%	-6%	-2%	-2%	-2%
		Plan of Action	16%	19%	17%	3%	3%	3%
		Overall	6%	7%	6%	0%	0%	0%
GPT-3.5	$t = 0.3, k = 2$	Symptoms	14%	18%	13%	9%	10%	10%
		Diagnosis	5%	4%	6%	10%	9%	10%
		HoP	9%	9%	9%	4%	5%	5%
		Plan of Action	19%	27%	21%	11%	8%	9%
		Overall	12%	15%	12%	9%	9%	9%
GPT-3.5	$t = 1.0, k = 2$	Symptoms	4%	3%	4%	11%	11%	12%
		Diagnosis	-5%	-8%	-5%	13%	12%	13%
		HoP	8%	13%	9%	7%	9%	7%
		Plan of Action	10%	7%	11%	13%	12%	13%
		Overall	4%	4%	5%	11%	11%	11%
GPT-3.5	$t = 1.0, k = 4$	Symptoms	-5%	-6%	-5%	10%	9%	10%
		Diagnosis	-8%	-13%	-9%	-2%	-2%	-2%
		HoP	-5%	-8%	-9%	-2%	-2%	-2%
		Plan of Action	-13%	-18%	-15%	-2%	-2%	-2%
		Overall	-8%	-9%	-8%	0%	0%	0%

Результаты модели Liquid 3B: модель Liquid 3B показала значительный прирост по всем метрикам, особенно при увеличении количества демонстрационных примеров ($k = 2$ и $k = 4$) и температуры ($t = 1.0$). Максимальный прирост наблюдается при параметрах $t = 1.0, k = 2$, где метрики ROUGE увеличились на 68%, а BERTScore — на 77%. Это подтверждает эффективность предложенного фреймворка для выбора релевантных демонстраций в few-shot обучении.

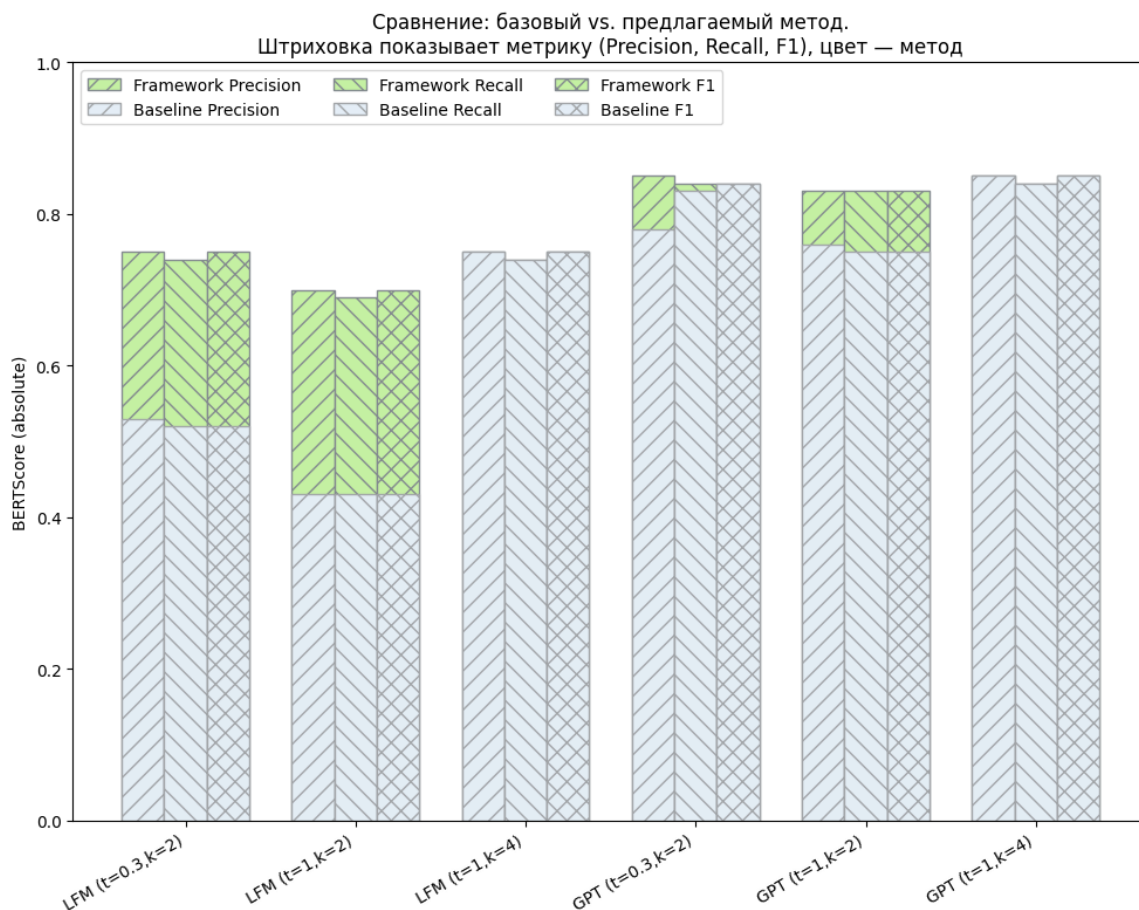


Рис.4.5. Сравнение базового подхода и предложенного метода по метрикам BERTScore (Precision, Recall, F1). Штриховка обозначает различные метрики, цвет указывает метод

Результаты модели GPT-3.5 Turbo: для GPT-3.5 Turbo прирост метрик оказался менее выраженным, но всё же значительным. Максимальные значения были достигнуты при $t = 1.0, k = 2$, где прирост BERT F1 составил 11%. Однако увеличение количества демонстрационных примеров ($k = 4$) привело к снижению метрик, что может свидетельствовать о переизбытке информации в инструкции, перегружающей модель.

На случайных диалогах ($t = 0.3, k = 2^*$) прирост отсутствовал, что, вероятно, связано с меньшей сложностью этих данных и слабой релевантностью демонстраций.

Сравнение между моделями: модель Liquid 3B демонстрирует лучшее качество суммаризаций по сравнению с GPT-3.5 Turbo, несмотря на меньший размер. Это подчёркивает важность правильной настройки гиперпараметров и релевантности демонстрационных примеров. В случае GPT-3.5 Turbo предложенный метод работает менее эффективно, что может быть связано с особенностями обработки длинных контекстов этой моделью.

Результаты эксперимента показывают, что использование фреймворка для выбора релевантных демонстрационных примеров приводит к значительному улучшению качества суммаризаций. Особенную выгоду предложенный метод демонстрирует на сложных и длинных диалогах, где базовый метод (случайный отбор демонстраций) оказывается менее эффективным.

Постобработка сыграла ключевую роль в объективной оценке результатов. Приведение текстов к единому формату с полями «Symptoms», «Diagnosis», «History of Patient» и «Plan of Action» позволило исключить искусственное искажение метрик из-за различий в структуре вывода модели и эталона. Как показали эксперименты, без постобработки прирост метрик мог быть занижен или завышен из-за несоответствий формата. Использование унифицированного подхода дало возможность честно оценить способность моделей генерировать релевантные и структурированные суммаризации.

Таким образом, результаты эксперимента подтверждают, что предложенная методика является перспективным решением для задач диалоговой суммаризации. Модель Liquid 3B, несмотря на меньший размер, показала сравнимые или даже лучшие результаты в сравнении с GPT-3.5 Turbo, что подчёркивает её эффективность в условиях ограниченных ресурсов. В то же время, крупные модели, такие как GPT-3.5 Turbo, также выигрывают от использования семантически релевантных демонстрационных примеров, особенно при оптимальном выборе гиперпараметров.

Предложенный подход открывает перспективы для дальнейших исследований, особенно в доменах с высокой сложностью контекста, таких как медицина и юриспруденция. Улучшение качества суммаризаций на сложных диалогах показывает, что семантический отбор примеров может стать ключевым инструментом в задачах автоматической обработки текста.

Эксперимент 1 подтвердил, что стратегия выбора демонстрационных примеров оказывает существенное влияние на качество суммаризаций диалогов. Полученные результаты демонстрируют, что семантически релевантный отбор примеров (Semantic ICL) обеспечивает устойчивое улучшение по сравнению с

случайным выбором (Baseline ICL) и сценарием без использования демонстраций (No ICL).

Для небольших моделей, таких как Liquid 3B, прирост по метрикам ROUGE и BERTScore был наиболее заметным, что подчёркивает способность предложенного подхода компенсировать ограничения моделей меньшего размера. В частности, улучшения достигали 50–70% на более сложных и длинных диалогах, таких как специализированные медицинские диалоги. Это особенно важно для задач, требующих высокой точности и структурированности, где даже небольшой прирост может существенно повлиять на качество работы системы.

Для крупных моделей, таких как GPT-3.5 Turbo, прирост оказался менее значительным, что связано с изначально высоким качеством их генерации. Тем не менее, даже для таких моделей семантический отбор демонстрационных примеров показал свою эффективность, обеспечив улучшение в диапазоне 5–15% по ключевым метрикам. Это доказывает, что предложенная методика остаётся актуальной для использования с передовыми моделями.

Анализ также выявил, что влияние стратегии выбора демонстрационных примеров варьируется в зависимости от сложности и длины диалогов. На простых и коротких текстах (например, SamSum) улучшения были менее выраженными, что связано с меньшей сложностью контекста. В то же время для длинных и сложных текстов, таких как медицинские диалоги, методика Semantic ICL показала свои преимущества наиболее ярко.

Полученные данные подчёркивают, что предложенный подход имеет универсальный характер, демонстрируя эффективность как для небольших, так и для крупных моделей, а также для различных типов данных. Эти результаты подтверждают практическую ценность использования фреймворка для автоматической суммаризации диалогов, особенно в доменах, требующих глубокого анализа контекста и адаптивного выбора обучающих примеров.

4.4. Эксперимент 2: Instruction Tuning на эталонных и синтетических данных

В данном эксперименте сравнивалось качество моделей, обученных методом *instruction tuning*, причём для обучения использовались два кардинально разных типа данных: полностью человечески аннотированные (далее – эталонные) и автоматически сгенерированные (далее – синтетические). Основным вопросом эксперимента состоял в том, насколько близко может подойти итоговое качество

суммаризаций на синтетических наборах к результатам ручной разметки, а также в том, является ли полностью эталонный подход единственным путём к высокому показателю на дообученных моделях.

Цель эксперимента—определить, в какой мере разные источники обучающих данных (эталонные, синтетические и их смесь) влияют на показатели суммаризации, когда модель дообучается с учётом инструкций. Основная гипотеза гласит, что полностью эталонные данные обеспечат наилучшее качество, однако синтетика может дать сравнимые результаты при существенно меньших затратах.

Модель-основа - Vikhrmodels/Vikhr-Llama-3.2-1B-Instruct, обладающая 1 миллиардом параметров и уже умеющая интерпретировать инструкции, выступала в роли дообучаемого решения. Все сценарии настраивались в формате *instruction tuning*, когда модель не просто обучается на паре «текст–суммаризация», а получает специальную инструкцию (prompt), уточняющую требования к результату.

С точки зрения обучающих подходов рассматривались:

- *Эталонная модель*: обученная исключительно на вручную размеченных диалогах из Bragovo/dsum_ru, где каждая аннотация максимально корректна и подлинна.
- *Синтетическая модель*: для неё использовались псевдо-аннотации, сгенерированные фреймворком без участия человека. Хотелось выяснить, приведёт ли такая схема к заметному ухудшению суммаризаций или сохранит сопоставимый уровень.

На обеих моделях применяли единый набор гиперпараметров: batch size = 32, 5 эпох обучения (Epochs = 5) и скорость обучения (Learning rate) равную 3×10^{-5} . В ходе предварительного анализа эти настройки показали разумный баланс между скоростью сходимости и качеством итогового вывода.

Для оценки использовались детализированные показатели BERTScore: Precision, Recall и F1. С помощью этой метрики можно выявить, насколько сгенерированные тексты (суммаризации) семантически совпадают с эталоном. Результаты сведены в Таблице 4.7.

Как следует из Таблицы 4.7, показатели BERTScore для эталонного и синтетического обучения оказались практически идентичными. В частности, эталонная модель продемонстрировала значение Precision равное 0.65, Recall равное 0.64 и F1 равное 0.65, что свидетельствует о том, что вручную размеченные суммаризации предоставляют достаточно точные эталоны для обучения модели. С другой

Модель	Precision	Recall	F1
Эталонная	0.65	0.64	0.65
Синтетическая	0.64	0.64	0.65

Таблица 4.7

Показатели BERTScore для эталонного и синтетического обучения

стороны, синтетическая модель показала значения Precision 0.64, Recall 0.64 и F1 0.65, что указывает на то, что автоматически сгенерированные данные способны обеспечить уровень семантической релевантности, сопоставимый с данными ручной разметки. Разница между моделями минимальна, что демонстрирует высокую эффективность псевдо-аннотаций, позволяющих достичь практически идентичных результатов по всем трём метрикам.

В итоге эксперимент показывает:

- **Эталонная модель** демонстрирует показатели BERTScore, где F1 составляет 0.65, подтверждая, что высокая достоверность ручной разметки способствует получению качественных суммаризаций.
- **Синтетические данные** также приводят к значению F1 равному 0.65, что свидетельствует о том, что автоматизированная генерация аннотаций способна обеспечить уровень качества, сравнимый с ручной разметкой, при этом позволяя значительно сократить затраты на создание обучающих наборов.

Применение оптимизации с использованием метода LoRA стало важным дополнением к процессу дообучения. Данный подход позволяет значительно сократить количество обучаемых параметров за счет введения низкоранговых матриц, адаптирующих лишь наиболее критичные компоненты модели, отвечающие за передачу семантической информации. Это приводит к уменьшению вычислительной нагрузки и снижению требований к оперативной памяти, что особенно актуально при работе с крупными моделями, такими как рассматриваемая в эксперименте. Более того, LoRA способствует ускорению процесса сходимости, поскольку обновляются только те веса, которые непосредственно влияют на итоговый результат.

Интеграция LoRA оптимизации в схему *instruction tuning* позволила добиться дополнительной экономии вычислительных ресурсов без ущерба для качества суммаризаций. Экспериментальные данные показали, что применение данного метода способствует улучшению обобщающих способностей модели, ускоряет процесс обучения и снижает риск переобучения. Такой подход открывает перспективы для

дальнейшего масштабирования и адаптации языковых моделей к новым задачам, позволяя эффективно использовать автоматизированные схемы дообучения даже в условиях ограниченных вычислительных мощностей и большого объёма данных.

Таким образом, полученные результаты демонстрируют, что синтетическая модель, обученная на автоматически сгенерированных данных, достигает показателей BERTScore, сопоставимых с моделью, обученной на эталонных данных. Факт равенства F1 значений (0.65) для обеих моделей свидетельствует о том, что применение метода *instruction tuning* на псевдо-аннотированных диалогах способно обеспечить качество, близкое к результатам ручной разметки, что значительно расширяет возможности эффективного и экономичного обучения крупных языковых моделей при минимизации затрат на создание обучающих наборов.

4.5. Выводы

Проведённый экспериментальный анализ предложенного фреймворка подтвердил его эффективность в задаче автоматической аннотации и суммаризации диалогов. Эксперименты продемонстрировали, что использование семантически релевантных демонстрационных примеров в *few-shot* обучении (Semantic ICL) приводит к значительному улучшению качества итоговых суммаризаций по сравнению со случайным выбором примеров (Baseline ICL). Особенно выражены эти преимущества в задачах с длинными и сложными текстами, требующими глубокого контекстуального понимания.

Анализ метрик (ROUGE, BERTScore, GPT-based evaluation) показал, что:

- Введение семантического выбора демонстрационных примеров приводит к улучшению качества суммаризаций, особенно на сложных корпусах данных, таких как медицинские диалоги.
- Для простых и коротких диалогов (например, SamSum) разница между методами Semantic ICL и Baseline ICL оказалась менее значительной, что может быть связано с меньшей сложностью контекста.
- Автоматически аннотированные данные оказались сопоставимы по качеству с вручную размеченными наборами, что подтверждается результатами *instruction tuning*. Это открывает перспективу значительного снижения затрат на создание обучающих наборов для языковых моделей без потери качества.

Кроме того, эксперимент с обучением модели (Vikhr-Llama-3.2-1B-Instruct) на синтетических данных продемонстрировал, что использование псевдо-аннотаций, сгенерированных фреймворком, даёт результаты, практически не уступающие эталонной разметке. Это подтверждает возможность широкого применения метода для генерации обучающих данных, снижая потребность в трудоёмкой ручной аннотации.

Таким образом, проведённые исследования демонстрируют, что предложенный подход является универсальным инструментом для задач автоматической аннотации и суммаризации диалогов. Он показывает высокую эффективность как при использовании небольших языковых моделей, так и в сочетании с крупными LLM.

ЗАКЛЮЧЕНИЕ

В данной выпускной квалификационной работе решалась задача автоматической аннотации и суммаризации диалогов в условиях дефицита размеченных данных. Эта проблема особенно актуальна для диалоговых систем, где качество и точность выходных данных напрямую зависят от доступности качественных обучающих наборов. В рамках исследования был разработан и реализован фреймворк, который позволяет автоматизировать процесс аннотации диалогов и создавать высококачественные наборы данных без значительных затрат на ручную разметку.

В ходе работы был проведён подробный анализ современных методов диалоговой суммаризации, включая использование больших языковых моделей в режиме ICL, подходов с RAG, а также традиционных методов, основанных на дообучении моделей. Изучение существующих подходов позволило выявить их ограничения и обосновать необходимость предлагаемого метода. В рамках исследования был предложен метод автоматической аннотации, основанный на отборе демонстрационных примеров с использованием предобученных энкодеров текстовых представлений (USER-bge-m3, GTE-base). Данная методика позволила интегрировать семантически релевантные демонстрационные примеры в промпт и использовать их в контексте *few-shot in-context learning*, что повысило качество итоговых аннотаций.

Разработанный подход был подвергнут всесторонней экспериментальной оценке на нескольких наборах данных, включая англоязычные диалоги SamSum, русскоязычные диалоги Bragovo/dsum_ru, а также специализированные медицинские диалоги. Проведённые эксперименты показали, что предложенный метод аннотации даёт значительное улучшение метрик ROUGE и BERTScore по сравнению с базовым методом случайного выбора демонстрационных примеров. Особенно важные результаты были получены в задачах суммаризации медицинских диалогов, где случайный отбор демонстрационных примеров показывал лишь ограниченное улучшение, тогда как использование предложенного фреймворка обеспечивало значительный прирост качества итоговых аннотаций. Данный факт подтверждает особую полезность разработанного метода в сложных и специализированных сценариях, требующих высокой точности и релевантности аннотаций.

Дополнительно был проведён эксперимент по дообучению языковых моделей с помощью техники *instruction tuning* на данных, созданных с использованием предложенного метода. Результаты показали, что модели, обученные на автома-

тически аннотированных данных, достигли качества, сопоставимого с моделями, обученными на вручную размеченных данных. Это подтверждает эффективность разработанного фреймворка и его потенциал в задачах автоматического создания обучающих наборов данных для дальнейшего обучения языковых моделей.

Практическая значимость проделанной работы заключается в том, что разработанный метод позволяет существенно сократить затраты на разметку данных, предоставляя автоматизированный способ формирования высококачественных обучающих наборов. Предложенный подход демонстрирует высокую универсальность, эффективно работая как с общими диалогами, так и со специализированными областями, такими как медицинская суммаризация, где точность аннотаций играет критически важную роль. Кроме того, экспериментальные данные, полученные в ходе работы, могут быть использованы для выбора оптимальной стратегии аннотации диалоговых данных, а также для дальнейшего развития методов автоматической генерации обучающих наборов с использованием LLM.

Проведённые эксперименты подтвердили эффективность разработанного фреймворка, показав, что использование семантически релевантных демонстрационных примеров значительно повышает качество суммаризаций по сравнению с методами, основанными на случайном отборе. Данный эффект наблюдается как на небольших языковых моделях, где выигрыш особенно велик, так и на более мощных моделях, таких как GPT-3.5, что подчёркивает применимость метода в широком диапазоне задач. Разработанная методика может найти применение в различных сферах, включая автоматизацию колл-центров, системы поддержки пользователей и медицинские сервисы, где требуется точное и структурированное резюмирование диалогов.

Итоговые результаты демонстрируют, что слияние RAG и контекстного обучения LLM обеспечивает надёжный и гибкий механизм автоматической аннотации диалогов. Эксперименты показали, что даже без ручной разметки можно достичь достаточно высоких показателей качества аннотирования согласно метрикам ROUGE и BERTScore. Сохраняя широкую применимость (от кратких повседневных диалогов до медицинских консультаций), фреймворк значительно уменьшает трудоёмкость подготовки обучающих наборов данных. Следовательно, предложенная методика открывает дополнительные возможности для эффективной обработки диалоговой информации в различных областях, требующих точного и масштабируемого решения.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

LLM	Large Language Model (Большая языковая модель).
ICL	In-Context Learning (Обучение в контексте).
RAG	Retrieval-Augmented Generation (Расширенная генерация)
NLP	Natural Language Processing (Обработка естественного языка)
SOTA	State-of-the-Art (Текущий уровень передовых технологий)
Few-shot	Обучение с малым количеством примеров
Zero-shot	Обучение без примеров
BERT	Bidirectional Encoder Representations from Transformers
GPT	Generative Pre-trained Transformer

СЛОВАРЬ ТЕРМИНОВ

Суммаризация диалогов — процесс создания краткого и содержательного резюме разговоров, включающего основные идеи и факты, выраженные в исходном диалоге.

Большие языковые модели (LLM) — нейросетевые архитектуры, обученные на больших объемах текстовых данных для выполнения задач обработки естественного языка, таких как генерация текста, перевод и резюмирование.

Трансформер (Transformer) — архитектура нейронной сети, основанная на механизме внимания (attention), которая используется для обработки последовательностей данных, таких как текст. Она лежит в основе многих современных языковых моделей, включая GPT и BERT.

Обучение в контексте (ICL) — метод использования языковых моделей, при котором обучение выполняется с использованием небольшого числа примеров, задаваемых в виде подсказок в процессе выполнения задачи.

Обучение с малым количеством примеров (Few-Shot Learning) — подход к обучению моделей, при котором модель успешно решает задачи, используя лишь несколько размеченных примеров.

Расширенная генерация (RAG) — метод генерации текста, сочетающий информационный поиск и генерацию, где модели используются для поиска релевантного контекста в базе данных и последующего построения ответа или резюме.

Синтетические данные — данные, созданные автоматически с использованием алгоритмов или моделей, в отличие от данных, собранных или размеченных вручную.

Обработка естественного языка (NLP) — область искусственного интеллекта, связанная с анализом, обработкой и генерацией человеческого языка с использованием алгоритмов машинного обучения.

Тонкая настройка (Fine-Tuning) — процесс дообучения предварительно обученной модели на специализированных данных для улучшения её производительности на конкретной задаче.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Русскоязычный набор диалогов и аннотаций. — URL: https://huggingface.co/datasets/bragovo/dsum_ru (дата обращения: 17.11.2024).
2. A survey on in-context learning / Q. Dong [и др.] // arXiv preprint arXiv:2301.00234. — 2022.
3. Abstractive text summarization using sequence-to-sequence rnns and beyond / R. Nallapati [и др.] // arXiv preprint arXiv:1602.06023. — 2016.
4. *Ahmed T., Devanbu P.* Few-shot training LLMs for project-specific code-summarization // Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. — 2022. — С. 1—5.
5. Automatic semantic augmentation of language model prompts (for code summarization) / T. Ahmed [и др.] // Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. — 2024. — С. 1—13.
6. Baichuan2-Sum: Instruction Finetune Baichuan2-7B Model for Dialogue Summarization / J. Xiao [и др.] // 2024 International Joint Conference on Neural Networks (IJCNN). — IEEE. 2024. — С. 1—8.
7. Baize: An open-source chat model with parameter-efficient tuning on self-chat data / C. Xu [и др.] // arXiv preprint arXiv:2304.01196. — 2023.
8. Benchmarking large language models for news summarization / T. Zhang [и др.] // Transactions of the Association for Computational Linguistics. — 2024. — Т. 12. — С. 39—57.
9. Effective and efficient conversation retrieval for dialogue state tracking with implicit text summaries / S. Lee [и др.] // arXiv preprint arXiv:2402.13043. — 2024.
10. Human-instruction-free llm self-alignment with limited samples / H. Guo [и др.] // arXiv preprint arXiv:2401.06785. — 2024.
11. In-context learning of large language models for controlled dialogue summarization: A holistic benchmark and empirical analysis / Y. Tang [и др.] // Proceedings of the 4th New Frontiers in Summarization Workshop. — 2023. — С. 56—67.
12. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning / M. Geng [и др.] // Proceedings of the 46th IEEE/ACM International Conference on Software Engineering. — 2024. — С. 1—13.
13. On learning to summarize with large language models as references / Y. Liu [и др.] // arXiv preprint arXiv:2305.14239. — 2023.

14. Overview of the MEDIQA-Chat 2023 Shared Tasks on the Summarization & Generation of Doctor-Patient Conversations / A. Ben Abacha [и др.] // Proceedings of the 5th Clinical Natural Language Processing Workshop / под ред. T. Naumann [и др.]. — Toronto, Canada: Association for Computational Linguistics, 2023. — С. 503—513. — DOI 10.18653/v1/2023.clinicalnlp-1.52. — URL: <https://aclanthology.org/2023.clinicalnlp-1.52/>.
15. *Pu X., Gao M., Wan X.* Summarization is (almost) dead // arXiv preprint arXiv:2309.09558. — 2023.
16. *Ramprasad S., Ferracane E., Lipton Z. C.* Analyzing LLM Behavior in Dialogue Summarization: Unveiling Circumstantial Hallucination Trends. — 2024. — arXiv: 2406.03487 [cs.CL]. — URL: <https://arxiv.org/abs/2406.03487>.
17. Revisiting the gold standard: Grounding summarization evaluation with robust human evaluation / Y. Liu [и др.] // arXiv preprint arXiv:2212.07981. — 2022.
18. *Sahu G., Laradji I. H.* MixSumm: Topic-based Data Augmentation using LLMs for Low-resource Extractive Text Summarization // arXiv preprint arXiv:2407.07341. — 2024.
19. Salmon: Self-alignment with principle-following reward models / Z. Sun [и др.] // CoRR. — 2023.
20. Self-instruct: Aligning language models with self-generated instructions / Y. Wang [и др.] // arXiv preprint arXiv:2212.10560. — 2022.
21. Training language models to follow instructions with human feedback / L. Ouyang [и др.] // Advances in neural information processing systems. — 2022. — T. 35. — С. 27730—27744.
22. Understanding factual errors in summarization: Errors, summarizers, datasets, error detectors / L. Tang [и др.] // arXiv preprint arXiv:2205.12854. — 2022.
23. Visual instruction tuning / H. Liu [и др.] // Advances in neural information processing systems. — 2024. — T. 36.
24. *Wang B., Liu Z., Chen N. F.* Instructive dialogue summarization with query aggregations // arXiv preprint arXiv:2310.10981. — 2023.
25. *Work W. M. I.-C. L.* Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? — 2022. — URL: <https://openreview.net/forum?id=cnRGMv-Ak7u> (дата обращения: 17.11.2024).
26. SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization. — URL: <https://huggingface.co/datasets/Samsung/samsum> (visited on 17.11.2024).

Основные модули фреймворка

Листинг 1.1

dataset_loader.py

```

from datasets import load_dataset
import os
from typing import Optional, Dict, Any, Literal
5 from datetime import datetime
import pyarrow as pa
import pyarrow.parquet as pq
import h5py
import pandas as pd
10
class DatasetLoader:
    def __init__(self, cache_dir: str = "/data",
        save_format: Literal["parquet", "hdf5"] = "parquet"):
        """
15 Инициализация загрузчика датасетов.
        Args:
        cache_dir: Директория для сохранения датасетов
        save_format: Формат сохранения ("parquet" или "hdf5")
        """
20 self.cache_dir = cache_dir
        self.format = save_format
        os.makedirs(cache_dir, exist_ok=True)

    def _save_parquet(self, data, filepath: str, metadata: Dict):
25 """Сохранение в формате Parquet"""
        # Handle both dataset objects and dictionaries
        if hasattr(data, 'column_names'):
            # Dataset object
            data_dict = {col: data[col] for col in data.column_names}
30 else:
            # Dictionary
            data_dict = data

        table = pa.Table.from_pydict(data_dict)
35
        # Добавляем метаданные в схему
        schema = table.schema
        for key, value in metadata.items():

```

```

schema = schema.with_metadata({key.encode(): str(value).encode
    ()})
40
table = table.cast(schema)
pq.write_table(table, filepath, compression='snappy')

def _save_hdf5(self, data, filepath: str, metadata: Dict):
45 """Сохранение в формате HDF5"""
    with h5py.File(filepath, 'w') as f:
        # Сохраняем данные
        dataset_group = f.create_group('data')

50 # Handle both dataset objects and dictionaries
    if hasattr(data, 'column_names'):
        # Dataset object
        items = [(col, data[col]) for col in data.column_names]
    else:
55 # Dictionary
        items = data.items()

    for key, values in items:
        if len(values) > 0: # Check if there are any values
60 if isinstance(values[0], str):
            dt = h5py.special_dtype(vlen=str)
            dataset_group.create_dataset(key, data=values, dtype=dt)
        else:
            dataset_group.create_dataset(key, data=values)

65 # Сохраняем метаданные
        metadata_group = f.create_group('metadata')
        for key, value in metadata.items():
            metadata_group.attrs[key] = str(value)

70
def load_from_hub(self,
    dataset_name: str,
    subset: Optional[str] = None,
    split: str = "train",
75 save: bool = True,
    trust_remote_code: bool = True) -> Dict[str, Any]:
    """
    Загрузка датасета с Hugging Face Hub.
    Args:
80 dataset_name: Название датасета на Hugging Face
    subset: Поднабор данных (если есть)
    split: Разбиение датасета (train/test/validation)

```

```

save: Сохранить ли датасет локально
trust_remote_code: Доверять ли удаленному коду для загрузки да
    тасета
85 Returns:
    Загруженный датасет
    """
    try:
        # Загрузка датасета напрямую с указанным split
90 dataset = load_dataset(
    dataset_name,
    name=subset,
    split=split,
    trust_remote_code=trust_remote_code
95 )

    # Получаем словарь с данными
    data_dict = {}
    for feature in dataset.features:
100 data_dict[feature] = dataset[feature]

    if save:
        # Генерация имени файла
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
105 filename = f"{dataset_name.replace('/', '_')}_{{split}}_{{
            timestamp}}.{{self.format}}"
        filepath = os.path.join(self.cache_dir, filename)

        metadata = {
            "dataset_name": dataset_name,
110 "subset": subset,
            "split": split,
            "download_date": timestamp,
            "num_examples": len(dataset),
            "columns": list(dataset.features.keys())
115 }

        if self.format == "parquet":
            self._save_parquet(dataset, filepath, metadata)
        else:
120 self._save_hdf5(dataset, filepath, metadata)

        print(f"Dataset saved to {filepath}")

        return data_dict
125

```



```

except Exception as e:
    raise Exception(f"Error loading dataset: {str(e)}")

def load_local(self, filename: str) -> Dict[str, Any]:
130 """
    Загрузка локально сохраненного датасета.
    Args:
    filename: Имя файла в cache_dir
    Returns:
135 Загруженный датасет
    """
    filepath = os.path.join(self.cache_dir, filename)

    if filename.endswith('.parquet'):
140 table = pq.read_table(filepath)
        metadata = {k.decode(): v.decode()
                     for k, v in table.schema.metadata.items()}
        return {
            'data': table.to_pydict(),
145 'metadata': metadata
        }

    elif filename.endswith('.hdf5'):
        with h5py.File(filepath, 'r') as f:
150 data = {k: list(v[:]) for k, v in f['data'].items()}
        metadata = dict(f['metadata'].attrs)
        return {'data': data, 'metadata': metadata}

    else:
155 raise ValueError(f"Unsupported file format: {filename}")

def save_dataset(self, dataset_dict: Dict[str, Any], filepath:
                  str) -> None:
    """
    Сохраняет датасет с эмбедингами.
160 Args:
    dataset_dict: Словарь с данными и метаданными
    filepath: Путь для сохранения
    """
    try:
165 if filepath.endswith('.parquet'):
        self._save_parquet(dataset_dict['data'], filepath,
                           dataset_dict['metadata'])
    elif filepath.endswith('.hdf5'):

```

```

self._save_hdf5(dataset_dict['data'], filepath, dataset_dict['
    metadata'])
else:
170 raise ValueError(f"Unsupported file format: {filepath}")

print(f"Enhanced dataset saved to {filepath}")
except Exception as e:
raise Exception(f"Error saving enhanced dataset: {str(e)}")

```

Листинг 1.2

embeddings.py

```

import torch
from transformers import AutoTokenizer, AutoModel
from torch.utils.data import DataLoader
5 from typing import List, Dict
import numpy as np
from tqdm import tqdm
import pickle
from datetime import datetime
10 import os

class EmbeddingGenerator:
def __init__(self, model_name: str, device: str = None,
    trust_remote_code: bool = True):
    """
15 Инициализация генератора эмбедингов.
    Args:
    model_name: Название модели из transformers.
    device: Устройство для вычислений ("cuda" или "cpu").
    trust_remote_code: Разрешить выполнение пользовательского ко
        да из репозитория модели.
20 """
    # Check device availability
    if device is None:
    device = "cuda" if torch.cuda.is_available() else "cpu"
    else:
25 if device == "cuda" and not torch.cuda.is_available():
    print("Warning: CUDA requested but not available. Using CPU
        instead.")
    device = "cpu"

    if device == "cpu":
30 print("Warning: Using CPU for computations. This might be
        slow for large datasets.")

```

```

else:
    print(f"Using device: {device}")

    self.device = device
35 print(f"Loading tokenizer and model from {model_name}")
    print(f"Trust remote code: {trust_remote_code}")

    self.tokenizer = AutoTokenizer.from_pretrained(
        model_name,
40 trust_remote_code=trust_remote_code
    )

    # Retry mechanism for model loading
    retries = 3
45 for attempt in range(retries):
    try:
        self.model = AutoModel.from_pretrained(
            model_name,
            trust_remote_code=trust_remote_code
50 ).to(self.device)
        break
    except FileNotFoundError as e:
        if attempt < retries - 1:
            print(f"Retrying model download ({attempt + 1}/{retries})...
                ")
55 else:
        raise Exception(f"Failed to load model after {retries}
            attempts. Error: {str(e)}")

    # Get embedding dimension from model config
    self.embedding_dim = self.model.config.hidden_size
60 print(f"Model embedding dimension: {self.embedding_dim}")

    self.max_model_length = self.tokenizer.model_max_length

    def _generate_filename(self, prefix: str) -> str:
65 """
        Generate filename for embeddings.
        Format: {prefix}_{model_name}_{batch_size}_{max_length}_{
            timestamp}.pkl
        """
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
70 model_short_name = self.model.config.name_or_path.split('/')
            [-1]
        return f"{prefix}_{model_short_name}_{timestamp}.pkl"

```

```

def _validate_params(self, max_length: int) -> int:
    """
75  Validates and adjusts the max_length parameter.
    Returns adjusted max_length.
    """

    if max_length > self.max_model_length:
        print(f"\nWarning: Requested max_length ({max_length})
            exceeds model's maximum ")
80  f"({self.max_model_length}). Setting to {self.
        max_model_length}.")
    return self.max_model_length
    return max_length

def mean_pooling(self, model_output, attention_mask):
85  """
    Perform mean pooling on the token embeddings using attention
        mask.
    """

    token_embeddings = model_output.last_hidden_state
    input_mask_expanded = attention_mask.unsqueeze(-1).expand(
        token_embeddings.size()).float()
90  return torch.sum(token_embeddings * input_mask_expanded, 1)
        / torch.clamp(input_mask_expanded.sum(1), min=1e-9)

def encode_batch(self, texts: List[str], batch_size: int =
    16, max_length: int = 512,
    save_path: str = "embeddings") -> np.ndarray:
    """
95  Кодирование батча текстов в эмбединги.
    Args:
        texts: Список текстов для кодирования.
        batch_size: Размер батча.
        max_length: Максимальная длина последовательности.
100  save_path: Путь для сохранения эмбедингов.
    Returns:
        Массив эмбедингов.
    """

    self.model.eval()
105  all_embeddings = []

    # Validate and adjust max_length
    max_length = self._validate_params(max_length)

110  # Show tokenization statistics

```

```

sample_tokens = [len(self.tokenizer.encode(text)) for text
                  in texts[:100]]
avg_length = sum(sample_tokens) / len(sample_tokens)
max_tokens = max(sample_tokens)
print(f"\nTokenization statistics (from first 100 texts):")
115 print(f"Average length: {avg_length:.1f} tokens")
print(f"Maximum length: {max_tokens} tokens")
print(f"Texts will be truncated to {max_length} tokens if
      longer\n")

# Process texts in batches
120 num_samples = len(texts)
embedding_dim = None # Will be set after first batch

for i in tqdm(range(0, num_samples, batch_size), desc="
               Encoding texts", unit="batch"):
    # Get current batch
125 batch_end = min(i + batch_size, num_samples)
    batch_texts = texts[i:batch_end]

    # Tokenize batch
    encoded = self.tokenizer(
130 batch_texts,
    padding=True,
    truncation=True,
    max_length=max_length,
    return_tensors='pt',
135 return_attention_mask=True
    ).to(self.device)

    with torch.no_grad():
        outputs = self.model(**encoded)
140 # Use mean pooling for sentence embeddings
        batch_embeddings = self.mean_pooling(outputs, encoded['
            attention_mask']).cpu().numpy()

    # Verify embedding dimension
    if batch_embeddings.shape[1] != self.embedding_dim:
145 raise ValueError(
    f"Unexpected embedding dimension in batch {i//batch_size +
      1}:\n"
    f"Expected: {self.embedding_dim}, Got: {batch_embeddings.
      shape[1]}"
    )

```

```

150 all_embeddings.append(batch_embeddings)

    try:
        embeddings = np.vstack(all_embeddings)
        print(f"\nFinal embedding shape: {embeddings.shape} "
155 f"({embeddings.shape[0]} samples, {embeddings.shape[1]}
            dimensions)")
    except ValueError as e:
        print("\nError while combining embeddings:")
        print(f"Number of batches: {len(all_embeddings)}")
        print(f"Embedding shapes: {[emb.shape for emb in
            all_embeddings]}")
160 raise e

    if save_path:
        os.makedirs(save_path, exist_ok=True)
        filename = self._generate_filename("batch")
165 filepath = os.path.join(save_path, filename)
        with open(filepath, 'wb') as f:
            pickle.dump(embeddings, f)
            print(f"Embeddings saved to {filepath}")

170 return embeddings

def encode_dataset(self, dataset: List[Dict[str, str]],
    text_key: str = "dialogue",
    batch_size: int = 16, max_length: int = 512,
    save_path: str = "embeddings") -> np.ndarray:
175 """
    Кодирование датасета в эмбединги.
    Args:
        dataset: Датасет в виде списка словарей.
        text_key: Ключ для доступа к тексту в словаре.
180 batch_size: Размер батча.
        max_length: Максимальная длина последовательности.
        save_path: Путь для сохранения эмбедингов.
    Returns:
        Массив эмбедингов.
185 """
    texts = [item[text_key] for item in dataset]
    embeddings = self.encode_batch(texts, batch_size=batch_size,
        max_length=max_length, save_path=None)

    if save_path:
190 os.makedirs(save_path, exist_ok=True)

```

```

filename = self._generate_filename("dataset")
filepath = os.path.join(save_path, filename)
with open(filepath, 'wb') as f:
    pickle.dump(embeddings, f)
195 print(f"Embeddings saved to {filepath}")

return embeddings

```

Листинг 1.3

data_preprocessing.py

```

import re
from typing import List, Dict, Union, Optional
from datasets import Dataset
5 from transformers import PreTrainedTokenizer

class DialoguePreprocessor:
    """Класс для предобработки диалогов"""

    10 def __init__(self, tokenizer: Optional[PreTrainedTokenizer]
        = None, max_length: int = 512):
        """
        Инициализация preprocessor
        Args:
            tokenizer: Токенизатор из transformers
            max_length: Максимальная длина последовательности
        15 """

        self.tokenizer = tokenizer
        self.max_length = max_length

    20 @staticmethod
    def basic_clean_dialogue(text: str) -> str:
        """
        Базовая очистка текста диалога
        Args:
            text: Исходный текст диалога
        25 Returns:
            Очищенный текст
        """

        # Упрощение меток вида "#Имя#:"
        30 text = re.sub(r"#([A-Za-zА-Яа-яёЁ]+)#:", r"\1:", text)
        # Убираем лишние пробелы
        text = text.strip()
        return text

```



```

35 def preprocess_dialogue(self, dialogue: str) -> str:
    """
    Предобработка одного диалога
    Args:
    dialogue: Текст диалога
40 Returns:
    Предобработанный текст
    """
    return self.basic_clean_dialogue(dialogue)

45 def preprocess_batch(self, examples: Dict) -> Dict:
    """
    Предобработка батча диалогов
    Args:
    examples: Словарь с полем dialogue
50 Returns:
    Словарь с полем input_text
    """
    dialogues = examples["dialogue"]
    cleaned = [self.preprocess_dialogue(d) for d in dialogues]
55 return {"input_text": cleaned}

def tokenize_batch(self, examples: Dict) -> Dict:
    """
    Токенизация батча текстов
60 Args:
    examples: Словарь с полем input_text
    Returns:
    Словарь с токенами
    """

65 if self.tokenizer is None:
    raise ValueError("Tokenizer is not initialized")

    return self.tokenizer(
        examples["input_text"],
70 truncation=True,
        padding="max_length",
        max_length=self.max_length
    )

75 def process_dataset(self,
    dataset: Dataset,
    num_proc: int = 4,
    batch_size: int = 32,
    tokenize: bool = True) -> Dataset:

```

```

80  """
    Полная обработка датасета
    Args:
    dataset: Исходный датасет
    num_proc: Количество процессов для параллельной обработки
85  batch_size: Размер батча
    tokenize: Нужно ли токенизировать тексты
    Returns:
    Обработанный датасет
    """
90  # Предобработка текстов
    processed = dataset.map(
        self.preprocess_batch,
        batched=True,
        num_proc=num_proc,
95  batch_size=batch_size
    )

    if tokenize:
    if self.tokenizer is None:
100  raise ValueError("Tokenizer is required for tokenization")

    # Токенизация
    processed = processed.map(
        self.tokenize_batch,
105  batched=True,
        num_proc=num_proc,
        batch_size=batch_size
    )

110  return processed

```

Листинг 1.4

clustering.py

```

import numpy as np
from sklearn.cluster import KMeans, DBSCAN,
    AgglomerativeClustering
from sklearn.metrics import (
5  silhouette_score, davies_bouldin_score,
    calinski_harabasz_score,
    adjusted_rand_score, adjusted_mutual_info_score
)
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

```

```

10 from hdbscan import HDBSCAN
   from umap import UMAP
   import matplotlib.pyplot as plt
   import plotly.express as px
   import plotly.graph_objects as go
15 from scipy.spatial.distance import cdist
   from kneed import KneeLocator
   import logging
   from typing import Dict, List, Tuple, Optional, Any, Union
   import warnings
20 from dataclasses import dataclass
   from sklearn.model_selection import train_test_split

   # Настройка логирования
   logging.basicConfig(level=logging.INFO)
25 logger = logging.getLogger(__name__)

   import warnings
   warnings.simplefilter(action='ignore', category=
       FutureWarning)

30 @dataclass
   class ClusteringResult:
       """Класс для хранения результатов кластеризации"""
       labels: np.ndarray
       metrics: Dict[str, float]
35 method: str
       params: Dict[str, Any]

   class DialogClustering:
       def __init__(self, embeddings: np.ndarray, original_data:
           Optional[List] = None, use_scaling: bool = True):
40 """
       Инициализация кластеризации с эмбедингами диалогов

       Args:
       embeddings: Массив эмбедингов диалогов
45 original_data: Исходные тексты диалогов для визуализации
       use_scaling: Применять ли стандартное масштабирование к эмбе
           ддингам.
       По умолчанию True. Установите False, если эмбединги уже нор
           мализованы
       или используется метрика, не требующая масштабирования (напр
           имер, косинусная близость).
       """

```

```

50 self._validate_input(embeddings, original_data)
self.use_scaling = use_scaling
self.embeddings = self._preprocess_embeddings(embeddings)
self.original_data = original_data
self.best_result: Optional[ClusteringResult] = None
55 logger.info(f"Инициализирован DialogClustering с {len(
    embeddings)} эмбедингами, use_scaling={use_scaling}")

def _validate_input(self, embeddings: np.ndarray,
    original_data: Optional[List]) -> None:
    """Валидация входных данных"""
    if embeddings is None or len(embeddings) == 0:
60 raise ValueError("Embeddings не могут быть пустыми")
    if not isinstance(embeddings, np.ndarray):
        raise ValueError("Embeddings должны быть numpy массивом")
    if original_data is not None and len(original_data) != len(
        embeddings):
        raise ValueError("Размер original_data должен совпадать с ра
            змером embeddings")
65

def _preprocess_embeddings(self, embeddings: np.ndarray) ->
    np.ndarray:
    """
    Предобработка эмбедингов

70 Args:
    embeddings: Входные эмбединги

    Returns:
    np.ndarray: Обработанные эмбединги

75 Note:
    По умолчанию применяется StandardScaler. Если ваши эмбединг
        и уже нормализованы
    или вы используете метрики, не требующие масштабирования (на
        пример, косинусная близость),
    установите параметр use_scaling=False в конструкторе.
80 """
    if not hasattr(self, 'use_scaling') or self.use_scaling:
        scaler = StandardScaler()
        with warnings.catch_warnings():
            warnings.filterwarnings("ignore", category=FutureWarning)
85 return scaler.fit_transform(embeddings)
    return embeddings

```

```

def _evaluate_clustering(self, labels: np.ndarray, data: np.
    ndarray) -> Dict[str, float]:
    """
90  Комплексная оценка качества кластеризации

    Args:
    labels: Метки кластеров
    data: Данные для оценки
95

    Returns:
    Dict[str, float]: Словарь метрик качества
    """

    if len(set(labels)) < 2:
100  return {"error": -1.0}

    # Учитываем метку -1 (шум) как отдельный кластер для вычисле
    ния метрик
    if -1 in labels:
        logger.info("Обнаружены точки шума (label = -1), они будут у
            чтены как отдельный кластер")
105

    metrics = {}
    try:
        metrics['silhouette'] = silhouette_score(data, labels)
        metrics['davies_bouldin'] = davies_bouldin_score(data,
            labels)
110  metrics['calinski_harabasz'] = calinski_harabasz_score(data,
            labels)

    # Оценка стабильности кластеризации
    # Для оценки стабильности используем тот же метод кластериза
    ции, что и основной
    if len(set(labels)) > 1:
115  # Разделяем данные на две части для оценки стабильности
        n_samples = len(data)
        indices = np.random.permutation(n_samples)
        n_subset = n_samples // 2 # Используем половину данных для
            каждой части

120  subset1_idx = indices[:n_subset]
        subset2_idx = indices[n_subset:2*n_subset] # Берем равные ч
            асти

        X_subset1 = data[subset1_idx]
        X_subset2 = data[subset2_idx]

```

```

125     # Обучаем модель на первой части
model = KMeans(n_clusters=len(set(labels)), random_state=42)
model.fit(X_subset1)

130     # Получаем предсказания для обеих частей
labels1 = model.predict(X_subset1)
labels2 = model.predict(X_subset2)

    # Оцениваем стабильность на равных по размеру частях
135     metrics['stability'] = adjusted_mutual_info_score(labels1,
        labels2)
    else:
        metrics['stability'] = 0.0
    except Exception as e:
        logger.warning(f"Ошибка при вычислении метрик: {str(e)}")

140     return metrics

def _estimate_optimal_clusters(self, data: np.ndarray) ->
    int:
    """
145     Оценка оптимального количества кластеров

    Args:
        data: Данные для кластеризации

    Returns:
        int: Оптимальное количество кластеров
    """
    # Метод локтя
    # Максимальное количество кластеров зависит от размера данн
    x
155     max_clusters = min(
        max(15, int(np.log2(len(data)))), # Минимум 15, максимум
            log2(N)
        len(data) // 2 # Не более половины объектов
    )
    logger.info(f"Оценка оптимального количества кластеров:
        max_clusters={max_clusters}")
160     distortions = []
    K = range(2, max_clusters + 1)

    for k in K:
        kmeans = KMeans(n_clusters=k, random_state=42)

```

```

165 kmeans.fit(data)
    distortions.append(kmeans.inertia_)

    # Используем KneeLocator для автоматического определения точ
    ки перегиба
    kl = KneeLocator(
170 list(K), distortions, curve='convex', direction='decreasing'
    )
    optimal_k = kl.elbow if kl.elbow else 3

    # Проверка через силуэтный анализ
175 silhouette_scores = []
    # Проверяем силуэт в диапазоне от 2 до max_clusters
    for k in range(2, max_clusters + 1):
        kmeans = KMeans(n_clusters=k, random_state=42)
        labels = kmeans.fit_predict(data)
180 try:
            score = silhouette_score(data, labels)
            silhouette_scores.append(score)
        except:
            continue

185 if silhouette_scores:
        silhouette_k = np.argmax(silhouette_scores) + 2
        # Берем среднее между методами
        optimal_k = int((optimal_k + silhouette_k) / 2)
190
    return optimal_k

def _optimize_dbscan_params(self, data: np.ndarray) -> Tuple
    [float, int]:
    """
195 Оптимизация параметров DBSCAN

    Returns:
    Tuple[float, int]: Оптимальные eps и min_samples

    Note:
    Вычисление cdist(data, data) имеет сложность  $O(N^2)$  и может
        быть
    очень ресурсоемким для больших наборов данных. Для  $N >$ 
        10,000
    рекомендуется использовать более эффективные методы.
    """
205 if len(data) > 10000:

```



```

logger.warning("Вычисление cdist для больших данных может бы
    ть медленным. ")
"Рекомендуется использовать подвыборку или более эффективные
    методы.")
# Автоматический расчет eps на основе распределения расстоян
    ий
distances = cdist(data, data)
210 eps_candidates = np.percentile(distances, [5, 10, 15, 20,
    25])
min_samples_candidates = range(3, 10)

best_score = -1
best_params = (0.5, 5) # Значения по умолчанию
215
for eps in eps_candidates:
    for min_samples in min_samples_candidates:
        try:
            dbSCAN(eps=eps, min_samples=min_samples)
220 labels = dbSCAN.fit_predict(data)
            if len(set(labels)) > 1: # Проверяем, что есть больше одног
                о кластера
            score = silhouette_score(data, labels)
            if score > best_score:
                best_score = score
225 best_params = (eps, min_samples)
        except:
            continue

    return best_params
230

def _optimize_hdbSCAN_params(self, data: np.ndarray) ->
    Tuple[int, int]:
    """
    Оптимизация параметров HDBSCAN

235 Returns:
    Tuple[int, int]: Оптимальные min_cluster_size и min_samples

    Note:
    Оптимизация параметров HDBSCAN может быть вычислительно затр
        атной,
240 особенно для больших наборов данных (N > 10,000). Рекомендуюе
        тся
    использовать подвыборку или ограничить диапазон параметров.
    """

```

```

if len(data) > 10000:
    logger.warning("Оптимизация HDBSCAN для больших данных может
        быть медленной. ")
245 "Рекомендуется использовать подвыборку или ограничить диапа-
        зон параметров.")
    min_cluster_sizes = range(5, 20, 2)
    min_samples_multipliers = [1, 2, 3] # Множители для
        min_samples

    best_score = -1
250 best_params = (5, 5) # Значения по умолчанию

    for min_cluster_size in min_cluster_sizes:
        for multiplier in min_samples_multipliers:
            min_samples = min_cluster_size * multiplier
255 try:
                clusterer = HDBSCAN(
                    min_cluster_size=min_cluster_size,
                    min_samples=min_samples,
                    cluster_selection_epsilon=0.0
260 )
                labels = clusterer.fit_predict(data)
                if len(set(labels)) > 1:
                    score = silhouette_score(data, labels)
                    if score > best_score:
265 best_score = score
                        best_params = (min_cluster_size, min_samples)
                    except:
                        continue

270 return best_params

def find_best_clustering(self, allowed_methods: Optional[
    Tuple[str]] = None) -> ClusteringResult:
    """
    Поиск наилучшего метода кластеризации и его параметров
275

    Args:
        allowed_methods: Кортеж допустимых методов кластеризации для
            перебора.
        Если None, используются все доступные методы.
        Доступные методы: 'kmeans', 'agglomerative', '
            gaussian_mixture',
280 'dbscan', 'hdbscan'

```

Returns:

ClusteringResult: Результат лучшей кластеризации

285 Note:

Используется комбинированная метрика с весами:

- silhouette_score: 0.4 (чем больше, тем лучше)
- davies_bouldin_score: 0.3 (чем меньше, тем лучше, поэтому используется 1 - db)
- calinski_harabasz_score: 0.1 (чем больше, тем лучше)
- 290 - stability: 0.2 (чем больше, тем лучше)

Эти веса выбраны эмпирически для баланса между компактностью кластеров,

их разделимостью и стабильностью. При необходимости можно настроить веса

в зависимости от конкретной задачи.

295 """

Все доступные методы кластеризации

```
all_methods = {
    'kmeans': lambda: KMeans(n_clusters=n_clusters,
                             random_state=42),
    'agglomerative': lambda: AgglomerativeClustering(
        n_clusters=n_clusters),
    300 'gaussian_mixture': lambda: GaussianMixture(n_components=
        n_clusters, random_state=42),
    'dbscan': lambda: DBSCAN(
        eps=self._optimize_dbscan_params(self.embeddings)[0],
        min_samples=self._optimize_dbscan_params(self.embeddings)
        [1]
    ),
    305 'hdbscan': lambda: HDBSCAN(
        *self._optimize_hdbscan_params(self.embeddings)
    )
}
```

310 *# Если не указаны допустимые методы, используем все*

if allowed_methods is None:

allowed_methods = tuple(all_methods.keys())

else:

Проверяем, что все указанные методы существуют

315 invalid_methods = set(allowed_methods) - set(all_methods.
 keys())

if invalid_methods:

raise ValueError(f"Недопустимые методы кластеризации: {
 invalid_methods}")

```

best_result = None
best_score = -float('inf')
320
    # Определяем оптимальное количество кластеров
    n_clusters = self._estimate_optimal_clusters(self.embeddings
        )
    logger.info(f"Оптимальное количество кластеров: {n_clusters}
        ")

325    # Создаем список методов для перебора на основе
        allowed_methods
    clustering_methods = [
        (method_name, all_methods[method_name])
        for method_name in allowed_methods
    ]

330
    for method_name, get_clusterer in clustering_methods:
        try:
            logger.info(f"Пробуем метод: {method_name}")
            clusterer = get_clusterer()
335            labels = clusterer.fit_predict(self.embeddings)

            # Оцениваем качество
            metrics = self._evaluate_clustering(labels, self.embeddings)
            if 'error' in metrics:
340                continue

            # Используем комбинированную метрику для оценки
            combined_score = (
                metrics.get('silhouette', 0) * 0.4 +
345                (1 - metrics.get('davies_bouldin', 1)) * 0.3 +
                metrics.get('calinski_harabasz', 0) * 0.1 +
                metrics.get('stability', 0) * 0.2
            )

350            if combined_score > best_score:
                best_score = combined_score
                params = {}
                if hasattr(clusterer, 'get_params'):
                    params = clusterer.get_params()
355            best_result = ClusteringResult(
                labels=labels,
                metrics=metrics,
                method=method_name,
                params=params
            )

```

```

360 )

    except Exception as e:
        logger.warning(f"Ошибка при использовании {method_name}: {
            str(e)}")
        continue

365
    if best_result is None:
        raise ValueError("Не удалось найти подходящий метод кластери-
            зации")

    self.best_result = best_result
370 logger.info(f"Лучший метод: {best_result.method} с метриками
        : {best_result.metrics}")
    return best_result

    def visualize_clusters(self, labels: Optional[np.ndarray] =
        None,
        n_components: int = 2,
375 interactive: bool = True) -> np.ndarray:
    """
    Визуализация кластеров с использованием UMAP

    Args:
    380 labels: Метки кластеров
        n_components: Количество компонент для визуализации
        interactive: Использовать ли интерактивную визуализацию

    Returns:
    385 np.ndarray: Массив с координатами точек в пространстве UMAP
    """
    if labels is None and self.best_result is not None:
        labels = self.best_result.labels

    390 try:
        reducer = UMAP(n_components=n_components, random_state=42)
        embedding_nd = reducer.fit_transform(self.embeddings)

        if interactive:
    395 hover_data = {}
            if self.original_data is not None:
                hover_data['text'] = self.original_data

            if n_components == 2:
    400 fig = px.scatter(

```

```

x=embedding_nd[:, 0],
y=embedding_nd[:, 1],
color=labels if labels is not None else None,
hover_data=hover_data,
405 title=f"Визуализация кластеров. Метод: {self.best_result.
      method}"
)
else:
fig = px.scatter_3d(
x=embedding_nd[:, 0],
410 y=embedding_nd[:, 1],
z=embedding_nd[:, 2],
color=labels if labels is not None else None,
hover_data=hover_data,
title=f" Визуализация кластеров. Метод: {self.best_result.
      method}"
415 )

fig.update_traces(
marker=dict(size=8),
hovertemplate="<br>".join([
420 "Кластер: %{color}",
"X: %{x}",
"Y: %{y}",
"Текст: %{customdata[0]}" if self.original_data is not None
      else "",
])
425 )
fig.show()
else:
plt.figure(figsize=(12, 8))
scatter = plt.scatter(
430 embedding_nd[:, 0],
embedding_nd[:, 1],
c=labels if labels is not None else None,
cmap='viridis'
)
435 if labels is not None:
plt.colorbar(scatter, label='Кластер')
plt.title("Визуализация кластеров")
plt.xlabel("УМАР компонента 1")
plt.ylabel("УМАР компонента 2")
440 plt.show()

return embedding_nd

```

```

except Exception as e:
445 logger.error(f"Ошибка в visualize_clusters: {str(e)}")
raise

def get_cluster_representatives(self, n_representatives: int
    = 3) -> Dict[int, List[int]]:
    """
450 Получение репрезентативных примеров для каждого кластера

    Args:
    n_representatives: Количество представителей от каждого клас
        тера

455 Returns:
    Dict[int, List[int]]: Словарь с индексами представителей для
        каждого кластера
    """
    if self.best_result is None:
        raise ValueError("Сначала необходимо выполнить кластеризацию
            ")

460 representatives = {}
    unique_labels = np.unique(self.best_result.labels)

    for label in unique_labels:
465 if label == -1: # Пропускаем выбросы
        continue

    # Получаем индексы точек в текущем кластере
    cluster_indices = np.where(self.best_result.labels == label)
        [0]
470 cluster_points = self.embeddings[cluster_indices]

    # Находим центроид кластера
    centroid = np.mean(cluster_points, axis=0)

475 # Вычисляем расстояния до центроида
    distances = cdist([centroid], cluster_points)[0]

    # Выбираем точки:
    # 1. Ближайшую к центроиду (типичный представитель)
480 # 2. Несколько разнообразных представителей
    central_idx = cluster_indices[np.argmin(distances)]

```



```

# Используем максимальное минимальное расстояние для разнообра
# зия
485 selected = [central_idx]
remaining_indices = list(set(cluster_indices) - {central_idx
    })

while len(selected) < n_representatives and
    remaining_indices:
    # Находим точку, максимально удаленную от уже выбранных
    max_min_dist = -1
490 best_idx = None

    for idx in remaining_indices:
        point = self.embeddings[idx].reshape(1, -1)
        min_dist = min(cdist(point, self.embeddings[selected])[0])
495
        if min_dist > max_min_dist:
            max_min_dist = min_dist
            best_idx = idx

500 if best_idx is not None:
    selected.append(best_idx)
    remaining_indices.remove(best_idx)
    else:
        break

505
    representatives[label] = selected

    return representatives

510 def visualize_cluster_distribution(self) -> None:
    """Визуализация распределения размеров кластеров"""
    if self.best_result is None:
        raise ValueError("Сначала необходимо выполнить кластеризацию
            ")

515 try:
    unique_labels = np.unique(self.best_result.labels)
    counts = [np.sum(self.best_result.labels == label) for label
        in unique_labels]

    fig = go.Figure(data=[
520 go.Bar(
    x=unique_labels,
    y=counts,

```

```

525     text=counts,
        textposition='auto',
    )
    ])

    fig.update_layout(
        title=f"Распределение размеров кластеров (метод: {self.
            best_result.method})",
530     xaxis_title="Номер кластера",
        yaxis_title="Количество объектов",
        showlegend=False
    )

535     fig.show()

    except Exception as e:
        logger.error(f"Ошибка в visualize_cluster_distribution: {str
            (e)}")
        raise

```

Листинг 1.5

database.py

```

# database.py (обновлённая версия)

from pymilvus import connections, FieldSchema,
    CollectionSchema, DataType, Collection, MilvusClient
5
class VectorDatabase:
    """
    Класс, инкапсулирующий логику взаимодействия с Milvus
    для хранения (диалог, эмбеddинг, суммаризация) и поиска реле
    вантных записей.
    10    """

    def __init__(
        self,
        uri: str,
        15     token: str,
        collection_name: str,
        embedding_dim: int,
        metric_type: str = "IP" # Inner Product, как одна из популя
            рных (косинус при нормированных векторах)
    ):
        20    """

```

```

Параметры:
- uri: URL для подключения к Milvus (например, "https://
  example.com")
- token: Токен для аутентификации
- collection_name: имя коллекции, где храним наши записи
25 - embedding_dim: размерность эмбедингов (например, 768)
- metric_type: тип метрики для поиска ("IP", "L2", "COSINE",
  "HAMMING" и т.п.)
"""
self.uri = uri
self.token = token
30 self.collection_name = collection_name
self.embedding_dim = embedding_dim
self.metric_type = metric_type

# Создаём соединение
35 connections.connect(
    alias="default",
    uri=self.uri,
    token=self.token
)

40
# Проверяем, что соединение установлено
if not connections.has_connection("default"):
    raise ConnectionError("Не удалось установить соединение с
      Milvus")

45
# Создаём клиент Milvus
self.client = MilvusClient(uri=self.uri, token=self.token)

# Подготавливаем/создаём схему коллекции
self._create_collection_if_not_exists()

50
def _create_collection_if_not_exists(self):
    """
    Проверяет существование коллекции Milvus, при отсутствии --
      создаёт её
    с необходимыми полями и индексом.
55 """
    # Проверяем, существует ли коллекция
    has_collection = self.client.has_collection(self.
      collection_name)
    if not has_collection:
        # Описываем поля
60 dialog_text_field = FieldSchema(

```

```

name="dialog_text",
dtype=DataType.VARCHAR,
max_length=12096,
is_primary=False,
65 description="Исходный текст диалога"
)

summary_field = FieldSchema(
name="summary",
70 dtype=DataType.VARCHAR,
max_length=2048,
is_primary=False,
description="Сгенерированная суммаризация"
)

75 # Поле embedding -- вектор
embedding_field = FieldSchema(
name="embedding",
dtype=DataType.FLOAT_VECTOR,
80 is_primary=False,
dim=self.embedding_dim
)

# Можно добавить "id" (auto id) как primary key
85 # Milvus v2 позволяет auto_id=True
auto_id_field = FieldSchema(
name="id",
dtype=DataType.INT64,
is_primary=True,
90 auto_id=True,
description="Уникальный идентификатор"
)

schema = CollectionSchema(
95 fields=[auto_id_field, dialog_text_field, summary_field,
embedding_field],
description="Коллекция для хранения диалогов, их эмбедингов
и суммаризаций"
)

# Создаём коллекцию
100 collection = Collection(name=self.collection_name, schema=
schema)

# Создаём индекс по embedding

```

```

# metric_type может быть "L2", "IP", "COSINE" и т.д.
index_params = {
105     "index_type": "IVF_FLAT",
    "metric_type": self.metric_type,
    "params": {"nlist": 128}
}
collection.create_index(
110 field_name="embedding",
    index_params=index_params
)

# При необходимости можно load-коллекцию в память
115 collection.load()
else:
    # Если коллекция существует, можно просто её загрузить:
    collection = Collection(name=self.collection_name)
    collection.load()

120
def upsert_dialog(
    self,
    dialog_text: str,
    embedding: list[float],
125 summary: str
) -> None:
    """
    Добавляет запись в Milvus, содержащую:
    - dialog_text: исходный текст диалога
130 - embedding: эмбединг (list[float], длина self.
        embedding_dim)
    - summary: итоговая выжимка
    """
    # Подготовим сущность для вставки
    entities = {
135     "dialog_text": [dialog_text],
    "embedding": [embedding],
    "summary": [summary]
    }

140
# Вставляем в коллекцию
collection = Collection(name=self.collection_name)
collection.insert([entities[field.name] for field in
    collection.schema.fields if field.name != "id"])

# Опционально: вызов flush
145 collection.flush()

```

```

def bulk_upsert_dialogs(
self,
dialog_texts: list[str],
150 embeddings: list[list[float]],
summaries: list[str],
batch_size: int = 100
) -> None:
    """
155 Массовая вставка данных в Milvus с использованием батчей.
    Args:
        dialog_texts: список текстов диалогов
        embeddings: список эмбедингов
        summaries: список суммаризаций
160 batch_size: размер пакета для вставки
    """

    assert len(dialog_texts) == len(embeddings) == len(summaries
        ), "All input lists must have same length"

    total_records = len(dialog_texts)
165 total_rt = 0  # total response time for insert

    print(f"Inserting {total_records} records into collection: {
        self.collection_name}")

    # Process in batches
170 for i in range(0, total_records, batch_size):
        batch_end = min(i + batch_size, total_records)

        # Format data as expected by Milvus
        rows = [
175 {
            "dialog_text": str(dialog_texts[j]),
            "embedding": embeddings[j],
            "summary": str(summaries[j])
        }
180 for j in range(i, batch_end)
    ]

    # Insert batch and measure time
    self.client.insert(
185 collection_name=self.collection_name,
        data=rows
    )

```

```

190 print(f"Total insert time: {round(total_rt, 4)} seconds")

    # Flush and measure time
    print("Flushing collection...")

195 def search_engine(
    self,
    query_embedding: list[float],
    top_k: int = 5,
    search_metric: str = None
    ) -> list[dict]:
200     """
    Выполняет поиск ближайших векторов в Milvus по полю "
        embedding".
    Можно указать search_metric, чтобы переопределить метрику (и
        наче берётся self.metric_type).
    Возвращает список словарей:
205     [
    {
        "id": 12345,
        "dialog_text": "...",
        "summary": "...",
        "distance": 0.85
210     },
    ...
    ]
    """

    if search_metric is None:
215     metric = self.metric_type
    else:
        metric = search_metric

    search_params = {
220         "metric_type": metric,
        "params": {"nprobe": 10}
    }

    if top_k == 0:
225     return []

    collection = Collection(name=self.collection_name)
    results = collection.search(
230     data=[query_embedding],
    anns_field="embedding",
    param=search_params,

```



```

limit=top_k + 1,  # увеличиваем limit на 1, так как первый э
                    элемент отбросим
output_fields=["dialog_text", "summary"]
)
235
if len(results) == 0:
    return []

hits = results[0][1:]  # пропускаем первый элемент
240 output = []
for hit in hits:
    row = {
        "id": hit.id,
        "dialog_text": hit.entity.get("dialog_text"),
245        "summary": hit.entity.get("summary"),
        "distance": hit.score
    }
    output.append(row)

250 return output

```

Листинг 1.6

llm_interaction.py

```

import os
import openai
import requests
5 import pandas as pd
from settings import OPENROUTER_API_KEY, DEFAULT_MODEL,
    TASK_DESCRIPTION, EMPTY_TASK_DESCRIPTION, INPUT_PREFIX,
    OUTPUT_PREFIX

def build_icl_prompt(examples, new_dialogue, max_examples=5,
10 preprocess_func=None):
    """
    Формирует оптимальный промпт для моделей типа Instruct (напр
        имер, OpenAI text-davinci, Mistral).

    Параметры:
    -----
15 examples : List[Dict[str, str]]
    Список демонстрационных примеров:
    {
        'dialog': '...', # текст диалога

```

```

    'summary': '...' # итоговая суммаризация
20 }
    new_dialogue : str
    Новый диалог, для которого нужно сгенерировать суммаризацию.
    max_examples : int
    Максимальное количество демонстрационных примеров.
25 preprocess_func : Callable[[str], str], optional
    Функция предобработки текста (если не указана, используется
        'strip').

    Возвращает:
    -----
30 str
    Оптимизированный промпт для Instruct-моделей.
    """

    if not isinstance(examples, list):
        raise ValueError("'examples' должен быть списком словарей.")
35 if not isinstance(new_dialogue, str):
        raise ValueError("'new_dialogue' должен быть строкой.")
    if preprocess_func is None:
        preprocess_func = lambda x: x.strip()

40 # Обрезаем примеры до max_examples
    used_examples = examples[:max_examples]

    # Если список примеров пуст, используем другой
        task_description
    if not used_examples:
45 new_task_text = f"{INPUT_PREFIX}{preprocess_func(
        new_dialogue)}\n{OUTPUT_PREFIX}"
    return EMPTY_TASK_DESCRIPTION + new_task_text

    # Создание текстов примеров
    examples_text = []
50 for i, ex in enumerate(used_examples, 1):
    dialog = preprocess_func(ex.get('dialog', ''))
    summary = preprocess_func(ex.get('summary', ''))

    if not dialog or not summary:
55 continue # Игнорируем пустые примеры

    examples_text.append(f"### Example {i} ###\n{INPUT_PREFIX}{
        dialog}\n{OUTPUT_PREFIX}{summary}\n")

    # Обработка нового диалога

```

```

60 new_dialogue_processed = preprocess_func(new_dialogue)
new_task_text = f"{INPUT_PREFIX}{new_dialogue_processed}\n{
    OUTPUT_PREFIX}"

# Формируем финальный промпт
prompt = TASK_DESCRIPTION + "\n\n".join(examples_text) + "\n
    \n" + "### Task ###\n" + new_task_text
65 return prompt

def call_openrouter_llm(
    prompt: str,
    model: str = "mistralai/mistral-7b-instruct",
70 openrouter_api_key: str = None,
    referer: str = "https://my.site.example",
    site_name: str = "MySiteOnOpenRouter",
    temperature: float = 0.3
) -> str:
75 """
    Запрашивает ответ у модели, размещённой на OpenRouter, испол
        зую библиотеку 'openai'.
    """

# 1. Если ключ явно не указан, берем из переменной окружения
80 api_key = openrouter_api_key or os.environ.get("
    OPENROUTER_API_KEY")
    if not api_key:
        raise ValueError("OpenRouter API key не найден. Укажите
            openrouter_api_key или установите OPENROUTER_API_KEY в
            env.")

# 2. Устанавливаем API ключ и базовый URL для OpenRouter
85 openai.api_key = api_key
    openai.api_base = "https://openrouter.ai/api/v1"

# 3. Формируем запрос на chat.completions
    headers = {
90     "HTTP-Referer": referer,
        "X-Title": site_name
    }

    completion = openai.ChatCompletion.create(
95     model=model,
    messages=[
        {
            "role": "user",

```

```

100     "content": prompt
    }
    ],
    temperature=temperature,
    headers=headers
    )

105     # 4. Извлекаем контент ответа
    return completion['choices'][0]['message']['content']

110 def get_available_models(api_key: str = None):
    """
    Fetch the list of available models from OpenRouter API.

    Args:
    api_key (str): Your OpenRouter API key.

115    Returns:
    list: A list of available models or an error message if the
        request fails.
    """
    api_key = api_key or OPENROUTER_API_KEY
120    if not api_key:
        raise ValueError("OpenRouter API key не найден. Укажите
            api_key или установите OPENROUTER_API_KEY в env.")

    url = "https://openrouter.ai/api/v1/models"
    headers = {
125         "Authorization": f"Bearer {api_key}",
         "Content-Type": "application/json"
    }

    try:
130         response = requests.get(url, headers=headers)
        response.raise_for_status() # Raise an error for HTTP error
            responses
        models = response.json() # Parse the JSON response
        return models
    except requests.exceptions.RequestException as e:
135     return {"error": str(e)}

    def load_file(file_path: str) -> pd.DataFrame:
        """
140     Load embeddings from a file.

```

```

Args:
file_path: Path to the file containing embeddings.
Returns:
DataFrame with embeddings and associated data.
"""
145 if file_path.endswith('.pkl'):
    return pd.read_pickle(file_path)
elif file_path.endswith('.csv'):
150 return pd.read_csv(file_path)
elif file_path.endswith('.parquet'):
    return pd.read_parquet(file_path)
else:
    raise ValueError(f"Unsupported file format: {file_path}")

```

Листинг 1.7

evaluation.py

```

import torch
import pandas as pd
from tqdm import tqdm
5 from rouge_score import rouge_scorer
import bert_score
from nltk.translate.bleu_score import sentence_bleu
from nltk.tokenize import word_tokenize
import logging
10 import warnings

# Отключение предупреждений о весах модели
logging.getLogger("transformers.modeling_utils").setLevel(
    logging.ERROR)
15 warnings.filterwarnings("ignore", category=UserWarning,
    module="bert_score")

def compute_rouge(references: list[str], hypotheses: list[
    str]) -> pd.DataFrame:
    """
20 Compute ROUGE metrics for pairs of reference and hypothesis
    texts.

    Args:
    references: List of reference texts to compare against
    hypotheses: List of hypothesis texts to evaluate
25

```

```

Returns:
pandas.DataFrame with columns for ROUGE-1, ROUGE-2, and
    ROUGE-L F-measure scores
"""
if len(references) != len(hypotheses):
30 raise ValueError("Длины списков 'references' и 'hypotheses'
    должны совпадать.")
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', '
    rougeL'], use_stemmer=True)
results = []
print("Вычисляем ROUGE для суммаризаций...")
for reference, hypothesis in tqdm(zip(references, hypotheses
    ), total=len(references), desc="ROUGE", unit="pair"):
35 scores = scorer.score(reference, hypothesis)
results.append({
    'rouge1': scores['rouge1'].fmeasure,
    'rouge2': scores['rouge2'].fmeasure,
    'rougeL': scores['rougeL'].fmeasure
40 })
return pd.DataFrame(results)

def compute_bertscore_batching(references: list[str],
    hypotheses: list[str], model_type: str = 'roberta-large',
    batch_size: int = 16) -> pd.DataFrame:
45 """
Compute BERTScore metrics for pairs of reference and
    hypothesis texts using batching.

Args:
references: List of reference texts to compare against
50 hypotheses: List of hypothesis texts to evaluate
model_type: Name of the pretrained model to use for
    BERTScore computation
batch_size: Number of text pairs to process in each batch

Returns:
55 pandas.DataFrame with columns for Precision, Recall and F1
    scores
"""
if len(references) != len(hypotheses):
raise ValueError("Длины списков 'references' и 'hypotheses'
    должны совпадать.")
scores = {"Precision": [], "Recall": [], "F1": []}
60 print(f"Using model: {model_type}")

```

```

for i in tqdm(range(0, len(references), batch_size), desc="
    Computing BERTScore", unit="batch"):
    batch_references = references[i:i + batch_size]
    batch_hypotheses = hypotheses[i:i + batch_size]
    P, R, F1 = bert_score.score(
65     batch_hypotheses,
        batch_references,
        model_type=model_type,
        verbose=False,
        device="cuda" if torch.cuda.is_available() else "cpu"
70     )
    scores["Precision"].extend(P.tolist())
    scores["Recall"].extend(R.tolist())
    scores["F1"].extend(F1.tolist())
    return pd.DataFrame(scores)
75

def compare_rouge(df1: pd.DataFrame, df2: pd.DataFrame,
    names: tuple[str, str] = ("Gold", "Experimental")) -> pd.
    DataFrame:
    """
    Compare ROUGE scores between two dataframes.
80

    Args:
    df1: First DataFrame containing ROUGE scores
    df2: Second DataFrame containing ROUGE scores
    names: Tuple of names for the two datasets being compared (
        default: ("Gold", "Experimental"))
85

    Returns:
    DataFrame with mean and median ROUGE scores for both
        datasets
    """
    required_columns = {'rouge1', 'rouge2', 'rougeL'}
    results = []
90     for df, name in zip([df1, df2], names):
        missing_columns = required_columns - set(df.columns)
        if missing_columns:
            raise ValueError(f"В датафрейме '{name}' отсутствуют столбцы
                : {', '.join(missing_columns)}")
95     metrics = {
        "Dataset": name,
        "ROUGE-1 Mean": df["rouge1"].mean(),
        "ROUGE-1 Median": df["rouge1"].median(),
        "ROUGE-2 Mean": df["rouge2"].mean(),

```

```

100     "ROUGE-2 Median": df["rouge2"].median(),
        "ROUGE-L Mean": df["rougeL"].mean(),
        "ROUGE-L Median": df["rougeL"].median(),
    }
    results.append(metrics)
105 return pd.DataFrame(results)

def compare_bertscores(df1: pd.DataFrame, df2: pd.DataFrame,
                        names: tuple[str, str] = ("Gold", "Experimental")) -> pd
                        .DataFrame:
    """
110 Compare BERTScores between two dataframes.

    Args:
    df1: First DataFrame containing BERTScores
    df2: Second DataFrame containing BERTScores
115 names: Tuple of names for the two datasets being compared (
        default: ("Gold Dataset", "Experimental Dataset"))

    Returns:
    DataFrame with mean and median Precision, Recall and F1
        scores for both datasets
    """
120 results = []
    for df, name in zip([df1, df2], names):
        required_columns = ["Precision", "Recall", "F1"]
        for col in required_columns:
            if col not in df.columns:
125 raise ValueError(f"Колонка '{col}' отсутствует в датафрейме
                '{name}'.")
        metrics = {
            "Dataset": name,
            "Precision Mean": df["Precision"].mean(),
            "Precision Median": df["Precision"].median(),
130 "Recall Mean": df["Recall"].mean(),
            "Recall Median": df["Recall"].median(),
            "F1 Mean": df["F1"].mean(),
            "F1 Median": df["F1"].median(),
        }
135 results.append(metrics)
    return pd.DataFrame(results)

```



```

def compute_bleu(references: list[str], hypotheses: list[str]
    ]) -> pd.DataFrame:
140 """
    Compute BLEU scores for pairs of reference and hypothesis
        texts.

    Args:
    references: List of reference texts to compare against
145 hypotheses: List of hypothesis texts to evaluate

    Returns:
    pandas.DataFrame with BLEU scores
    """
150 if len(references) != len(hypotheses):
    raise ValueError("Длины списков 'references' и 'hypotheses'
        должны совпадать.")

    results = []
155 print("Вычисляем BLEU для суммаризаций...")
    for reference, hypothesis in tqdm(zip(references, hypotheses
        ), total=len(references), desc="BLEU", unit="pair"):
        ref_tokens = [word_tokenize(reference)]
        hyp_tokens = word_tokenize(hypothesis)
        score = sentence_bleu(ref_tokens, hyp_tokens)
160 results.append({'bleu': score})

    return pd.DataFrame(results)

```

Содержимое /scripts

Листинг 2.1

calculate_embeddings.py

```
import sys
import os
from pathlib import Path
5 import argparse
import json
import numpy as np
from typing import Dict, Any

10 # Добавляем путь к src в PYTHONPATH
sys.path.append(os.path.join(os.path.dirname(__file__), '..',
                             , 'src'))

from dataset_loader import DatasetLoader
from embeddings import EmbeddingGenerator
15

def load_download_config(data_dir: Path) -> Dict[str, Any]:
    """Загружает конфигурацию скачивания датасета"""
    config_path = data_dir / "download_config.json"
    if not config_path.exists():
20 raise FileNotFoundError(
    f"Download configuration not found at {config_path}. "
    "Please run download_dataset.py first."
    )

25 with open(config_path, "r", encoding="utf-8") as f:
    return json.load(f)

def validate_text_column(dataset: Dict[str, Any], text_key:
    str) -> None:
    """Проверяет наличие и валидность текстовой колонки"""
30 if text_key not in dataset:
    available_columns = list(dataset.keys())
    raise ValueError(
    f"Text column '{text_key}' not found in dataset. "
    f"Available columns: {available_columns}"
35 )
```

```

def main():
    parser = argparse.ArgumentParser(description='Calculate
        embeddings for downloaded dataset')
    parser.add_argument('--data_dir', type=str, default='data',
40     help='Directory with downloaded dataset')
    parser.add_argument('--dataset_file', type=str, required=
        True,
        help='Name of the dataset file in data_dir')
    parser.add_argument('--model', type=str, default='bert-base-
        multilingual-cased',
        help='Model name from HuggingFace for embeddings')
45     parser.add_argument('--batch_size', type=int, default=16,
        help='Batch size for processing')
    parser.add_argument('--max_length', type=int, default=512,
        help='Max sequence length for tokenizer')
    parser.add_argument('--text_key', type=str, default='text',
50     help='Key for accessing text in dataset')
    parser.add_argument('--output_dir', type=str, default='
        embeddings',
        help='Directory to save embeddings')
    parser.add_argument('--device', type=str, choices=['cuda', '
        cpu'], default=None,
        help='Device for computation (cuda/cpu). Default: auto-
        detect')
55     parser.add_argument('--trust_remote_code', action='
        store_true',
        help='Allow execution of custom code from model repository')
    args = parser.parse_args()

    # Создаем необходимые директории
60     data_dir = Path(args.data_dir)
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    try:
65     # Загружаем конфигурацию датасета
    config = load_download_config(data_dir)

    # Инициализация загрузчика датасета
    dataset_loader = DatasetLoader(cache_dir=str(data_dir))
70

    # Загрузка сохраненного датасета
    print(f"Loading dataset from: {args.dataset_file}")
    dataset_dict = dataset_loader.load_local(args.dataset_file)

```

```

75  # Валидация датасета и колонки с текстом
    if 'data' not in dataset_dict or 'metadata' not in
        dataset_dict:
    raise ValueError("Invalid dataset format: missing 'data' or
        'metadata' keys")
    validate_text_column(dataset_dict['data'], args.text_key)

80  # Генерируем имя столбца для эмбедингов
    embedding_column = f"embedding"

    # Проверяем, что колонка для эмбедингов не существует
    if embedding_column in dataset_dict['data']:
85  raise ValueError(f"Column {embedding_column} already exists
        in dataset")

    # Инициализация генератора эмбедингов
    print(f"\nInitializing embedding model: {args.model}")
    embedding_generator = EmbeddingGenerator(
90  model_name=args.model,
        device=args.device,
        trust_remote_code=True
    )

    # Расчет эмбедингов
95  print("\nCalculating embeddings...")
    print(f"Text column: {args.text_key}")
    print(f"Target embedding column: {embedding_column}")
    print(f"Batch size: {args.batch_size}")
100 print(f"Max sequence length: {args.max_length}")

    # Use encode_batch instead of encode_texts
    embeddings = embedding_generator.encode_batch(
105  texts=dataset_dict['data'][args.text_key],
        batch_size=args.batch_size,
        max_length=args.max_length,
        save_path=None # We'll save embeddings ourselves
    )

110 # Сохраняем сырые эмбединги в отдельный файл
    embeddings_path = output_dir / f"{Path(args.dataset_file).
        stem}_raw_embeddings.npy"
    np.save(str(embeddings_path), embeddings)

    # Добавляем эмбединги как новую колонку
115 dataset_dict['data'][embedding_column] = embeddings.tolist()

```

```

# Обновляем метаданные
dataset_dict['metadata'].update({
    'embedding_info': {
120         'model': args.model,
        'text_column': args.text_key,
        'embedding_column': embedding_column,
        'embedding_size': embeddings.shape[1]
    }
125 })

# Сохраняем обновленный датасет
enhanced_dataset_path = output_dir / f"{Path(args.
    dataset_file).stem}_enhanced.{args.dataset_file.split
    ('.')[1]}"
dataset_loader.save_dataset(dataset_dict, str(
    enhanced_dataset_path))
130

print("\nProcessing completed successfully!")
print(f"Original columns: {[col for col in dataset_dict['
    data'].keys() if col != embedding_column]}")
print(f"Added embedding column: {embedding_column} (dim={
    embeddings.shape[1]})")
print(f"Raw embeddings saved to: {embeddings_path}")
135 print(f"Enhanced dataset saved to: {enhanced_dataset_path}")

except Exception as e:
    print(f"Error occurred while processing: {str(e)}")
    if hasattr(e, '__traceback__'):
140         import traceback
        traceback.print_exc()
        sys.exit(1)

if __name__ == "__main__":
145     main()

```

Листинг 2.2

download_dataset.py

```

import sys
import os
from pathlib import Path
5 import argparse
import json

```

```

# Добавляем путь к src в PYTHONPATH
sys.path.append(os.path.join(os.path.dirname(__file__), '..',
    , 'src'))

10 from dataset_loader import DatasetLoader

def save_download_config(args: argparse.Namespace, save_path
    : Path) -> None:
    """Сохраняет конфигурацию загрузки для последующего использо
        вания"""
15 config = {
    "dataset_name": args.dataset,
    "subset": args.subset,
    "split": args.split,
    "format": args.format
20 }

    config_path = save_path / "download_config.json"
    with open(config_path, "w", encoding="utf-8") as f:
        json.dump(config, f, indent=2, ensure_ascii=False)
25 print(f"Download configuration saved to {config_path}")

def main():
    parser = argparse.ArgumentParser(description='Download and
        save dataset from HuggingFace')
    parser.add_argument('--dataset', type=str, required=True,
30 help='Dataset name on HuggingFace (e.g., "mozilla-foundation
        /common_voice_11_0")')
    parser.add_argument('--subset', type=str, default=None,
        help='Dataset subset (e.g., "ru")')
    parser.add_argument('--split', type=str, default='train',
        help='Dataset split (train/test/validation)')
35 parser.add_argument('--format', type=str, choices=['parquet',
        , 'hdf5'],
        default='parquet', help='Format for saving dataset')
    parser.add_argument('--output_dir', type=str, default='data',
        ,
        help='Directory to save the dataset')
    args = parser.parse_args()
40

# Создаем путь для сохранения
save_path = Path(args.output_dir)
save_path.mkdir(parents=True, exist_ok=True)

45 # Инициализация загрузчика датасета

```

```

dataset_loader = DatasetLoader(
    cache_dir=str(save_path),
    save_format=args.format
)

50
try:
    # Загрузка и сохранение датасета
    print(f"Loading dataset: {args.dataset}")
    print(f"Subset: {args.subset or 'None'}")
55
    print(f"Split: {args.split}")
    print(f"Format: {args.format}")

    dataset = dataset_loader.load_from_hub(
        dataset_name=args.dataset,
60
        subset=args.subset,
        split=args.split,
        save=True,
        trust_remote_code=True # Добавляем явное указание
                                trust_remote_code
    )

65
    # Сохраняем конфигурацию загрузки
    save_download_config(args, save_path)

    # Выводим информацию о загруженном датасете
70
    print("\nDataset download completed successfully!")
    if dataset and len(dataset) > 0:
        first_key = list(dataset.keys())[0]
        print(f"Number of examples: {len(dataset[first_key])}")
        print(f"Available columns: {list(dataset.keys())}")
75
    print(f"Dataset saved to: {save_path}")
    else:
        print("Warning: Dataset appears to be empty")

    except Exception as e:
80
        print(f"Error occurred while downloading dataset: {str(e)}")
        sys.exit(1)

    if __name__ == "__main__":
        main()

```

Листинг 2.3

sample_dataset.py

import sys

```

import os
from pathlib import Path
import argparse
import random
from typing import Dict, Any

# Add src to PYTHONPATH
sys.path.append(os.path.join(os.path.dirname(__file__), '..',
                             , 'src'))

from dataset_loader import DatasetLoader

def sample_dataset(data: Dict[str, Any], sample_size: int,
                  random_seed: int = 42) -> tuple[Dict[str, Any], int]:
    """
    Create a random sample from the dataset.
    Returns:
    Tuple of (sampled data, original size)
    """
    # Get the length of any column (they should all be the same length)
    original_size = len(next(iter(data.values())))

    # Validate sample size
    if sample_size > original_size:
        print(f"Warning: Requested sample size ({sample_size}) is
              larger than dataset size ({original_size})")
        sample_size = original_size
    elif sample_size == original_size:
        print(f"Warning: Sample size equals dataset size ({
              original_size}). No sampling needed.")
    return data, original_size

    # Set random seed for reproducibility
    random.seed(random_seed)

    print(f"\nSampling {sample_size} rows from {original_size}
          total rows...")

    # Generate random indices
    indices = random.sample(range(original_size), sample_size)

    # Sample each column
    sampled_data = {}
    for key, values in data.items():

```



```

sampled_data[key] = [values[i] for i in indices]

return sampled_data, original_size
45

def main():
    parser = argparse.ArgumentParser(description='Create a
        random sample from a dataset')
    parser.add_argument('--input_file', type=str, required=True,
        help='Input dataset file name in data directory')
50    parser.add_argument('--sample_size', type=int, required=True
        ,
        help='Number of samples to select')
    parser.add_argument('--random_seed', type=int, default=42,
        help='Random seed for reproducibility')
    parser.add_argument('--data_dir', type=str, default='data',
55    help='Directory containing the dataset')
    args = parser.parse_args()

    try:
        # Initialize dataset loader
60    dataset_loader = DatasetLoader(cache_dir=args.data_dir)

        # Load the dataset
        print(f"Loading dataset from: {args.input_file}")
        dataset_dict = dataset_loader.load_local(args.input_file)
65

        # Create sample
        print(f"Creating sample of {args.sample_size} rows...")
        sampled_data, original_size = sample_dataset(
            dataset_dict['data'],
70    args.sample_size,
            args.random_seed
        )

        # Update metadata
75    dataset_dict['metadata'].update({
        'sampling_info': {
            'original_size': original_size,
            'sample_size': args.sample_size,
            'random_seed': args.random_seed,
80    'sampling_ratio': f"({args.sample_size / original_size}
            :.2%}"
        }
    })

```

```

85      # Create output filename
input_stem = Path(args.input_file).stem
output_file = f"{input_stem}_sampled_{args.sample_size}.{
    args.input_file.split('.')[-1]}"
output_path = os.path.join(args.data_dir, output_file)

90      # Save sampled dataset
dataset_dict['data'] = sampled_data
dataset_loader.save_dataset(dataset_dict, output_path)

print("\nSampling completed successfully!")
print(f"Original dataset size: {original_size:,}")
95      print(f"Sampled dataset size: {args.sample_size:,}")
print(f"Sampling ratio: {(args.sample_size / original_size)
    :.2%}")
print(f"Output saved to: {output_path}")

except Exception as e:
100      print(f"Error occurred: {str(e)}")
      sys.exit(1)

if __name__ == "__main__":
    main()

```